Vijay Baskar — AI Interview Bank

Full question bank structured Basic → Intermediate → Advanced. Red = question. Blue = simple answer + interview answer. Green = example from real work. 347 questions (77 basic, 115 intermediate, 28 error-codes, 127 advanced). Every answer is complete sentences: a simple explanation, an interview answer, and a real-work example.

How to use this book. Intern / fresher: Level 1 end-to-end, then ML + HTTP/SQL from Level 2. Mid: Level 2, plus the 25 industry questions at the start of Level 3. Senior / staff / principal: Level 3, and be ready to drop to a Level 1 SQL index or Git force-push if they probe fundamentals. Speak only known metrics: $2M indoor-nav beacon savings, sub-100ms edge inference, sub-5% trading drawdown, 70% ANR reduction, 30% app performance, 500K users / 40% faster backends. TAN Intelligence is an independent venture, not a second employer.

How this book is structured

Level 1 — Basic (77 questions). Fresher fundamentals first: Python, OOP, data structures and Big-O, HTTP/REST/JSON/JWT, SQL, Git, Linux, Java/Kotlin/Android/Flutter, tests, CI, and config. If you cannot explain a LEFT JOIN or a decorator, do not skip ahead.

Level 2 — Intermediate (115 questions). Core ML, computer vision and spatial AI, FastAPI/Spring Boot backends, mobile production (ANR, coroutines, Flutter), LLM/RAG/agents intro, trading hygiene, and cloud. This is the mid-level IC loop: ship a model, an API, and a client without pretending to be a research lab.

Error codes (28 questions). HTTP 400/401/403/404/409/422/429/500/502/503/504, OpenAI and Anthropic codes such as insufficient_quota, context_length_exceeded, rate_limit, content_filter, model_not_found, and timeout, plus 422 validation, vector-DB failures, GPU OOM, and tokenizer mismatch. Full-sentence meaning and what you would actually do.

Level 3 — Advanced (127 questions). The 25 most-asked industry questions first, then RAG at scale, agents/tools/memory, evals/safety/MLOps, serving (vLLM, KV cache, LoRA, SSE), system design, and leadership stories from ezyCart, indoor navigation, and TAN. Production GenAI, not a notebook.

Identity. This bank is for Vijay Baskar (LinkedIn vb1707). Do not mix with the Arjun Dev profile. Examples use Retailetics work (ezyCart, Lisa, vision/SLAM, ONNX/TF Lite) and Colan mobile/backend history. If you did not measure it, do not invent a dollar figure.

Level 1 — Basic (fresher fundamentals)

Python, OOP, data structures, HTTP, SQL, Git, Linux, Java/Kotlin/Flutter, tests and CI. Answer in two layers: a sentence a new grad would say, then the interview version.

Python

Q1. What is Python and why is it used in AI?

Answer (simple). Python is a readable programming language that looks close to plain English, which is why beginners can learn it quickly. It is used in AI because the main libraries for data, training, and serving already live there, so you can try an idea without switching languages. You still have to measure later, because a notebook that feels fast is not automatically a production service.

Answer (interview). Python is a dynamically typed language with a huge ecosystem around NumPy, Pandas, PyTorch, and FastAPI, which is why it became the default for AI work. The trade-off is that pure Python is slower than compiled languages on CPU-bound loops, and the GIL means only one thread runs Python bytecode at a time. In production I prototype in Python, keep serving in Python when I/O or native extensions do the heavy work, and I profile before I rewrite a hot path. At Retailetics the AI services sit in FastAPI, while ONNX and TF Lite take vision off the interpreter so we stay under 100ms.

Example. At Retailetics I serve Lisa and the trolley inference APIs from Python FastAPI, and training plus ONNX export stay in Python as well.

Example. A notebook prototype is not production until it has tests, type checks, pinned dependencies, and a packaging path into a container.

Q2. What is the difference between a list, tuple, set, and dict?

Answer (simple). A list is an ordered row of items that you can change, like cart lines you keep appending. A tuple is an ordered row that you lock after you create it, which is useful for fixed records. A set is a bag of unique values, so duplicates disappear. A dict is a labeled locker: you look up a value by a key instead of scanning the whole row.

Answer (interview). A list is ordered and mutable with amortized O(1) append. A tuple is immutable, and it is hashable only if every item inside is hashable, which is why tuples can be dict keys and lists cannot. A set stores unique items with average O(1) membership. A dict is a hash map from key to value and is the default structure for JSON-shaped data. I choose among them by whether I need order, uniqueness, mutability, or keyed lookup, and I do not use a list scan when a set or dict would be O(1).

Example. On ezyCart I cache known SKUs in a set for O(1) membership checks, and each cart is a list of dicts with sku and qty.

Example. I also put device event_ids into a set so the same scan cannot be charged twice if the trolley retries.

Q3. What is a list comprehension?

Answer (simple). A list comprehension is a compact way to build a new list by walking another collection and applying a rule. You read it as 'make this value for each item if the condition is true.' It is great for short transforms and messy when you nest several of them.

Answer (interview). The form [f(x) for x in xs if pred(x)] is eager, so it builds the whole list in memory at once. A generator expression in parentheses is lazy and yields items as you iterate, which is the better default for large streams. Nested comprehensions often hide bugs, so I switch to a plain loop when the logic has more than one filter or a side effect. In production I use comprehensions for small mappings at the API boundary and generators when I stream tokens or file rows.

Example. When Lisa reads a cart I collect SKUs with skus = [line['sku'] for line in cart if line['qty'] > 0], which drops empty lines in one readable pass.

Q4. What are *args and **kwargs?

Answer (simple). Sometimes a function should accept extra arguments without listing every name in advance. Extra unnamed values land in *args as a tuple, and extra named values land in **kwargs as a dict. That is how wrappers can forward whatever the inner function needs.

Answer (interview). *args is a tuple of leftover positional arguments and **kwargs is a dict of leftover keyword arguments. They are the standard way to write decorators, logging wrappers, and thin adapters that forward into FastAPI or Pydantic without copying every parameter. The trade-off is that a sloppy signature hides mistakes until runtime, so I still type the public API and only use *args/**kwargs at the edges. In production I keep the HTTP contract explicit and use forwarding inside helpers that should not care about every field.

Example. A timing decorator around a Lisa tool function uses *args and **kwargs so I can wrap any handler without rewriting its signature.

Q5. What is a decorator?

Answer (simple). A decorator is a wrapper that adds behavior around a function without copying the function body. You write @retry above a function and Python hands that function to the wrapper. It is a clean way to add timing, auth, or logging in one place.

Answer (interview). A decorator is syntactic sugar for f = deco(f), and functools.wraps should copy the original name and docstring so traces stay readable. Common production uses are FastAPI route registration, retries, auth checks, and timing. Closures let the decorator hold state such as a counter or a circuit breaker. The trade-off is magic: too many stacked decorators make call stacks hard to follow, so I keep them thin and test the wrapped behavior. In production I decorate inference handlers to log duration that I can roll

into p95, not to hide business logic.

Example. I time FastAPI inference handlers with a decorator that records duration so we can watch whether ONNX stays under 100ms.

Q6. What is a generator? What is yield?

Answer (simple). A generator is a function that hands you items one at a time instead of building a giant list first. The yield keyword pauses the function, remembers where it was, and continues on the next request. That is how you stream a long answer without filling memory.

Answer (interview). yield suspends the stack frame and turns the function into an iterator, so memory stays proportional to one chunk rather than the whole result. Generators are the right shape for streaming LLM tokens, reading large files, and paging database rows. send and throw exist, but I keep generators simple and treat disconnect as a StopIteration path. The trade-off versus a list is that you can walk a generator only once unless you rebuild it. In production Lisa streams SSE from a generator that yields each token and stops when the client disconnects.

Example. Lisa streams SSE tokens with a generator that yields each LLM chunk and then stops cleanly on disconnect so we do not keep paying for unused tokens.

Q7. Mutable vs immutable types?

Answer (simple). An immutable value cannot be changed in place, so any edit creates a new object, like a new string. A mutable value can be poked, like appending to a list or updating a dict. Mixing them carelessly is how shared default arguments surprise you later.

Answer (interview). Immutable types include int, str, tuple, and frozenset. Mutable types include list, dict, and set. Hashable dict keys must be immutable, which is why a list cannot be a key. Mutable default arguments such as def f(x=[]) are a classic bug because the empty list is created once and shared across calls. In production I treat request bodies as immutable after validation and I deepcopy nested agent state when I checkpoint, so a later mutation cannot leak into the previous snapshot.

Example. I never use a mutable dict as a default cache argument in a function, because that cache would be shared across every caller by accident.

Q8. What is the GIL?

Answer (simple). The GIL is Python's Global Interpreter Lock, a lock that lets only one thread run Python bytecode at a time. Think of it as a single microphone in a meeting: many people can wait, but only one talks Python at once. Waiting on the network can still overlap, which is why threads help I/O even with the GIL.

Answer (interview). The Global Interpreter Lock serializes Python bytecode, so CPU-bound Python threads do not get true multi-core speedup. I/O-bound work still benefits from threads because a thread releases the GIL while it waits on sockets or disk. For heavy CPU I use multiprocessing, a process pool, or native extensions such as ONNX Runtime and TF Lite, which release the GIL while they run compiled kernels. In production on ezyCart I do not fight the GIL for vision: inference runs in native ONNX or TF Lite under 100ms, and FastAPI uses async plus a worker pool so the event loop is not stuck in Python loops.

Example. TF Lite and ONNX inference drop into native code, so the GIL is not the cart's bottleneck. The NPU or CPU kernel is.

Q9. == vs is? shallow vs deep copy?

Answer (simple). == asks whether two values look the same, while is asks whether they are the exact same object in memory. A shallow copy makes a new outer box but still shares the inner boxes. A deep copy clones the nested boxes too, which is what you want before you mutate a snapshot.

Answer (interview). is compares identity and == compares equality, so two equal strings may or may not be the same object. copy.copy is shallow and copy.deepcopy walks nested containers. The trade-off is cost: deepcopy is safer for nested dicts and lists, but it is slower, so I do not clone huge tensors by accident. In production, agent or cart state that will be checkpointed needs a deep copy of nested lists, otherwise a later mutation leaks into the previous snapshot.

Example. When we snapshot Lisa or LangGraph-style state, mutating a nested list without a deep copy can leak changes into the previous checkpoint and make retries lie.

Q10. What are exceptions and context managers?

Answer (simple). try and except are a safety net for errors you can handle, such as a missing file or a closed socket. A context manager, used with with, always tidies up at the end, like closing a file even if you crash. Together they keep cleanup from depending on every path remembering to close things.

Answer (interview). I catch specific exceptions and let unexpected ones bubble so we get a real failure instead of a silent wrong charge. with open, with lock, and contextlib.contextmanager guarantee release of files, mutexes, and HTTP clients. Swallowing bare Exception in a checkout path is a production incident waiting to happen. In production a WebSocket disconnect is an expected exception: we cancel the LLM job and we never retry a payment from that path.

Example. On the trolley WebSocket, if the client disconnects I catch it, cancel the in-flight Lisa job, and I do not retry a charge that may already have been sent.

Q11. What is virtualenv / venv / pip / requirements vs lock files?

Answer (simple). A venv is a private library folder so one project's packages do not fight with another project's packages. pip installs into that folder, and a requirements file lists what you need. A lock file goes further and pins the exact versions that were tested.

Answer (interview). Virtualenv or venv isolates dependencies, pip installs them, and a requirements.txt is a human list while poetry.lock or uv.lock is a fully resolved graph. The trade-off is convenience versus reproducibility: unpinned installs drift, and that drift shows up as 'works on my laptop' outages. In production I pin every runtime package in the Docker image and I install the same lock in staging and prod. I never pip install an unpinned package onto a live cart service.

Example. The cart service image pins FastAPI, Pydantic, and onnxruntime so staging and production run the same versions we tested.

Q12. What is typing in Python? Optional, list[str], Protocol?

Answer (simple). Type hints are labels that tell humans and tools what a function expects, such as a string SKU or a list of names. They do not run the program by themselves, but checkers can catch many mistakes before you ship. Optional means a value may also be missing, which is None in Python.

Answer (interview). PEP 484 type hints are the contract I put on public functions, and Pydantic enforces those types at the API boundary. mypy or pyright in CI catches drift that unit tests miss. Optional[X] is X | None, list[str] is preferred over List[str] on Python 3.9+, and Protocol describes a structural interface without forcing inheritance. The trade-off is a little annotation cost for a lot of safety. In production add_to_cart(sku: str, qty: int) is rejected by Pydantic if qty is negative before the agent proceeds.

Example. Lisa's add_to_cart(sku: str, qty: int) uses Pydantic so qty=-1 never reaches the cart service.

Q13. sync vs async Python? async/await?

Answer (simple). Synchronous code waits for one thing to finish before it starts the next, like a cashier who serves one customer at a time. Async lets the same worker start other waiting work, such as network calls, without blocking the whole restaurant. You write async def and await the slow I/O.

Answer (interview). asyncio runs an event loop, and FastAPI async def is the right shape for I/O such as database calls and WebSockets. The trade-off is that CPU work inside async def stalls every other connection on that loop. In production I keep request handlers async, and I run ONNX inference in a thread or process pool so a 30ms native call cannot freeze token streaming. Mixing blocking libraries into async without a pool is a classic p95 killer.

Example. ezyCart fans device WebSockets in async FastAPI, and ONNX inference runs in a worker pool so the event loop stays free.

Q14. What is a module vs package? __init__.py? PYTHONPATH?

Answer (simple). A module is one Python file you can import. A package is a folder of modules, often with an __init__.py that marks it as a package. PYTHONPATH tells Python where to look for those folders if they are not already installed.

Answer (interview). Imports should be explicit, packages should expose a small public surface, and circular imports are a design smell that I break by moving shared types into a thin module. __init__.py can re-export, but I keep it light so importing a package is not expensive. PYTHONPATH hacks work locally and fail in Docker, so production images install the package with pip. In the Lisa stack I split retrieval, agents, and serving into packages so tests can mock tools without importing the whole service.

Example. I keep retrieval, agent tools, and FastAPI serving in separate packages so tests can mock a retriever without booting the full Lisa service.

Object-oriented programming

Q15. What is OOP? Class vs object?

Answer (simple). Object-oriented programming means you group data and the actions on that data into a class, which is the cookie cutter. An object is one cookie made from that cutter, with its own values. You talk to the object through methods instead of scattering the same logic across files.

Answer (interview). A class defines fields and methods; an object is a runtime instance. Encapsulation, abstraction, inheritance, and polymorphism are the usual pillars, but in services I prefer composition over deep inheritance because long hierarchies become brittle. The trade-off is that OOP can hide control flow if you over-abstract, so I keep class trees shallow and interfaces small. In production a Detector base with YOLODetector and RTDETRDetector sharing preprocess is enough polymorphism; I do not build a six-level vision framework.

Example. On ezyCart a Detector base class lets YOLODetector and RTDETRDetector share the same preprocess and postprocess interface while swapping the model file.

Q16. What are the four pillars of OOP?

Answer (simple). Encapsulation hides the messy internals so callers cannot break the rules. Abstraction shows a simple face, like charge() instead of every card-network detail. Inheritance reuses a parent, and polymorphism means the same action can behave differently for different types.

Answer (interview). Interviewers want examples, not slogans, so I walk through encapsulation, abstraction, inheritance, and polymorphism on a real interface. SOLID is a useful smell detector (single responsibility, open for extension, Liskov, small interfaces, depend on abstractions) but it is not a religion to gold-plate a trolley app. The trade-off of inheritance-heavy designs is fragility when a subclass needs only one method. In production I put a PaymentGateway interface in front of NFC and card implementations so checkout calls the same charge() method.

Example. Checkout talks to a PaymentGateway interface with NFC and card implementations, so the same charge() call works for both without the UI knowing the vendor.

Q17. Overloading vs overriding? Abstract class vs interface?

Answer (simple). Overriding means a child class replaces a parent method with its own version. Overloading means the same name with different arguments, which Java supports and Python usually handles with default arguments or *args. An abstract class or interface is a contract you cannot construct until someone fills in the missing methods.

Answer (interview). Java and Kotlin distinguish overloading from overriding; Python does not overload by signature in the Java sense and uses defaults, *args, or singledispatch instead. Abstract classes in Python are ABC types, and Protocol is a structural interface. Abstract means you cannot construct it and subclasses must implement the required methods. The trade-off is rigidity versus a clear contract. In production I define a CheckoutUseCase with execute() that Kotlin can override, and I never pretend a static Java method is polymorphic.

Example. Kotlin CheckoutUseCase overrides execute() for each payment rail, and I do not treat a Java static helper as a polymorphic override.

Q18. What is composition vs inheritance?

Answer (simple). Inheritance says a child is a kind of parent, like a YOLO detector is a detector. Composition says an object has a helper, like Lisa has a retriever. When you only need a helper, has-a is usually safer than is-a.

Answer (interview). Deep inheritance trees break when a subclass needs only part of the parent. Composition injects collaborators, which is easier to test and swap. The trade-off is a few more constructor arguments in exchange for a shallower, more honest design. In production I inject a Retriever into Lisa rather than making the agent a subclass of LlamaIndex, and I inject tracers and tool boxes the same way.

Example. Lisa has a retriever, a tool box, and a tracer as composed parts. It is not a subclass of LlamaIndex.

Q19. What is encapsulation and why private fields?

Answer (simple). Encapsulation means the messy bits stay inside the object so callers cannot break the rules. Private fields are the walls that stop someone from setting a cart total to zero by hand. Callers use methods that keep the data consistent.

Answer (interview). Python uses a _prefix convention and name mangling for __, while Java and Kotlin can enforce real privacy. I validate in constructors and keep derived fields computed, not stored as writable public state. The trade-off is a little more API surface for a lot fewer invariant bugs. In production the cart total is computed from lines, and callers cannot set total=0 because that would desync checkout from what was scanned.

Example. ezyCart computes cart total from line items, so a caller cannot set total=0 by hand and skip a scanned SKU.

Data structures and Big-O

Q20. What is Big-O? Best / average / worst case?

Answer (simple). Big-O is a way to guess how much work grows when the pile of data grows, ignoring constant factors. Best case is the lucky path, average is typical, and worst case is the painful path you must still survive. Memory growth matters too, not only time.

Answer (interview). Common classes are O(1), O(log n), O(n), O(n log n), and $O(n^2)$, and I state whether I mean time or memory and which case. Amortized O(1) append is not the same as worst-case O(n) when a list resizes. The trade-off of a fancy structure is often memory or implementation risk. In production I will rerank fifty recipe chunks with a cross-encoder, but I will not pretend that is fine for ten million documents, because that is O(k) model calls with a huge k.

Example. Lisa reranks about fifty retrieved recipe chunks, which is a bounded O(k) model cost. I would never rerank the whole catalogue.

Q21. Array vs linked list vs hash map vs tree vs heap?

Answer (simple). An array is a row of seats with fast index access. A linked list is a chain that is cheap to splice but slow to jump into the middle. A hash map is lockers with labels, a tree is a family tree that stays ordered, and a heap is a priority line that always exposes the next most urgent item.

Answer (interview). Arrays and Python lists are cache-friendly for scans and index access. Hash maps give expected O(1) lookup and fail under bad hash functions or huge load factors. Trees and heaps give ordered or priority operations at O(log n), and unbalanced trees degrade. In production event_id membership is a hash set, and the next-soonest retry is a heap, because those match the access pattern instead of forcing everything into a list.

Example. Processed scan event_ids live in a hash set, and the next retry time for a failed IoT message lives on a heap so we always pop the soonest job.

Q22. Stack vs queue vs deque? BFS vs DFS?

Answer (simple). A stack is last-in first-out, like a pile of plates. A queue is first-in first-out, like a ticket line. A deque lets you add or remove from both ends, which is handy when you need both behaviors.

Answer (interview). DFS walks with a stack or recursion and goes deep first. BFS walks with a queue and explores neighbors before grandchildren. Agent graphs need a visited set and a step budget the same way graph search does, or a tool loop never ends. In production I treat Lisa's agent loop like a bounded graph walk: a visited set of node plus tool signature, and a max-step cap, rather than unbounded recursion.

Example. Lisa's loop detection is a visited set of (node, tool-signature) plus a step budget, which is the same idea as not revisiting nodes in BFS or DFS.

Q23. What is a graph? Directed vs undirected?

Answer (simple). A graph is a set of dots called nodes and the links between them called edges. In a directed graph each link has an arrow, so A to B is not the same as B to A. An undirected graph is a two-way street.

Answer (interview). LangGraph-style agents are directed graphs with state on the nodes, and a DAG has no cycles. Agents often need a bounded cycle, such as retrieve then tool then answer then retrieve again, which is fine only with a cap. The trade-off of adding edges is more power and more ways to loop forever. In production Lisa is planner to retriever to tool to answer, with a loop edge capped at N steps.

Example. The Lisa graph is planner, retriever, tool, then answer, with a loop edge that is hard-capped so a bad tool result cannot spin forever.

Q24. Binary search? When does it work?

Answer (simple). Binary search guesses the middle of a sorted list, throws away the half that cannot contain the answer, and repeats. It is fast because each step halves the remaining work. It only works when the data is sorted and you can jump to an index.

Answer (interview). Binary search is O(log n) on random-access sorted data, and off-by-one errors in the low/high window are the usual bugs. It does not help an unsorted SKU list; that needs a hash index. Interpolation search is a variant when values are uniformly distributed, but I rarely need it. In production catalogues I index by SKU hash rather than binary-searching an unsorted Python list.

Example. I do not binary-search unsorted SKU lists on the trolley. I use a hash index so lookup stays O(1) as the catalogue grows.

Q25. Recursion vs iteration? What is a stack overflow?

Answer (simple). Recursion is a function calling itself until a base case stops it. Iteration is a loop that you control with a counter or a stack you own. If recursion goes too deep, the call stack overflows and the program crashes.

Answer (interview). Python does not optimize tail recursion, so a deep recursive walk will hit sys.getrecursionlimit. I prefer iteration or an explicit stack for deep trees and for any agent loop. The trade-off of recursion is prettier code versus a hidden depth bomb. In production I never recurse unbounded on tool calling; Lisa has a step counter that stops the graph.

Example. A tool-calling agent that recurses on every tool result will overflow or burn tokens. Lisa uses a step counter and an explicit graph instead.

Q26. What is hashing? Collisions?

Answer (simple). Hashing is a blender that turns a key into a locker number so you can jump there instead of scanning. Two different keys can land in the same locker, and that is a collision. A good map still works; it just chains or probes to separate those keys.

Answer (interview). Hash maps depend on a load factor and a collision strategy such as chaining or open addressing. In Java the hashCode and equals contract must stay consistent or HashMap silently breaks. Hash flooding is a security issue if untrusted keys can force many collisions. In production SKU strings hash into a dict, and I do not implement my own hash table unless I have a measured reason.

Example. Cart SKUs are hashed into a dict for lookup. A broken hashCode in a Java service would turn that map into a linked list and show up as checkout latency.

HTTP, REST, and JSON

Q27. What is HTTP? Methods GET POST PUT PATCH DELETE?

Answer (simple). HTTP is the request and response language browsers and apps use to talk to servers. GET reads, POST creates, PUT replaces the whole resource, PATCH changes a piece, and DELETE removes it. You should not use GET for something that charges a card, because GET is supposed to be a safe read.

Answer (interview). GET should be safe and not change server state. PUT and DELETE are idempotent; POST often is not unless you add an idempotency key. Status classes are 2xx success, 4xx client fault, and 5xx server fault. In production the catalogue is GET /products/{sku} and add-to-cart is POST /cart/items, and I refuse designs that charge a card on GET because crawlers and retries would double-bill.

Example. ezyCart uses GET /products/{sku} for the catalogue and POST /cart/items for scans. We never charge a card on a GET.

Q28. What is REST?

Answer (simple). REST is a style for APIs where you name resources with URLs, use HTTP verbs, and keep each call stateless. The server does not remember your previous click in hidden session magic; you send what it needs each time. It is a good fit for catalogues and a poor fit for a live video-like cart stream.

Answer (interview). Resource-oriented HTTP with JSON is what most teams mean by REST, and full HATEOAS is rare. You still need versioning, pagination, and typed error bodies. GraphQL lets the client pick fields, and RPC hides resources behind procedures. In production the product catalogue is REST, while live cart telemetry is a WebSocket, because polling REST would waste battery and add latency.

Example. The ezyCart catalogue is a REST resource API. Live trolley telemetry uses WebSockets into FastAPI and Spring Boot instead of REST polling.

Q29. What are HTTP status codes 200, 201, 204, 400, 401, 403, 404, 409, 429, 500, 503?

Answer (simple). 200 means OK, 201 means created, and 204 means success with no body. 400 means the client sent junk, 401 asks who you are, 403 says you may not, 404 means missing, 409 is a conflict, and 429 asks you to slow down. 500 means we broke, and 503 means the service is overloaded or down for a bit.

Answer (interview). 401 is unauthenticated and 403 is authenticated but not allowed, and mixing them confuses clients and security reviews. 409 is the conflict or idempotency collision, 429 is rate limit, and 502 or 504 point at a gateway. Tools and agents should not get HTTP 200 with an error payload, because they will treat it as success. In production an unknown SKU is 404, a duplicate event id returns the original result with 200 or 204, and a Bedrock outage surfaces as 503 with a fallback path.

Example. add_to_cart of an unknown SKU returns 404. A duplicate scan event_id returns 200 or 204 with the original result so retries stay idempotent.

Q30. What is idempotency?

Answer (simple). Idempotency means doing the same request twice does not double the side effect. If the trolley retries a scan, you should still add the item once. Payments need an idempotency key for the same reason.

Answer (interview). PUT is naturally idempotent; POST needs an idempotency key if it creates a charge or a cart line. Exactly-once delivery is hard across networks, so the usual design is at-least-once delivery plus an idempotent handler keyed by event_id. The trade-off is extra storage of processed keys versus duplicate charges. In production a scan event_id that we already processed is ignored, and a reused payment key returns the original receipt.

Example. If the same barcode scan arrives twice with one event_id, we process it once. A reused payment idempotency key returns the original receipt instead of charging again.

Q31. Headers, cookies, CORS, TLS?

Answer (simple). Headers are sticky notes on the request, such as who you are and what format you speak. TLS is the encrypted tunnel so those notes are not readable on the wire. CORS is the browser's guest list for which websites may call your API, and cookies are small bits of state the browser stores.

Answer (interview). Authorization Bearer tokens, Content-Type, and correlation ids belong in headers. CORS is a browser rule, not an API security control, so a mobile app or server can still call you. TLS should be everywhere, and cookies that hold sessions need HttpOnly, Secure, and SameSite. In production the Android ezyCart app uses bearer tokens, secrets never go in query strings, and Lisa does not put Bedrock keys on the phone.

Example. The ezyCart mobile app sends bearer tokens in headers. We never put secrets or member identifiers in query strings that will be logged.

Q32. What is JSON? JSON vs XML vs protobuf?

Answer (simple). JSON is nested labeled boxes of text, numbers, lists, and true or false, and it is the common language of web APIs. XML is older tagged text that is more verbose. Protobuf is compact binary with a schema, which is better when you send many small messages per second.

Answer (interview). JSON is the right default for public APIs and LLM tool calls because humans and models can read it. Protobuf or Avro is better for high-QPS internal telemetry where schema evolution and bytes matter. The trade-off is debuggability versus size and strict schemas. In production Lisa tools speak JSON, and I would consider protobuf only for high-rate IoT if JSON encoding showed up in profiles.

Example. Lisa tool calls are JSON. High-rate trolley telemetry could move to protobuf on the wire if JSON encoding ever became the measured bottleneck.

Q33. Query params vs path vs body?

Answer (simple). The path says which thing you mean, such as /products/EZY-1044. Query parameters are filters and options, such as search text or page size. The body carries the payload of a write, such as a basket JSON.

Answer (interview). GET should not rely on a body, PII should not sit in query strings because they leak into logs and referrers, and pagination should prefer a cursor over a huge offset. Path params identify a resource; query params filter it. In production search is GET /products?q=pasta and Lisa assist is POST /assist with the basket in the body so we do not log the whole cart on a query string.

Example. Catalogue search is GET /products?q=pasta. Lisa assist is POST /assist with the basket JSON in the body so cart contents are not pasted into access logs.

Q34. REST vs GraphQL vs gRPC vs WebSockets vs SSE?

Answer (simple). REST is ask and answer on resources. GraphQL lets the client pick fields. gRPC is fast binary RPC, WebSockets are a two-way phone call, and SSE is a one-way radio from server to client. You pick the one that matches the traffic shape, not the one that sounds newest.

Answer (interview). CRUD catalogues fit REST or GraphQL. Chat token streams fit SSE. Device telemetry and live cart state fit WebSockets. gRPC is strong inside a service mesh and awkward in a browser. In production Lisa streams tokens over SSE, trolley sensors use WebSockets into FastAPI and Spring Boot, and the product catalogue stays REST.

Example. Lisa streams answers over SSE. Trolley sensors use WebSockets into FastAPI and Spring Boot so scans are not polled over REST.

SQL

Q35. What is SQL? Primary key vs foreign key?

Answer (simple). SQL is the language you use to talk to tables in a database. A primary key uniquely names a row, like an order id. A foreign key points at another table's key, like an order pointing at a customer, so you cannot leave orphan rows as easily.

Answer (interview). Constraints belong in the database because application code will forget them under a bug or a race. Normalization reduces duplicate facts; denormalization copies them for read speed. In production orders.id is the primary key, orders.customer_id is a foreign key, and cart lines cannot exist without a cart because that would orphan scanned items at checkout.

Example. orders.id is a primary key and orders.customer_id is a foreign key. Cart lines are not allowed to exist without a parent cart, so we do not orphan scans.

Q36. INNER vs LEFT vs RIGHT vs FULL join?

Answer (simple). An INNER join keeps only rows that match on both sides. A LEFT join keeps every left row even when the right side has no match, filling gaps with NULL. RIGHT is the mirror, and FULL keeps unmatched rows from both sides.

Answer (interview). The common analytics bug is an accidental INNER join that drops products with no inventory row, which then looks like missing SKUs. NULLs from outer joins must be handled or aggregates lie. I default to LEFT when the left side is the entity I must still show. In production I LEFT JOIN products to inventory so a missing stock row still shows the SKU on a staff list.

Example. Staff inventory views LEFT JOIN products to stock so a SKU with no stock row still appears, instead of silently disappearing from an INNER join.

Q37. WHERE vs HAVING? GROUP BY?

Answer (simple). WHERE filters individual rows before you group them. GROUP BY folds rows that share a key into one bucket, and HAVING filters those buckets after you aggregate. That is why you cannot put SUM in WHERE the same way you put it in HAVING.

Answer (interview). A typical query is SELECT sku, SUM(qty) FROM lines GROUP BY sku HAVING SUM(qty) > 10, and columns in WHERE should be indexed if the table is large. Mixing WHERE and HAVING incorrectly is a fresher bug that produces empty dashboards. In production merchant dashboards group sales by SKU and day in SQL, not in Python loops after downloading the whole table.

Example. A merchant dashboard for top SKUs sold per day is a GROUP BY sku, day query with HAVING on the summed quantity, not a Python loop over every line.

Q38. What is an index? Clustered vs secondary? When does it hurt?

Answer (simple). An index is like the back of a book: it makes finding a value faster, but you pay extra storage and slower writes. A clustered index is the order the rows are stored; a secondary index is an extra lookup structure. Too many indexes make every checkout write update many trees.

Answer (interview). B-trees are the default, hash indexes help equality, and GIN or pgvector help arrays and embeddings. Composite index column order must match the query. EXPLAIN ANALYZE is how I prove an index helps, because guessing is how you slow writes. In production I index orders(customer_id, created_at) and I would not claim a 40% backend speedup without an EXPLAIN, which is the same discipline we used when Colan backends got about 40% faster.

Example. I add INDEX on orders(customer_id, created_at) and run EXPLAIN ANALYZE before claiming a speedup. At Colan we measured about 40% faster backends the same way, not by adding indexes blindly.

Q39. What is a transaction? ACID?

Answer (simple). A transaction is a bundle of steps that all happen or none happen, like charge and write the receipt together. ACID means Atomic, Consistent, Isolated, and Durable. If the insert fails, the charge should not stay behind.

Answer (interview). Isolation levels range from read committed to repeatable read to serializable, and each trades throughput for fewer anomalies. Deadlocks happen when two transactions grab locks in opposite orders. Distributed checkout often needs a saga because you cannot have one ACID transaction across a bank and your database. In production I charge and emit a receipt in one database transaction where I can, and I never keep a charge if the receipt insert fails.

Example. Checkout writes the receipt in the same transaction as marking the cart paid. If the insert fails, we do not keep a dangling charge record.

Q40. Normalization (1NF, 2NF, 3NF) vs denormalization?

Answer (simple). Normalization stores each fact once so you do not update it in five places. First, second, and third normal form are steps that remove repeating groups and facts that depend on the wrong key. Denormalization copies a fact on purpose for faster reads, such as a product name printed on a receipt.

Answer (interview). OLTP systems stay closer to 3NF so updates stay consistent. OLAP and receipts often denormalize because history must not change when a SKU is renamed tomorrow. Event sourcing is another way to keep history without wide mutable rows. In production the catalogue is normalized, and a receipt line stores the name as it was at purchase so historical receipts stay stable.

Example. We denormalize the catalogue name onto a receipt line so a later SKU rename cannot rewrite yesterday's shopper receipt.

Q41. SQL injection and how to prevent it?

Answer (simple). SQL injection happens when you glue user text into a query like tape, and the user types a command instead of a name. The fix is to use parameters so the database treats input as data, never as code. You also give the database user as few powers as it needs.

Answer (interview). Prepared statements, bound parameters, and ORMs that escape correctly are the baseline. Least-privilege database users limit the blast radius. LLM or MCP SQL tools are dangerous without allowlists because the model can be tricked into writing a DROP. In production sku comes from the request and we execute with a bound parameter, never an f-string.

Example. The cart lookup is execute('... WHERE sku=%s', [sku]). I never build that WHERE clause with an f-string from request text.

Q42. NULL behavior in SQL? Three-valued logic?

Answer (simple). NULL means unknown, not zero and not false. In SQL, NULL = NULL is not true, because two unknowns are not known to be equal. You check IS NULL, and you use COALESCE when you need a fallback value.

Answer (interview). Three-valued logic is true, false, and unknown, so WHERE gluten_free = FALSE will not return NULL rows. Aggregates skip NULLs, which can hide missing data. Unique indexes treating multiple NULLs as distinct depends on the engine. In production a missing gluten_free flag stays NULL, and Lisa must not say the item is not gluten-free.

Example. If gluten_free is NULL on a product, Lisa must not claim the item is not gluten-free. Unknown is not the same as false.

Q43. OFFSET pagination vs keyset/cursor?

Answer (simple). OFFSET pagination skips the first N rows and then reads a page, which gets slower as N grows because the database still walks those rows. Cursor or keyset pagination remembers the last id or timestamp and continues from there. For a huge catalogue, the cursor stays fast.

Answer (interview). OFFSET 100000 is often a full scan plus skip, which is why admin pages die at high page numbers. Keyset on (created_at, id) is stable if the sort is unique. The trade-off is that you cannot jump to page 500 as easily. In production the product admin list uses cursor pagination, not page=5000.

Example. The product admin list pages with a cursor on (created_at, id). We do not offer page=5000 with OFFSET because that would scan tens of thousands of rows.

Q44. What is N+1 query problem?

Answer (simple). N+1 means you fetch a list of N items and then run one extra query per item. Fifty cart lines become fifty extra product lookups, which is a thousand tiny trips if the cart is large. The fix is to join or to batch with IN.

Answer (interview). ORMs hide N+1 because lazy relations look like field access. I eager-load, join, or batch ids, and I trace SQL in staging so a new relation does not ship a surprise. The trade-off of a join is a wider row versus dozens of round trips. In production loading fifty cart lines then fifty product queries is a bug; we join or batch instead.

Example. Loading fifty ezyCart lines and then fifty product queries is the N+1 I watch for. We join products in one query or batch SKUs with IN.

Git

Q45. What is Git? commit, branch, merge, rebase?

Answer (simple). Git is a time machine for code that stores snapshots called commits. A branch is an alternate timeline where you try a feature without breaking the main line. Merge combines timelines, and rebase replays your commits on top of a newer base.

Answer (interview). A commit is a snapshot with a message, a branch is a pointer, merge creates a merge commit, and rebase rewrites your local commits onto another tip. I never rebase public shared history casually because it forces teammates to recover. Pull requests are the review gate. In production work I use a feature branch, merge to main after review, and I do not rewrite main.

Example. I land interview-guide work on a feature branch, merge to main after review, and then publish. I do not rebase shared main to make the history look linear.

Q46. git clone, status, add, commit, push, pull, fetch?

Answer (simple). clone copies a repository, status shows what changed, and add stages files for the next snapshot. commit saves that snapshot locally, push shares it, fetch downloads without merging, and pull fetches and integrates. Those seven commands are most of daily Git.

Answer (interview). fetch updates remote refs without touching your working tree; pull is fetch plus merge or rebase. origin is your copy's default remote, and detached HEAD means you are not on a branch. .gitignore keeps build junk and secrets out. In production I never commit .env files or model secrets, and I only pin public artifacts that are meant to be public.

Example. I never commit .env files, API keys, or model weights. Public PDFs that are meant to ship with the site are pinned on purpose.

Q47. merge conflict? how to resolve?

Answer (simple). A merge conflict means two people edited the same lines and Git will not guess. You open the markers, keep the combined truth, run tests, and then commit the resolution. Smaller pull requests make this rare.

Answer (interview). Conflicts are resolved in the working tree, not by force-pushing away someone else's work. I prefer smaller PRs and frequent merges from main to reduce conflict size. After resolving I run the relevant tests because a naive keep-both can duplicate or drop logic. In production if two prompt files conflict, I keep both eval cases rather than dropping tests to make Git happy.

Example. If two prompt files conflict, I keep both eval cases and re-run the tests. I do not delete a test file just to clear conflict markers.

Q48. What is a pull request / code review?

Answer (simple). A pull request is a request for teammates to read your change before it lands on the main branch. Review is for correctness, security, and missing tests, not only for nits. You describe why you changed it, what could break, and how you tested.

Answer (interview). A good PR states the why, the risk, and the test plan. Reviewers should look for behavioral bugs and security issues first. Nitpicking style without a linter wastes time. In production a prompt change should attach eval before-and-after scores so we do not ship a quieter but less grounded Lisa.

Example. Prompt-change pull requests for Lisa must attach eval scores before and after so a fluent regression cannot hide behind a green unit test.

Q49. HEAD, stash, tag, cherry-pick?

Answer (simple). HEAD is the commit you are standing on. stash pockets dirty files so you can switch branches, a tag names a version, and cherry-pick copies one commit onto another branch. Tags are how you mark a release you might need to roll back to.

Answer (interview). Tags should be immutable release names such as a prompt or model version. Cherry-pick is for hotfixes and is easy to duplicate if you later merge the original branch. stash is local and easy to lose, so I do not stash production fixes overnight. In production I tag a last-known-good prompt so we can roll Lisa back without hunting through the log.

Example. I tag the last-known-good Lisa prompt so a bad eval on main can roll back to a named version instead of a remembered hash.

Q50. force push risks?

Answer (simple). A force push rewrites the shared history and can erase a teammate's commit that you never pulled. It is how people lose work on a branch everyone is using. If you must rewrite a private branch, force-with-lease is the safer form.

Answer (interview). git push --force overwrites the remote tip. --force-with-lease still fails if someone else pushed, which is the minimum bar. Protected main should forbid force pushes. In production I never force-push main or a public site branch without checking, because that can drop a release or a teammate's fix.

Example. I never force-push main or the public site branch without checking who else pushed. A lease still beats a blind force.

Linux, OS, and networking

Q51. What is a process vs thread?

Answer (simple). A process is a running program with its own memory. Threads are workers inside that process that share memory, which is why they can race. Isolation is easier between processes; sharing is cheaper between threads.

Answer (interview). Processes isolate crashes and memory; threads share an address space and need locks or immutability. Python's GIL further limits CPU threads, so CPU-heavy work often wants processes anyway. Containers isolate processes, not magic. In production one FastAPI process owns the HTTP loop, a thread pool runs ONNX, and a separate process would host something like vLLM if we served a local LLM.

Example. ezyCart serving is one FastAPI process with a thread pool for ONNX. A local LLM would sit in a separate process rather than on the API loop.

Q52. stdin, stdout, stderr, exit codes?

Answer (simple). stdin is input, stdout is normal output, and stderr is the complaint channel. An exit code of zero means success, and any other number means failure to whoever started the program. Continuous integration treats a non-zero exit as a red build.

Answer (interview). In Kubernetes I log structured JSON to stdout and keep stderr for true process errors. CI fails the job on non-zero, so eval scripts must exit 1 when quality drops. Mixing human logs and machine metrics on unstructured stdout makes scraping painful. In production an eval job that sees groundedness drop exits 1 and that blocks deploy.

Example. If Lisa's groundedness eval drops, the job exits 1 so CI cannot promote that prompt. A zero exit would ship a regression.

Q53. What is DNS, IP, port, TCP vs UDP?

Answer (simple). An IP address is the building, and a port is the apartment inside it. DNS is the phone book that turns a name into that address. TCP is a reliable phone call that retries lost packets, and UDP is a postcard that might get lost.

Answer (interview). TLS usually sits on TCP. WebSockets start as HTTP and then upgrade. UDP is fine for some realtime media and a bad idea for checkout because lost packets would mean lost money. In production the cart app connects with wss to the IoT gateway on port 443 so stores do not open odd ports.

Example. The ezyCart app connects with wss to the IoT gateway on port 443. Checkout never rides UDP because a dropped packet is not acceptable.

Q54. What is a deadlock? Race condition?

Answer (simple). A deadlock is two waiters each holding what the other needs, so both freeze. A race is when the winner depends on lucky timing, such as two scans updating quantity at once. Both show up when shared state is updated without a clear owner.

Answer (interview). Fixes include lock ordering, timeouts, immutable messages, and putting updates on a single-threaded queue per cart. Optimistic versioning on a row detects lost updates without holding long locks. I reproduce with stress tests, not with hope. In production two services must not update the same cart line without a version column or a per-cart serial queue.

Example. If two services update the same cart line without optimistic versioning, a late retry can wipe a scan. We serialize per cart or version the row.

Java, Kotlin, Android, and Flutter (fresher)

Q55. Java vs Kotlin for Android?

Answer (simple). Kotlin is the modern language for Android: less boilerplate, null-safety, and coroutines. Java still appears in older modules and libraries, and Kotlin can call it. For a new Android screen I would write Kotlin unless the surrounding code is a Java island.

Answer (interview). Kotlin's null safety, data classes, and coroutines cut a class of crashes and callback nests that Java Android code accumulated. Jetpack Compose is Kotlin-first. Interop with Java is real, so a mixed module is normal. In production the ezyCart Android app is Kotlin talking to existing Java libraries rather than a rewrite for its own sake.

Example. The ezyCart Android app is Kotlin and still links existing Java libraries, which is the interop story I would give in an interview.

Q56. What is null safety in Kotlin?

Answer (simple). In Kotlin, String cannot be missing, and String? might be missing. The compiler nags you to check before you dereference. That nags away a whole class of null-pointer crashes.

Answer (interview). Safe calls ?., Elvis ?:, and explicit checks are the tools; !! is an escape hatch I avoid because it reintroduces NPEs. Java interop types are platform types and need care at the boundary. The trade-off is a little more annotation for far fewer production crashes. In production add_to_cart takes sku: String, not String?, so a missing SKU fails at compile or at the API boundary.

Example. add_to_cart uses sku: String rather than String?, so a missing SKU cannot silently become a null crash on the trolley client.

Q57. What is an Activity / Fragment / ViewModel?

Answer (simple). An Activity is a screen, a Fragment is a reusable piece of a screen, and a ViewModel holds the data that should survive rotation. If you put the cart only in the Activity, turning the phone empties it. The ViewModel outlives the view.

Answer (interview). MVVM means the UI observes a ViewModel, and the ViewModel talks to repositories, not to Android views. SavedStateHandle covers process death beyond rotation. The trade-off of stuffing logic in an Activity is untestable screens. In production the cart screen ViewModel holds line items so rotation does not empty the trolley UI.

Example. The ezyCart cart screen ViewModel holds line items. Rotating the phone does not empty the cart because the list is not stored only in the Activity.

Q58. What is Jetpack Compose vs XML views?

Answer (simple). Jetpack Compose describes UI as functions of state, so you say what the screen should look like for this cart. XML views were nested layout files that you mutated by hand. Compose redraws from state instead of hunting findViewById.

Answer (interview). Recomposition, remember, and unidirectional data flow are the Compose model, and interop with old views exists for mixed screens. XML is still in older apps and is not wrong, just more mutation-heavy. In production I would use Compose for new Android cart UI, and I would not rewrite a stable XML checkout only for fashion.

Example. Staff tools also use Flutter, which is another declarative UI. New Android cart screens can use Compose without forcing a rewrite of a stable XML checkout.

Q59. What is Flutter / Dart? Widget tree?

Answer (simple). Flutter is a toolkit that paints its own UI on iOS and Android from one Dart codebase. You build a tree of widgets, which are descriptions of UI, and Flutter draws them. That is why one staff app can run on a phone and on a TV.

Answer (interview). Widgets are immutable descriptions; state lives in State objects, BLoC, or Provider. Platform channels reach native APIs when you need hardware. Flutter is excellent for staff tools; native Kotlin may still win for the tightest camera and NFC paths. In production we ship a Flutter staff and TV monitor and a Flutter SDK lite for small vendors, while the trolley client stays closer to native where hardware is hot.

Example. We use a Flutter staff and TV monitor plus a Flutter SDK lite for small vendors, while the hardware-heavy trolley path stays native where it must.

Q60. What is an APK / Play release / ProGuard?

Answer (simple). An APK is the install package you put on a phone. Google Play is the store that ships it, with version codes and staged rollouts. ProGuard or R8 shrinks and obfuscates the code so the package is smaller and stack traces need a mapping file.

Answer (interview). Signing, versionCode, staged rollout, and mapping files for crash deobfuscation are part of a real release, not an afterthought. CI with Jenkins, GitLab, or Bitrise should build, test, and upload. The trade-off of obfuscation is smaller APKs versus harder crash reading if you lose mappings. At Colan from 2012 to 2021 we used that pipeline discipline to cut ANR about 70% and lift performance about 30% for around 500K users, and I always say how we measured ANR rather than waving at a number.

Example. At Colan, CI and release hygiene helped cut ANR about 70% and lift performance about 30% for around 500K users. I treat mapping files as part of the release, because an obfuscated crash without them is a black box.

Q61. HTTP on mobile: Retrofit/OkHttp vs Ktor?

Answer (simple). Retrofit and OkHttp are the usual Android pair for calling REST, and Ktor is another HTTP client that fits Kotlin Multiplatform. They all send HTTP; the difference is interceptors, coroutines, and sharing code. You still need timeouts and you still must not log personal data.

Answer (interview). OkHttp interceptors handle auth, retries, and logging, and I strip PII from those logs. Timeouts and backoff belong in the client, not only in the UI. Certificate pinning is for high-threat apps and has operational cost when certs rotate. In production ezyCart Android calls the backend with this stack and keeps a WebSocket client for live IoT, not a polling loop.

Example. The ezyCart Android app calls REST for catalogue and cart, and it holds a WebSocket client for live IoT scans instead of polling.

Q62. What is MVVM vs MVP vs MVC?

Answer (simple). These are ways to split the screen from the rules so one file does not become a god object. MVC puts a controller in the middle, MVP uses a presenter that talks to a view interface, and MVVM lets the view observe a ViewModel. On Android, MVVM with Flow is the common modern choice.

Answer (interview). MVVM plus LiveData or Flow keeps Android UI as a renderer of state. MVP is more explicit and more boilerplate. MVC Activities became god objects historically. I test ViewModels without an emulator. In production CartViewModel unit tests cover scan and total logic without spinning a device.

Example. CartViewModel unit tests cover adding a line and totaling the cart without an emulator. That is the point of MVVM on ezyCart.

Q63. Coroutines / Flow vs RxJava vs callbacks?

Answer (simple). Coroutines are Kotlin's readable way to write async code that still looks like a sequence. Flow is a stream of values over time, like a live list of scans. Callbacks nest, and RxJava is a powerful older stream library that many apps still have.

Answer (interview). Structured concurrency means a scope owns child jobs and cancels them together, which callbacks do not give you for free. I use Dispatchers.IO for disk and network and Main for UI updates. RxJava is fine in legacy modules; I do not mix three async styles in one screen. In production I collect device events on IO and update the cart UI on Main.

Example. Device scan events are collected on Dispatchers.IO and the cart UI updates on Main. Cancellation of the screen scope stops the collector.

Q64. What is SOLID in one sentence each?

Answer (simple). Single responsibility means one job per class. Open-closed means you extend without editing a stable core. Liskov means a subclass must not break the parent's promises, interface segregation means small interfaces, and dependency inversion means you depend on abstractions. Together they are a smell detector, not a trophy wall.

Answer (interview). I use SOLID as a review language, not as a checklist to gold-plate a trolley app. A 2000-line Activity violates single responsibility because scanning, pay, and ads are three jobs. The trade-off of over-applying SOLID is a forest of one-method types. In production I split those jobs into separate types and I keep the payment interface small.

Example. A 2000-line Activity that scans, pays, and shows ads violates single responsibility. On ezyCart those jobs are split so a checkout bug does not live inside the camera loop.

Tests, CI, security, and config

Q65. What is JSON Schema / OpenAPI?

Answer (simple). JSON Schema is a contract that says which fields exist and what types they have. OpenAPI uses that idea to document a REST API so humans and machines agree. If the contract says qty is a positive integer, a client can fail before it calls you.

Answer (interview). FastAPI generates OpenAPI from Pydantic models, so the docs and the validation stay in one place. Breaking field changes need a version, not a silent rename. Generated clients stub from the spec. Agent tool JSON schemas are the same idea as OpenAPI, just aimed at models. In production Lisa tools have JSON schemas, and humans get OpenAPI for the HTTP API.

Example. Lisa tool arguments use JSON Schema the same way the FastAPI cart API publishes OpenAPI for human clients. Both are contracts, not suggestions.

Q66. What is CI vs CD?

Answer (simple). Continuous integration means every change is built and tested automatically. Continuous delivery or deployment means a green build can be released, sometimes all the way to production. Together they stop 'it worked on my machine' from becoming the release process.

Answer (interview). A serious pipeline runs unit tests, lint, security scans, evals for AI, image builds, and a staging gate with rollback. CI without CD still helps; CD without tests is a faster way to ship bugs. In production a Lisa prompt change is blocked if eval groundedness drops, which is the AI equivalent of a red unit test.

Example. A prompt change that drops Lisa groundedness fails CI and does not deploy. That is CD gated on evals, not only on compile.

Q67. Unit vs integration vs e2e tests?

Answer (simple). A unit test checks one function with fakes around it. An integration test checks several real pieces together, such as a retriever and an index. An end-to-end test walks a user journey, which is powerful and slow, so you only keep a few.

Answer (interview). The test pyramid says many unit tests, fewer integration tests, and a handful of e2e paths. I mock LLMs in unit tests, contract-test tools against schemas, and keep a few real cart flows. In production I unit-test Pydantic tool args and integration-test retriever plus rerank against a fixture index, rather than calling a live model from every PR.

Example. I unit-test Pydantic tool arguments and integration-test retriever plus rerank with a fixture index. A full trolley e2e is reserved for a few smoke paths.

Q68. What is a race in checkout if two scans arrive?

Answer (simple). If two scans hit the cart at once, both might read qty as 1 and both write qty as 2, and you lose an item, or both might add and you double-count. A race means the result depends on timing. Checkout needs a single owner for that cart's updates.

Answer (interview). The production pattern is a per-cart lock or a serial queue plus idempotent event ids, often with an optimistic version on the cart row. At-least-once IoT delivery makes this mandatory, not optional. I would rather serialize one cart than corrupt a bill. In production the same barcode twice still increments once per unique event_id.

Example. If the same barcode arrives twice, quantity increases once per unique event_id. A second copy of that event is ignored instead of double-charging.

Q69. How do you read a stack trace?

Answer (simple). You start at the root cause, often at the bottom or in a Caused by line, and then you walk the frames that led there. Each frame is a file and line that was on the stack. On Android, obfuscated names need a mapping file before they mean anything.

Answer (interview). Python shows the latest call at the bottom; Java often has Caused by chains. I never guess from the first line of a huge Android ANR dump. Mapping.txt from R8 turns obfuscated frames back into class names. In production ANR traces that sit on main-thread I/O tell me to move that work off main, which is the same class of fix behind Colan's ANR drop.

Example. ANR traces that show disk or network on the main thread mean I move that work off main. That is the reading skill behind cutting ANR, not a mystery Play Console number.

Q70. What is UTF-8?

Answer (simple). UTF-8 is a way to store every language's characters as one or more bytes. English letters take one byte, and many other scripts take more. Character length is not the same as byte length, which matters when you slice strings.

Answer (interview). JSON defaults to UTF-8, and mixing Latin-1 with UTF-8 is how recipe names turn into garbage. I never truncate a string by raw bytes in the middle of a character. In production Tamil recipe names in Lisa must round-trip through the API, the index, and the phone UI without mojibake.

Example. Tamil recipe names in Lisa must round-trip through JSON, the retriever, and the client. If we sliced by bytes we would break characters.

Q71. Environment variables vs config files vs secret managers?

Answer (simple). Config is settings such as log level or feature flags. Secrets are passwords and API keys, and they must not be committed next to config. Environment variables are the usual way to inject both, but a secret manager is where the passwords should live.

Answer (interview). Twelve-factor config lives in the environment, Kubernetes secrets or a vault hold credentials, and each stage has its own values. Rotation matters as much as storage. In production Bedrock keys never sit in the Android app; they live on the server and come from a secret manager into the FastAPI process.

Example. Bedrock keys never ship in the ezyCart Android app. They live in a server-side secret manager and are injected into FastAPI as environment values.

Q72. What is pagination, filtering, sorting in APIs?

Answer (simple). You do not send the whole catalogue in one response. You send a page, you let the client filter, and you sort with a stable key. Unbounded queries are how a staff screen takes down the database.

Answer (interview). Stable sort keys, a max page size, and cursor pagination are the production defaults. I reject unbounded sort-plus-filter combinations that cannot use an index. Total counts are expensive; I do not compute them if the UI does not need them. In production staff stock-audit lists aisles page by page.

Example. Staff stock-audit lists aisles page by page with a max page size. We reject a request that asks for the whole store's SKUs in one shot.

Q73. What is a 12-factor app (name a few)?

Answer (simple). Twelve-factor is a set of habits for services: store config in the environment, keep processes stateless, treat logs as streams, and scale by adding processes. You do not have to recite all twelve in an interview. You should show that you build services that can restart and scale without sticky memory.

Answer (interview). I name config in env, stateless processes, backing services as attached resources, and logs to stdout, then I stop reciting. Sticky in-memory sessions fail when you add a replica. In production the cart API is stateless, session or cart state lives in Redis or the database, and logs go to stdout for the cluster to collect.

Example. The cart API processes are stateless. Cart session state lives in Redis or the database, and logs go to stdout rather than a local file.

Q74. GET caching and Cache-Control / ETag?

Answer (simple). Browsers and CDNs can reuse a GET response if you say they may. Cache-Control sets how long, and an ETag is a fingerprint so the client can ask whether the copy is still good. Personalized prices must not sit in a shared cache.

Answer (interview). If-None-Match with ETags saves bytes on unchanged catalogues. Authorization headers usually mean the response is private. Caching a member price on a CDN would leak discounts across shoppers. In production public product images are cached and membership prices are never cached shared.

Example. Public product images are cached at the CDN. A membership price is never stored in a shared GET cache because the next shopper could see the wrong number.

Q75. What is a JWT? Risks?

Answer (simple). A JWT is a signed blob that says who you are and what you may do until it expires. Anyone who steals that blob looks like you for the rest of its lifetime. That is why it must be short-lived and stored carefully on a phone.

Answer (interview). I check exp, iss, and aud, keep access tokens short, and use refresh tokens with rotation. Revocation of a JWT is hard, which is the main trade-off versus a server session. I never put JWTs in web localStorage if XSS is in play. In production a staff-app JWT cannot call shopper-only routes and gets 403.

Example. A staff-app JWT is rejected with 403 on shopper-only routes. Stealing that token still must not grant checkout-as-shopper.

Q76. Hash vs encryption vs encoding?

Answer (simple). Hashing is a one-way blender: you cannot get the original password back. Encryption is a lock with a key, so you can unlock later. Encoding such as Base64 only changes shape and is not secret, even if it looks messy.

Answer (interview). Passwords should be salted hashes such as bcrypt or argon2, PII at rest should be encrypted, and Base64 in an LLM prompt is still plaintext to the model. Mixing these words in an interview is a red flag. In production I would never Base64 a card PAN and call it encrypted in a Lisa prompt.

Example. I do not Base64 a card PAN and call it encrypted inside an LLM prompt. Encoding is not a security control on ezyCart or Lisa.

Q77. What is stdout logging vs a debugger?

Answer (simple). A debugger lets you step live through code on your machine. Logs tell the story later in production when you cannot attach a debugger. You want structured logs with a request id so you can follow one cart across services.

Answer (interview). I debug locally with a debugger and I rely on structured logs and traces in production. For LLM paths that means a tracer such as LangSmith or an equivalent, not print statements. A request_id or cart_id must follow the shopper from Android through the WebSocket into RAG spans. In production I correlate cart_id across those systems instead of SSHing into a pod to attach pdb.

Example. I correlate cart_id across the Android client, the WebSocket, and Lisa RAG spans. That is how I debug a wrong recipe in production without attaching a debugger.

Level 2 — Intermediate (core ML, CV, backend, mobile)

Machine learning, computer vision, APIs, mobile production, LLM/RAG intro, trading, cloud. Connect every answer to a system you would actually ship.

Machine learning fundamentals

Q78. Supervised vs unsupervised vs reinforcement learning?

Answer (simple). Supervised learning is like a teacher handing you the right answers, so the model maps inputs to known labels. Unsupervised learning is grouping similar things without those labels, the way you might sort a messy pantry by how items look. Reinforcement learning is trial and error with a score after each action, more like practicing a game than copying a worksheet.

Answer (interview). Supervised learning needs labelled examples such as detection boxes or sentiment tags, and that is the default for checkout vision because a wrong SKU has a clear right answer. Unsupervised methods such as clustering and embedding models are useful for exploration, retrieval, and pretraining, but you still judge them with a supervised or human eval. Reinforcement learning trains an agent with rewards, and it is expensive and unstable for checkout or payment flows, so you should prefer supervised models plus rules plus evals before you ever reward a live agent. In an interview, say which data you have, which error is costly, and why you would not RL a payment agent first.

Example. YOLO on cart cameras is supervised detection, while recipe retrieval uses embeddings judged with labelled queries. Do not put a reinforcement-learning agent on a live payment path until a supervised plus rules stack is already beating a baseline.

Q79. Train / validation / test split? Why not peek at test?

Answer (simple). Train is practice, validation is the dress rehearsal, and test is the real exam you sit once. If you keep peeking at the exam while you study, the score stops meaning you can do new work. You hide the test set so you cannot secretly memorize it.

Answer (interview). Fit parameters only on train. Use validation to choose models, thresholds, and hyperparameters. Report the test set once at the end, and if you retune after seeing test you have burned it. For time-series and trading labels you must split chronologically so the future cannot leak into the past, and when tuning is heavy you use nested cross-validation so the inner loop never sees the outer holdout. An honest split is part of the claim, not a formality.

Example. If you shuffle trading labels at random, tomorrow's close sneaks into today's features and the backtest lies. For store vision, hold out a whole store or a night-shift camera that the training set never saw, then quote that number.

Q80. Overfitting vs underfitting? Regularization?

Answer (simple). Overfitting is memorizing the homework so the model looks perfect at home and fails on a new aisle. Underfitting is using a model that is too simple to notice the real pattern. Regularization is a penalty that says do not be fancier than the data can support.

Answer (interview). You diagnose with learning curves: a large train-versus-val gap is high variance; both errors high is high bias. Regularization includes L2 and L1 weights, dropout, early stopping, data augmentation, and simply using a smaller model. Talk about the actual gap and the failure slice rather than reciting a list of tricks. For detection, a 99 percent train mAP with a collapse on night-shift cameras is overfitting to daytime lighting, not a reason to add more layers.

Example. A detector with 99 percent train mAP and about 40 percent on night-shift cameras is overfit to daytime. Regularize with night-aisle data, photometric augmentation, and early stopping, then re-check the night slice before you ship.

Q81. Bias vs variance?

Answer (simple). Bias is always missing the dartboard in the same direction, like a model that never sees small SKUs. Variance is scattered shots that change wildly when you draw a new sample. You want to know which error you actually have before you add data or add capacity.

Answer (interview). High bias usually means the model cannot represent the pattern, so you add features or capacity after you have confirmed the labels are sound. High variance means the model is too sensitive to the training draw, so you want more data, stronger regularization, or a simpler hypothesis. Error decomposition is a diagnostic you run on slices, not a slogan you recite. A linear model on SKU images underfits; a huge net on two hundred labels overfits.

Example. A linear classifier on SKU photos underfits because the decision boundary is too stiff. A huge network trained on only a few hundred labelled bottles overfits and will not survive a new store's lighting.

Q82. Precision, recall, F1, accuracy, ROC-AUC, PR-AUC?

Answer (simple). Accuracy can lie when 99 percent of frames have no product, because a model that always says empty still looks clever. Precision asks, of the alarms you raised, how many were true. Recall asks, of the true items, how many you actually found.

Answer (interview). Start from a confusion matrix and the cost of each error, not from a single headline metric. For imbalanced detection you read the precision-recall curve and an F-beta that matches the business cost, because ROC-AUC can look flattering when negatives dominate. Thresholds should come from the cost of a false charge versus a missed item, and you should quote per-class numbers for rare SKUs. F1 is a handy summary only after you have said which error hurts more.

Example. A false-positive SKU at checkout is a precision problem, and a missed item is a recall problem. We set the detector threshold from those two costs, not from the nicest mAP slide.

Q83. What is a loss function? Cross-entropy vs MSE?

Answer (simple). A loss function is a score of how wrong the model is on a batch, and training tries to shrink that score. Cross-entropy is the usual score for classification. Mean squared error is the usual score when you predict a number.

Answer (interview). Use cross-entropy for class probabilities, MSE or MAE for regression, and a composite of box, class, and objectness losses for detectors such as YOLO. The loss you optimize is not the business metric you ship on, so you must track checkout-error rate or groundedness beside the training curve. If the loss falls while the store metric stalls, you are optimizing the wrong proxy. Say that out loud in an interview so you do not sound like you train only to minimize a notebook plot.

Example. We optimize a detection loss in PyTorch, but we ship on checkout-error rate, not only mAP. A pretty loss curve that still charges the wrong oil bottle is not a win.

Q84. Gradient descent, learning rate, batch size?

Answer (simple). Gradient descent takes steps downhill on the loss surface. The learning rate is how large each step is. The batch size is how many examples you look at before you take a step.

Answer (interview). SGD and Adam are the common optimizers. A learning rate that is too large diverges and a rate that is too small crawls, and large batches usually need a scaled learning rate rather than a copied one. Mixed precision can speed the same step on GPU without changing the algorithm. When you fine-tune with LoRA you use a small learning rate on the adapter so you do not blow up the frozen base.

Example. Fine-tune a LoRA adapter with a small learning rate and leave the base weights frozen. If you copy a full-model learning rate onto that adapter you can wreck a model that was already useful.

Q85. Backpropagation in one minute?

Answer (simple). The model makes a guess, the loss says how wrong it was, and backpropagation sends that error backward so each layer knows how to change. You do not have to hand-write the derivatives. You do have to notice when the backward pass has gone insane.

Answer (interview). Backprop is the chain rule run through the computation graph, which autograd in PyTorch does for you. Vanishing and exploding gradients show up as a dead or NaN loss, and residuals, careful initialization, and gradient clipping are the usual treatments. You will not hand-derive YOLO backprop in an interview, but you must know that a NaN loss means the graph, the inputs, the learning rate, or mixed precision is broken. Debugging the backward pass is part of owning a training job.

Example. You do not hand-derive YOLO backpropagation. You do need to know that a NaN loss on a cart-camera fine-tune means stop, inspect inputs and learning rate, and do not export that checkpoint.

Q86. What is a neural network? MLP vs CNN vs transformer?

Answer (simple). A neural network is stacked layers that turn numbers into a useful prediction. An MLP is fully connected layers that see everything at once. A CNN looks at local image patches, and a transformer uses attention so distant tokens can talk.

Answer (interview). Inductive bias is the prior you bake in: CNNs assume grid locality, transformers assume that pairwise attention is worth the cost, and modern detectors such as RT-DETR mix both. MLPs are still the right default for small tabular nets, but they waste parameters on images. For a cart camera you want a CNN or transformer detector; for text RAG you want a transformer language model plus a separate embedding model. Choose the family from the data shape and the latency budget, not from fashion.

Example. The ezyCart camera path is a CNN or transformer detector such as YOLO or RT-DETR. Lisa's recipe RAG uses a transformer LLM plus an embedding model, not an MLP over raw pixels.

Q87. Embeddings for non-text (images)?

Answer (simple). An image embedding turns a photo into a point on a map so similar photos sit near each other. You can search with that map. You should not bill a shopper from a nearest neighbor alone.

Answer (interview). Contrastive training such as CLIP and metric-learning losses produce image vectors you can index. Visual search and shelf similarity are good uses; checkout charging is not, unless you also have a calibrated classifier or detector and a confidence threshold. Domain shift still applies, so a studio product shot may not sit next to a glare-filled cart frame. Version the embedding model the same way you version text embeddings, because a silent upgrade reshuffles the map.

Example. You can find similar products from a shelf photo with an image embedding index. You must not bill from embedding nearest-neighbor alone without a detector, a threshold, and a human-auditable SKU decision.

Q88. Data leakage?

Answer (simple). Leakage is letting the exam answers into the homework, so the model looks brilliant for a fake reason. It often hides in a timestamp, a customer id, or a scaler that saw the test set. The score then collapses the first day you go live.

Answer (interview). Classic leaks include the target sitting inside a feature, future timestamps, the same customer on both sides of a split, and a scaler or encoder fit on train plus test. You audit features by asking whether this number would have been knowable at prediction time. For trading, tomorrow's close in today's feature is leakage. For vision, putting frames from the same video in train and test is a milder cousin of the same bug.

Example. Using tomorrow's close as a feature for today's trading signal is leakage. Fitting StandardScaler on train plus test is leakage too, and both make a notebook look smarter than production.

Q89. Cross-validation vs walk-forward?

Answer (simple). Cross-validation rotates folds so every slice gets a turn as the exam. Walk-forward slides a window through time so you always train on the past and test on the later days. Random folds are the wrong exam when time matters.

Answer (interview). Random K-fold leaks time because a later day can train a model that is then scored on an earlier day. Trading, demand, and any labelled stream with drift need walk-forward evaluation, often with an embargo or purge gap so nearby labels cannot leak. Nested walk-forward is how you tune without peeking. In an interview, name the unit you split on, such as day, store, or customer, not just the number of folds.

Example. TAN indicators are judged with walk-forward splits, not a shuffled five-fold. A 5-fold shuffle would let future bars tutor the past and inflate the story.

Q90. Class imbalance? How to handle?

Answer (simple). If only one in a hundred frames has a rare SKU, the model can ignore that SKU and still look accurate. Accuracy then hides the miss. You have to count the rare class on purpose.

Answer (interview). Handle imbalance with more rare labels first, then class weights, resampling, focal loss, and threshold tuning on a validation slice that still has the rare class. Always report per-class recall, not only micro averages. Collecting more bananas will not teach a rare oil SKU. For checkout, a missed rare item is often more expensive than another common false negative, so the threshold is a cost decision.

Example. Rare SKUs at ezyCart checkout need extra labelled boxes, not just more bananas. We track per-class recall for those SKUs and refuse to ship on overall accuracy alone.

Q91. Feature scaling? Normalization vs standardization?

Answer (simple). Scaling puts features on a similar yardstick so one loud column does not shout over the others. Normalization usually squeezes values into a range. Standardization centers them and divides by spread.

Answer (interview). Fit the scaler on train only and persist it with the model, or you have leakage and train-serving skew. Trees such as XGBoost care less about scale; neural nets and linear models care more. Heavy-tailed features often want a quantile transform rather than a naive z-score. Leave one-hot columns as zeros and ones; do not z-score them into nonsense.

Example. For a tabular promo model, scale the numeric features with a train-only scaler and leave one-hot flags as zero or one. Dump that scaler inside the sklearn pipeline so serving cannot forget it.

Q92. XGBoost / gradient boosting vs random forest vs linear model?

Answer (simple). A linear model is a straight rule you can read. A random forest is many trees voting. Gradient boosting is trees that take turns correcting the last tree's mistakes.

Answer (interview). Start tabular work with a linear baseline, then try XGBoost when you need interactions and non-linearities. Random forests are strong and less fussy about scale, but boosting usually wins on well-tuned tabular leaderboards. Neural nets earn their keep when you have huge data or mixed modalities such as images plus text, not when you have a few thousand promo rows. In an interview, say what you would ship first and what would have to beat it.

Example. For who buys which promo, ship an XGBoost baseline before you reach for an LLM classifier. If the boosted trees already rank well, the language model has to beat that number, not just sound smarter.

Q93. What is sklearn Pipeline and why?

Answer (simple). A Pipeline is a recipe that packages preprocessing and the model so you cannot forget a step at serving time. It fits on train folds only. You save the whole object, not a loose scaler in one file and a model in another.

Answer (interview). Put imputation, scaling, and encoding inside the pipeline so cross-validation cannot leak, then joblib-dump the fitted pipeline as the unit you deploy. The alternative is a notebook that scaled on all rows and a production job that scaled differently. Pipelines also make it obvious which columns the model expects. That discipline matters more than the brand of the estimator.

Example. Merchant forecasting can be one pipeline of imputer, scaler, then XGBoost. Serving loads that single object so the live features take the same path the validation fold took.

Q94. Calibration of probabilities?

Answer (simple). A 70 percent score should be right about 70 percent of the time if you want to treat it as a probability. Raw model scores are often more like ranks than chances. Calibration is the repair job that checks that claim.

Answer (interview). You inspect reliability diagrams and, when needed, fit Platt scaling or isotonic regression on a held-out set. Detection confidence from YOLO is not automatically a probability, and a 0.9 box score is not a 90 percent chance until you measure it on store data. Downstream systems that charge, abstain, or page a human need calibrated scores. Recalibrate when the camera, the store, or the class mix shifts.

Example. Do not treat a YOLO 0.9 as a 90 percent chance without calibration on store cameras, and if the night-shift curve is off, gate checkout on a store-specific threshold.

Q95. What is batch vs online vs mini-batch learning?

Answer (simple). Batch learning looks at all the data before it takes a step. Online learning looks at one example at a time. Mini-batch is the usual middle path that trains on a small group each step.

Answer (interview). Most production computer vision is mini-batch training, offline or batch evaluation, and online inference per frame. True online learning in production is rare because you need drift detection, label delay handling, and a rollback. Periodic retrain on newly labelled frames is the practical pattern. Say which of those three you are doing, because interviewers use the words loosely.

Example. We retrain nightly on newly labelled cart frames, which is periodic mini-batch learning, not true online learning. Inference still runs per camera frame with a frozen exported model.

Q96. Train/serving skew?

Answer (simple). Skew is when the model trained on clean tables but production feeds it messy live data. The notebook metric then has nothing to do with the camera. You fix it by making train and serve share the same feature code.

Answer (interview). Use the same preprocess path, log the served features so you can replay them, and put schema checks at the boundary. Resolution, color format, normalization, and letterbox settings are common vision skews. For tabular models, a different missing-value fill or timezone is enough to wreck a score. If you cannot replay a production request through the training preprocess, you do not actually know why the model failed.

Example. Training used RGB images at 640 pixels while the device sent NV12 at 320 pixels, and mAP collapsed. Matching the preprocess, including color convert and letterbox, restored the number.

Q97. What is a baseline and why start there?

Answer (simple). A baseline is a dumb simple method you must beat before you are allowed to ship something clever. If you cannot beat it, the clever thing is not earning its complexity. Interviewers want to hear what you compared against.

Answer (interview). Useful baselines include majority class, last-value forecast, BM25-only retrieval, and a keyword FAQ. You freeze the eval set, measure the baseline, then only keep the new system if groundedness, latency, or error rate actually moves. Complexity that does not beat the baseline is a hobby. This is also how you defend RAG or an LLM classifier against a cheaper lookup.

Example. Measure a BM25-only shopping FAQ against full recipe RAG on the same labelled questions. If groundedness does not lift, do not ship the extra retrieval stack.

Q98. PyTorch: tensor, autograd, nn.Module, DataLoader?

Answer (simple). A tensor is a grid of numbers the GPU can chew. Autograd tracks how those numbers should change when the loss moves. An nn.Module is the model, and a DataLoader feeds it batches.

Answer (interview). You put tensors on cpu or cuda, wrap training in a Module so parameters are registered, and use a DataLoader for shuffled batched input. For inference you want eval mode and no_grad or inference_mode so dropout, batch-norm, and autograd behave. Automatic mixed precision is a training-speed tool, not a substitute for a correct forward. Export to ONNX with a dummy tensor that matches production shape, or the runtime will surprise you.

Example. Export YOLO to ONNX with a dummy tensor of the production image size. If you export at 640 and the cart sends 320, you have built train-serving skew into the file.

Q99. eval() vs no_grad() vs inference_mode()?

Answer (simple). eval() changes layers such as dropout and batch-norm so they behave as they do at test time. no_grad() stops tracking gradients so you do not waste memory. inference_mode() is a stricter PyTorch inference context that also blocks view-tracking surprises.

Answer (interview). You usually want both eval() and a no-grad context at serve time in PyTorch. Forgetting eval() leaves dropout on and makes checkout detections flicker. no_grad() alone does not switch batch-norm to running stats. An ONNX Runtime session is inherently inference, so this pair is about the PyTorch prototype you use before export.

Example. If a PyTorch prototype forgets eval(), dropout stays on and cart boxes jitter frame to frame. The ONNX Runtime session we ship is inherently inference, but the prototype that produced that file must have been in eval mode.

Q100. ONNX export pitfalls?

Answer (simple). Not every PyTorch operator exists in ONNX, so export can succeed and still be wrong. Dynamic shapes and non-maximum suppression are the usual pain. You must check numerical parity on a golden image, not only that the file was written.

Answer (interview). Pin the opset, simplify the graph when you can, and test box or logit parity against PyTorch before you hand the file to TensorRT or TF Lite. Custom NMS plugins are a frequent TensorRT break. Keep a golden clip or image in CI so a later export cannot silently drift. Dynamic batch and dynamic spatial size are features you enable on purpose, not defaults you assume.

Example. A custom NMS plugin broke TensorRT on the kiosk path. We compared boxes on a fixture clip against PyTorch and only then accepted the engine.

Q101. JAX: grad, jit, vmap in one line each?

Answer (simple). grad takes derivatives of a Python function. jit compiles that function so later calls run as compiled XLA. vmap runs the same function across a batch without you writing the batch loop.

Answer (interview). JAX is functional and XLA-backed, which is a different habit from PyTorch's imperative modules and autograd tapes. You choose it when the team and the serving stack already live there, not because the three words sound smart. For cart vision a PyTorch-native detector with an ONNX export is the boring path that ships. Rewriting YOLO in JAX for fashion is a schedule risk.

Example. If the team is PyTorch-native for computer vision, do not rewrite YOLO in JAX to look modern. Export the PyTorch model to ONNX and spend the time on store eval.

Q102. TensorFlow vs PyTorch vs JAX — how do you choose?

Answer (simple). Think of them as three workshops. You pick the workshop that already has the model, the deploy path, and the people who can debug it at 2 a.m. Switching later is possible if you export, but it is never free.

Answer (interview). PyTorch currently owns most detection research and our YOLO path. TensorFlow still matters when TF Lite on Android is the constraint. JAX matters when you want XLA and a functional stack. The reversible choice is to train where the team is strong and export ONNX when the operator set allows it, then pick TensorRT or TF Lite from the device. Do not start from a framework war; start from the runtime you must hit.

Example. Detection trains in PyTorch, exports to ONNX, and runs TF Lite on the Android cart. That path is a choice about devices and people, not a claim that one framework is universally better.

Q103. What is transfer learning?

Answer (simple). Transfer learning starts from a model that already saw many pictures or words, then teaches it your smaller dataset. You are borrowing a head start. You still have to check that the new domain did not break the borrowed features.

Answer (interview). The usual recipe is freeze the backbone, train a new head, then unfreeze lightly with a small learning rate. The domain gap matters: a supermarket cart is not COCO, and a recipe question is not Wikipedia. If the gap is large you need more labelled store data, not only a fancier pretrained checkpoint. Transfer learning is how we fine-tune YOLO on store SKUs rather than training from scratch.

Example. Fine-tune YOLO on labelled ezyCart SKUs rather than training a detector from random weights. Freeze first, then unfreeze the backbone gently once the head is stable.

Q104. What is data augmentation for images? Risks?

Answer (simple). Augmentation makes extra training pictures by flipping, cropping, or changing lighting so the model sees more variety. You only do this on the training set. If you overdo it you can destroy the very clues the model needs.

Answer (interview). Photometric augments such as brightness and mild color jitter help with store lighting; geometric augments such as scale and translation help with camera pose. Do not flip text or OCR, do not recolor SKUs until oil and sauce look interchangeable, and do not pretend augmentation replaces real night-aisle captures. Apply augmentation only on train, and keep a clean validation set that looks like production. Targeted real data for glare and occlusion beats cartoonish transforms.

Example. Do not color-jitter so hard that SKU colors become unrecognizable. Keep real night-aisle frames in the set, because no amount of synthetic brightness will fully replace them.

Q105. Label noise and how it shows up?

Answer (simple). Wrong boxes teach the model wrong habits, the way a misspelled flashcard teaches the wrong word. The loss can still fall. Production then disputes the same SKU the annotators already confused.

Answer (interview). You find label noise in per-class confusion, in annotator disagreement, and in a model that confidently repeats a systematic misbox. Treatments include consensus labelling, QA sampling, and losses that are less eager to overfit a wrong box, but the first fix is the labels. A high mAP on noisy labels is not a store win. Checkout disputes are often cheaper to prevent in the labelling tool than in a later model tweak.

Example. If annotators misbox oil as sauce, the detector will cause checkout disputes on those bottles. Fix the labels and the guidelines first, then retrain, rather than stacking more augmentations on the same mistake.

Q106. mAP, IoU, NMS for detection?

Answer (simple). IoU is how much two boxes overlap. NMS throws away extra boxes that are sitting on the same object. mAP summarizes precision across classes and overlap thresholds so you can compare detectors.

Answer (interview). People quote mAP at IoU 0.5 or a range up to 0.75, and those protocols are not interchangeable. Soft-NMS and transformer detectors such as RT-DETR change how duplicates are suppressed, and RT-DETR aims to avoid classic NMS in the core. High mAP can still fail on glare and occlusion, so you also need a checkout-oriented metric such as wrong-charge rate. Two boxes on one bottle should collapse to one after NMS; if they do not, you will double count.

Example. Two boxes on one bottle should become one after NMS, and we report mAP at 0.5 together with a checkout-oriented error rate because a pretty mAP can still charge the shopper twice.

Q107. Classification vs detection vs segmentation vs tracking vs OCR?

Answer (simple). Classification says what is in the picture. Detection says what and where with a box. Segmentation paints the pixels, tracking keeps the same object over time, and OCR reads the text.

Answer (interview). Checkout vision is usually detection plus tracking so a banana is not counted in every frame as a new item. Segmentation is heavier and useful when you need a mask, such as spill or hand-over-item cases. OCR is for shelf labels, price tags, and packed text, not for recognizing a bottle by its shape. Pick the task from the decision you must make, not from the fanciest head you can attach.

Example. Use OCR for shelf labels and detection for items in the cart camera. Add tracking on the tray so the same banana is not billed thirty times as the shopper holds it still.

Computer vision and spatial AI

Q108. What is computer vision in production?

Answer (simple). Production vision is cameras plus models plus hardware plus monitoring, not a notebook demo that looked good on one clip. You have to say what happens when confidence is low. You also have to keep labelling new failures.

Answer (interview). Define the task, the metric, the lighting and occlusion domain, the latency budget, the privacy rule, and the fallback when the model abstains. Then build a dataset flywheel so production errors become labels. ezyCart self-checkout vision plus kiosk is this loop, not a single YOLO checkpoint. Fail closed on low confidence so a confused frame does not invent a SKU. Hardware thermal limits and model updates are part of the same design.

Example. ezyCart self-checkout vision plus the kiosk must fail closed on low confidence. A pretty demo clip is not production until lighting slices, latency, and a human fallback are specified.

Q109. YOLO family (v8/v10/v11) — what to say?

Answer (simple). YOLO is a family of fast one-shot detectors that look at the image once and predict boxes. Bigger versions are usually more accurate and slower. You pick a size the same way you pick an engine for a trolley, not from a blog's COCO chart.

Answer (interview). Choose nano or small for a cart NPU and a larger variant for a back-office GPU, then benchmark on your own video, not on COCO screenshots. Talk about license, export path to ONNX or TF Lite, and how NMS is handled in the version you picked. v8, v10, and v11 differ in architecture details, but the interview is about device budget, eval set, and operations. Keep one eval set when you swap family members so the comparison is fair.

Example. On the cart NPU we try nano or small YOLO; on a back-office GPU we can afford a larger weight. Both are scored on the same store clips so a COCO blog post cannot pick the winner.

Q110. What is RT-DETR?

Answer (simple). RT-DETR is a real-time detection transformer. It is still a detector that outputs boxes and classes, just with different guts than YOLO. You compare it on your device, not on a paper's headline.

Answer (interview). It uses a set-prediction style and typically avoids classic NMS in the core, which is the talking point interviewers expect. You still compare mAP, small-object recall, milliseconds, and memory on the same image size and precision as YOLO. Transformer detectors can be kinder to crowded shelves and harsher on a tiny NPU. Bake them off with identical preprocessing so you are not comparing two different letterboxes.

Example. Bake off YOLO versus RT-DETR on aisle clips with identical image size and precision. Keep the winner only if store mAP and sub-100ms latency both hold on the target device.

Q111. OpenCV's role if you have YOLO?

Answer (simple). OpenCV is the Swiss Army knife around the network: resize, undistort, draw, and classic vision. YOLO does not replace those chores. You still need honest pixels before the detector sees them.

Answer (interview). Run camera calibration, undistort, color convert, and letterbox in OpenCV or an equivalent, then feed the tensor to ONNX or TF Lite. Do not rewrite undistort in PyTorch on a device just to look neural. Postprocess drawing and simple rules also live here. The network is one stage in a pipeline whose other stages are still computer vision.

Example. Undistort the cart camera with OpenCV, then run the ONNX detector. If you skip undistort, every downstream box is in a bent geometry even when YOLO is right.

Q112. Camera calibration, distortion, extrinsics?

Answer (simple). Lenses bend pictures, especially at the edges of a cart camera. Calibration measures that bend so geometry is honest. Extrinsics say how the camera sits relative to the robot, the IMU, or the store.

Answer (interview). Intrinsics describe focal length and distortion inside one camera. Extrinsics describe pose between cameras, or between a camera and an IMU, which visual-inertial odometry needs. Bad calibration becomes SLAM drift, wrong aisle distances, and product-finder paths that miss the bay. Time sync belongs in the same conversation, because a beautiful extrinsics matrix with a 40 millisecond IMU lag still drifts.

Example. Kimera-VIO needs camera-IMU calibration and clock sync. If those are sloppy, indoor navigation drift is a calibration problem long before it is a model problem.

Q113. What is SLAM?

Answer (simple). SLAM means you build a map while you also figure out where you are. It is simultaneous localization and mapping, not a single neural net. It fails in boring ways such as blank walls and moving people.

Answer (interview). A typical stack has a front end that tracks features, a back end that optimizes poses, and loop closure that recognizes a place you have seen before. Failure modes include textureless walls, crowds, and lighting changes, so you need an aisle-level error budget and a fallback. Indoor navigation that replaced thousands of Bluetooth beacons and saved over 2 million dollars is still a SLAM-plus-operations story, not a claim that drift vanished. Explain the error and the failure modes in the same breath as the savings.

Example. Indoor navigation replaced thousands of Bluetooth beacons and saved over 2 million dollars. In an interview I still explain aisle-level error, textureless aisles, and what the trolley does when tracking drops.

Q114. VIO vs VO vs wheel odometry?

Answer (simple). Visual odometry estimates motion from the camera alone. Visual-inertial odometry adds an IMU so blur and fast turns have another vote. Wheel odometry counts wheel turns and will lie when the wheels slip.

Answer (interview). These sensors are complementary, and they only fuse well if clocks are synced and IMU biases are estimated. Smooth supermarket tiles are a classic wheel-slip surface, while a camera can go blind in a motion-blurred turn where the IMU still sees acceleration. VIO is the usual fusion for a trolley. Do not quote wheel odometry as ground truth on wet floors.

Example. On smooth store tiles the wheels may slip while the camera blurs in a fast turn. VIO lets the IMU carry the pose through that moment, which is why the trolley uses visual-inertial fusion rather than wheels alone.

Q115. Kimera-VIO — what is it and what must you not overclaim?

Answer (simple). Kimera-VIO is open-source visual-inertial odometry in the Kimera family. It estimates pose from a camera plus an IMU. It is not a full 'SLAM is solved' claim by itself.

Answer (interview). Know the inputs, which may be mono or stereo plus IMU, the pose output, the compute budget, and the assumption that calibration and time sync are good. Validate drift on real store routes with an aisle-level error, not a hallway demo. Loop closure and a store map are extra systems; running VIO is not the same as a complete SLAM product. Do not tell an interviewer that indoor navigation is finished because Kimera-VIO started.

Example. We use Kimera-VIO as the motion estimate that feeds the product-finder path. That is a pose stream, not a claim that store-scale SLAM is a solved research problem.

Q116. Spatial AI?

Answer (simple). Spatial AI is not only saying this is a chair. It is saying the chair is two meters to the left in a frame the trolley understands. Geometry and localization sit next to perception.

Answer (interview). You combine perception, geometry, localization, and sometimes planning, and you keep coordinate frames honest among camera, IMU, map, and store. A SKU in the map frame and a trolley in the map frame let you plan a path between them. If those frames drift, the product finder sends people to the wrong bay. Spatial AI is a systems word; in an interview, name the frames and the error you tolerate.

Example. The product finder puts a SKU in the store map frame and the trolley in that same frame, then plans a path between them. If VIO drift grows, that path is wrong even when the SKU classifier is right.

Q117. Edge vs cloud inference for vision?

Answer (simple). Edge inference runs on the device in your hand or on the trolley. Cloud inference runs on a faraway GPU. Edge is faster and still works if Wi-Fi dies; cloud can host heavier models.

Answer (interview). Choose using privacy, bandwidth, thermal limits, update mechanics, and a fallback when the network or the accelerator fails. A hybrid is common: a tiny on-device model for the live path, and a cloud model for hard cases you can afford to delay. Streaming every cart frame to a hosted LLM or Bedrock is the wrong default because of cost, privacy, and latency. Sub-100ms ONNX or TF Lite on the cart is the live checkout path.

Example. Run ONNX or TF Lite on the cart for sub-100ms detections. Do not stream every camera frame to Bedrock; that blows privacy, bandwidth, and the latency budget.

Q118. TensorRT vs TF Lite vs ONNX Runtime?

Answer (simple). ONNX is the suitcase you pack the model into. TensorRT is a fast unpacker on NVIDIA. TF Lite is the unpacker you take to phones and many edge chips.

Answer (interview). The real differences are operator coverage, quantization tools, and how you profile them on the target device. After every export you run golden tests so a missing op did not silently change boxes. Android carts typically want TF Lite or ONNX Runtime; NVIDIA kiosks can take TensorRT. Pick from the hardware you already have, then verify milliseconds and numerical parity, not from a vendor slide.

Example. The Android cart path uses TF Lite or ONNX Runtime, while an NVIDIA kiosk can use TensorRT, and both must pass the same golden-image box comparison after export.

Q119. How do you get sub-100ms vision?

Answer (simple). You time every stage, shrink the slowest one, then recheck quality. The resume number must say whether it is end-to-end or model-only. A fast network behind a slow resize is still a slow trolley.

Answer (interview). Profile resolution, quantization, batch size of one versus larger batches, CPU affinity, and memcpy. End-to-end latency includes camera, color convert, letterbox, infer, NMS, and tracking, and that is the number a shopper feels. Quantizing or switching YOLO size only helps after you know which stage dominates. Sub-100ms ONNX or TF Lite is a measured pipeline, not a model-card claim.

Example. If resize is 40 milliseconds, infer is 30, and NMS is 5, you fix resize first. That is how a cart pipeline gets under 100 milliseconds without magically shrinking the network.

Q120. Tracking IDs across frames?

Answer (simple). Tracking gives the same banana the same name as it moves so you do not count it thirty times. Detection alone is forgetful. Checkout needs memory across frames.

Answer (interview). ByteTrack and SORT-style trackers combine IoU with optional appearance features, and they must handle occlusion when a hand covers the item. You reset identities when the cart or tray is empty so stale IDs cannot resurrect. Scan-bag-pay flows track objects on the tray, not independent per-frame detections. If tracking breaks, you get double charges even when the detector is fine.

Example. In scan-bag-pay we track objects on the tray rather than treating every frame as a new set of SKUs. A still banana keeps one ID until the tray is empty.

Q121. Lighting, glare, occlusion — domain shift?

Answer (simple). A model trained at noon can fail at night or under a window reflection. That is domain shift, not a random bug. You measure each lighting world on its own.

Answer (interview). Split eval by scenario: store LED, window glare, night aisle, and heavy occlusion. Collect targeted data for the failing slice, use auto-exposure carefully, and gate on confidence when the frame is from a world you do not cover. One blended mAP hides a glare disaster. Production vision lives or dies on those slices.

Example. We report separate metrics for store LED versus window-glare sets rather than one blended mAP. If glare recall drops, we collect that slice instead of training longer on easy noon frames.

Q122. Privacy for cameras in a store?

Answer (simple). Do not keep faces if you do not need them, and say clearly what you store. A cart camera is not a license to build a face database. Shoppers should be able to hear the retention rule in one sentence.

Answer (interview). Minimize: process on device when you can, drop frames that are not needed for the SKU event, blur or avoid storing faces, set retention, and log access. Regulation and retailer policy sit beside the model card. Checkout vision can keep the SKU event without keeping a shopper's face. If a hosted model would see raw video, that is a privacy decision, not only a latency decision.

Example. Checkout vision can drop frames that are not needed for the SKU event and avoid storing faces. That is a product rule, not an afterthought once the detector works.

Backend, APIs, and DevOps

Q123. Why FastAPI for AI services?

Answer (simple). FastAPI is a Python API framework with type checks and auto-generated docs, sitting next to the same language as PyTorch and ONNX. That is why AI services like it. It is not automatically the system of record for a whole retailer.

Answer (interview). You get Pydantic validation, async handlers, OpenAPI, and dependency injection, which are ideal for vision and RAG orchestration. CPU or GPU inference must be offloaded from the event loop, and you still need timeouts, auth, and tracing. At Retailetics the Python FastAPI services sit beside a Java Spring Boot estate behind stable contracts. Choose FastAPI where the ML libraries live; do not rewrite every business ledger into Python for fashion.

Example. Vision and recipe RAG orchestration run in FastAPI. The broader Java estate stays on Spring Boot, and the two talk through versioned contracts rather than one language winning.

Q124. Pydantic / bean validation?

Answer (simple). Validation checks the form before you accept it, whether that form is JSON from an LLM or an HTTP body. Garbage in is how checkout invents a negative quantity. You reject at the boundary, not deep in the agent.

Answer (interview). Pydantic on FastAPI and bean validation on Spring Boot are the same idea: typed schemas, useful error messages, and versioned contracts. LLM tool JSON is untrusted input and must pass the same validation as a public HTTP body. Invalid payloads fail closed with a 4xx, not with a guessed default that charges a shopper. Schema evolution needs compatibility rules so an old Android app does not crash on a new field.

Example. LLM tool JSON and HTTP bodies both go through Pydantic. A model that emits qty=-1 is rejected at the boundary before add-to-cart can run.

Q125. Spring Boot in one minute?

Answer (simple). Spring Boot is the Java way to build services with batteries included: web, security, config, and actuators. It is how a large retailer already runs. Python FastAPI does not have to replace it.

Answer (interview). You coexist with FastAPI behind stable contracts, and you care about thread pools, backpressure, and observability the same way you would in any production JVM service. IoT WebSocket gateways, auth, and device identity often live here because the Android estate already speaks that stack. Observability actuators and mature ops are the reason it stays. In an interview, describe the boundary, not a holy war between Java and Python.

Example. The IoT WebSocket gateway in Spring Boot talks to the Android app, while FastAPI handles vision and RAG. Duplicate scans must still be idempotent across that boundary.

Q126. Microservices vs modular monolith?

Answer (simple). Microservices are many small programs talking over the network. A modular monolith is one program with clear internal walls. You split only when a boundary pays rent in scaling or ownership.

Answer (interview). Every extra service buys you independent deploys and a bill of network failure, distributed data, and versioning. Start modular in one repo with hard module boundaries, then extract when a workload such as vision inference scales differently or a team needs a separate lifecycle. Do not split add-to-cart into a dozen services. Do split GPU inference if its autoscaling and failure modes would poison the checkout API.

Example. Do not split add-to-cart into twelve services. Do split vision inference when it needs its own GPU pool, timeouts, and deploy cadence from the REST catalogue API.

Q127. How do WebSockets stay reliable for IoT?

Answer (simple). You keep the call up, number the messages, retry without doubling the work, and notice when the other side has died. A trolley that reconnects must not charge twice. Heartbeats are how you learn the cable is dead.

Answer (interview). Device identity, a versioned schema, timestamps or sequence numbers, heartbeats, backoff, acknowledgements, idempotency keys, and replay on reconnect are the checklist, and each item is a sentence in the design, not a buzzword. Restore state from a source of truth after reconnect rather than trusting the in-memory socket. At-least-once delivery is the default, so consumers must be idempotent. FastAPI and Spring Boot sockets in the ezyCart path have to agree on that contract.

Example. ezyCart devices talk to Android through FastAPI and Spring Boot WebSockets. Duplicate scan events must not double-charge, so every charge carries an idempotency key that survives reconnect.

Q128. REST vs WebSocket vs SSE in the trolley stack?

Answer (simple). REST is for asking or changing a thing and getting an answer. WebSockets are for live device chatter both ways. Server-sent events are for a one-way token stream to the UI.

Answer (interview). Do not multiplex every concern onto one socket unless you have a reason, because backpressure, auth refresh, and retries differ. SKU lookup is a REST call. IMU and scan events are WebSocket messages. Lisa's tokens are SSE so a tablet can render words as they arrive and cancel when the shopper walks away. Mixing those without queues is how you drop IMU samples while an LLM prompt blocks.

Example. SKU lookup stays REST, IMU and scan events stay on WebSockets, and Lisa tokens stream over SSE. That split keeps a slow recipe answer from stalling live device events.

Q129. How do you debug a slow API?

Answer (simple). Measure first, find the slow station, change one thing, and measure again. Guessing is how you cache the wrong layer. Percentiles tell you who is actually waiting.

Answer (interview). Look at p50, p95, and p99, then traces, SQL EXPLAIN, N+1 queries, payload size, and downstream timeouts. Indexes, query rewrites, and caches are different medicines. The Colan work that made responses about 40 percent faster is a backend story only if you can say whether indexing, caching, or a query rewrite moved the needle. Never quote an average alone while the tail is on fire.

Example. When responses got about 40 percent faster in the Colan-era backend work, the honest interview answer is which measurement moved: indexing, caching, or query rewrite. I would put the trace beside that number.

Q130. Cache invalidation basics?

Answer (simple). A cache is a sticky note of the answer so you do not redo the work. When the truth changes you must throw the note away. That throw is the hard part.

Answer (interview). Use TTLs for naturally aging data, explicit purge when a catalogue updates, and stampede protection so a miss does not melt the origin. Never cache ACL-sensitive personalized prices without the shopper or tenant in the key. Semantic caches for RAG need the same discipline plus a content-hash of the retrieved sources. A cache without an invalidation story is a correctness bug with extra latency wins.

Example. Recipe RAG can use a semantic cache keyed by tenant plus basket hash, and it must miss when the catalogue or the policy chunk changes. Personalized member prices never share that key.

Q131. Message queues vs request/response?

Answer (simple). A queue is a waiting line for work that can happen in a second, not in this HTTP request. Request and response is for the shopper who is still holding the phone. You pick based on whether the caller can wait.

Answer (interview). Systems such as Celery, SQS, or Kafka give you at-least-once delivery, so consumers must be idempotent and you need backpressure and poison-message handling. Embedding ten thousand new products belongs on a queue; the HTTP handler only enqueues. Request and response stays for lookups with a tight SLO. Mixing those without a timeout is how a catalogue re-embed blocks checkout.

Example. A POST to re-embed new products only enqueues a Celery job. The HTTP request does not wait for ten thousand vectors, and the worker is idempotent on product id.

Q132. Celery / background jobs?

Answer (simple). Background jobs do slow homework in the back room so the counter stays fast. They retry when the world is flaky. They must not retry a charge that already succeeded.

Answer (interview). You design retries, acknowledgements, poison queues, idempotency keys, and visibility timeouts as if duplicates will happen, because they will. Nightly re-embeds, report generation, and video export belong here. Anything with money needs a stronger exactly-once story than default Celery retries. Observability of queue depth is part of the API's health.

Example. A nightly job re-embeds changed catalogue pages. If it runs twice, the same product id writes the same vector, so the retry cannot create a second truth.

Q133. GraphQL when, not when?

Answer (simple). GraphQL lets a client ask for exactly the fields it wants. The risk is that the server now does a surprise amount of work. It is not a replacement for a live event stream.

Answer (interview). You need query cost analysis, persisted queries, and an N+1 defense, usually DataLoader-style batching. Staff apps that stitch product details from many resources can benefit. Live IMU samples and scan events do not belong on GraphQL subscriptions if you already have a WebSocket schema with backpressure. Treat GraphQL as an API style with a denial-of-service surface, not as a magic bus.

Example. A staff app can load product details through GraphQL. Live IMU from a trolley stays on the existing WebSocket path, not a GraphQL subscription.

Q134. Authn vs authz? RBAC vs ABAC?

Answer (simple). Authentication is who you are, and authorization is what you may do. RBAC grants power by role, and ABAC grants it by attributes and policies on the request.

Answer (interview). You still check per resource, isolate tenants, and give services identities, often with mTLS, rather than trusting a role string in a JWT forever. Shoppers cannot read other carts. Staff roles cannot export PII dumps without audit. ABAC helps when store, shift, and data class matter more than a single role name, but RBAC is the readable default until those rules exist.

Example. A shopper cannot read another cart, and a staff role cannot export a PII dump without an audit trail. That is authorization on a resource, not only a logged-in flag.

Q135. Rate limiting and backpressure?

Answer (simple). Rate limiting slows noisy clients so one shopper or one bot cannot knock the API over. Backpressure is the same idea inside your own queues. Fairness per tenant matters as much as a global cap.

Answer (interview). Token buckets, 429 responses with Retry-After, load shedding, and queue limits are how you survive. An LLM gateway should cap tokens per minute per store so one prompt loop cannot burn the budget. Backpressure on WebSockets means you drop or sample IMU rather than unbounded buffering. Limits without identity just punish a NAT; limits with a tenant key protect the estate.

Example. The LLM gateway limits tokens per minute per store so one bot cannot burn the budget. A 429 with Retry-After is better than an unbounded Bedrock bill.

Q136. Health checks vs readiness vs liveness?

Answer (simple). A liveness check says restart me if I am dead. A readiness check says do not send traffic yet. They are not the same probe, and mixing them causes restart storms.

Answer (interview). Do not make liveness call a GPU or a lock you might already hold, because a hang then becomes a deadlock restart loop. Readiness should wait until the ONNX Runtime session is loaded. Liveness should stay cheap. Kubernetes will do exactly what you asked, including killing a process that was only slow on a cold model load.

Example. The process is ready when the ONNX Runtime session is loaded, not merely when the HTTP port is open. A crash loop is not ready, and a GPU hang must not be the liveness probe.

Q137. Docker: image, layer, registry, non-root?

Answer (simple). A Docker image is a box with the app and its libraries. Layers cache the build so you do not recopy the world every time. A registry stores those boxes, and you should not run them as root.

Answer (interview). Use a small base, pin digests, scan CVEs, run as non-root, keep secrets out of layers, and add a HEALTHCHECK. Multi-stage builds keep compilers out of the runtime image. Distroless or slim runtimes are for FastAPI plus ONNX Runtime. Reproducible tags beat latest.

Example. The FastAPI plus ONNX Runtime image is a multi-stage build with a slim runtime, non-root user, and no Bedrock keys in the layers. We pin the digest that staging already ran.

Q138. docker vs VM vs process?

Answer (simple). A VM virtualizes hardware, while a container virtualizes the operating-system userland. A process is just a program on the box you already have.

Answer (interview). You trade isolation against overhead. Cloud APIs belong in containers because you want a uniform rollout. On-cart inference is often a process or a library inside the Android app, because nested virtualization is wasted weight on a trolley. Do not container-religion a device that only needs a TF Lite interpreter.

Example. Cloud APIs run in containers. On-cart inference is a process or a native library, not a Docker daemon on the trolley.

Q139. CI/CD for mobile + backend together?

Answer (simple). You need two pipelines that meet at a contract test, because the app and the API ship on different days. The contract is the handshake. Feature flags are how you survive a mismatch.

Answer (interview). Android may flow through Bitrise or GitLab while the API has its own tests, and you version the event schema. An app that expects a new event field must not ship before the gateway understands it. Staged rollouts and flags let you pause one side. Mobile plus backend CI is coordination, not one giant job that paints a green check.

Example. If the Android app expects a new WebSocket event schema, it must not roll out before the gateway. Contract tests in both pipelines are the gate, not a Slack promise.

Q140. Observability: logs, metrics, traces?

Answer (simple). Logs are stories about what happened. Metrics are numbers you can graph. Traces are the path of one request across services, including the LLM and the database.

Answer (interview). Use RED or USE as a starting language, watch cardinality so a cart_id label does not explode Prometheus, and attach exemplars from a spike to a trace. One trace id should follow Android through FastAPI, retrieval, the model, and the WebSocket write. Logs without a trace id are a scavenger hunt. LLM calls belong on that trace or you will never explain a slow Lisa answer.

Example. A cart_id and trace_id start on Android and follow the request through RAG and the WebSocket write. That is how you debug a slow recipe without grepping three systems by time.

Q141. What is p95 latency and why not only average?

Answer (simple). Average hides the slow shoppers. p95 is the wait that 95 percent of requests beat, so 5 percent are worse. Those 5 percent are often the people who complain.

Answer (interview). Put SLOs on p95 and p99, not only on the mean. Tails come from garbage collection, cold starts, retrieval, and the language model, and Lisa's time-to-first-token p95 matters more than a pretty mean if 5 percent of shoppers wait eight seconds. Quote which timer you mean: model-only or end-to-end. Averages are how you miss a stall.

Example. Lisa time-to-first-token p95 matters more than the mean if 5 percent of shoppers wait eight seconds. We SLO that tail, not a notebook's average token time.

Q142. Connection pooling?

Answer (simple). Pooling reuses database connections instead of shaking hands every request. Handshake time adds up under a kiosk burst. A pool that is too large just knocks over the database.

Answer (interview). Size the pool against the database max_connections, set timeouts, and hunt leaks because a leaked checkout connection is a production incident. Spring Boot's HikariCP and FastAPI async pools both need burst math for kiosks. A pool of one per request is not a pool. Watch wait time, not only active connections.

Example. Spring Boot Hikari and the FastAPI async pool are sized for a kiosk burst, not for a quiet laptop demo. If pool wait climbs, we have a leak or an undersized database, not a mysterious slow API.

Q143. Pagination, filtering, idempotency keys in public APIs?

Answer (simple). Public APIs must bound how much you return, how you filter, and how a retry avoids double work. This is the same idea as basic SQL and HTTP, now with SLOs and abuse in mind. A 2 gigabyte GET is not a feature.

Answer (interview). Cap page size, prefer cursor pagination for stable snapshots, and require signed idempotency keys with a TTL on creates and payments. Heavy exports become an async job plus a download link. Filters that hit unindexed columns will blow p95. Idempotency is how a flaky mobile network does not double-charge.

Example. A staff audit export is an async job plus a download link, not a 2 gigabyte GET. The create call carries an idempotency key so a retry does not start a second export.

Q144. 500K users — what do you clarify?

Answer (simple). You clarify whether 500 thousand means registered accounts, daily actives, or concurrent sessions. Design to the peak queries per second, not a vanity count. A chat product with a million daily users is a different bottleneck from a retail account database.

Answer (interview). Load-test the actual peak, then apply caches, read replicas, and queued writes where they pay. The Colan-era figure of about 500 thousand users with responses about 40 percent faster is a backend story when you quote the measurement window and the workload. One million daily-active chat users would bottleneck GPUs, not the same layer. Never let an interviewer treat those numbers as interchangeable.

Example. Five hundred thousand registered users is not one million daily-active chat users. I would state which of those the Colan 500K figure was, and I would not pretend a GPU chatbot has the same bottleneck.

Mobile (Android / Flutter)

Q145. Jetpack: Room, Hilt, Navigation, WorkManager, DataStore?

Answer (simple). Room is the local database. Hilt injects dependencies. Navigation moves between screens, WorkManager runs deferred work, and DataStore holds small preferences.

Answer (interview). These libraries are lifecycle-aware, which is why they beat home-grown singletons on Android. You write migrations for Room, fakes for Hilt in tests, and you treat WorkManager as best-effort deferred work, not as a real-time IMU path. DataStore replaces most SharedPreferences cases with a safer API. Together they are how an ezyCart Android app survives a socket drop without inventing a new architecture every screen.

Example. Cache the last cart locally in Room if the socket drops, then reconcile on reconnect. That is Room plus an idempotent sync, not a reason to skip the gateway.

Q146. Kotlin Coroutines + Flow on Android?

Answer (simple). Coroutines make async work readable, and Flow is a stream of values over time, often UI state. You cancel work when the screen goes away. That cancellation is the point.

Answer (interview). viewModelScope cancels when the ViewModel clears, which is how you avoid leaking a WebSocket collector. StateFlow is state; SharedFlow is events; backpressure still exists if a device floods you. Collect on the right dispatcher so you never parse JSON on the main thread. Device events should flow into a ViewModel and then into Compose, not into a God Activity.

Example. Device events arrive as a Flow, the ViewModel holds StateFlow for the screen, and Compose collects it. If the user leaves checkout, viewModelScope cancellation stops the collector.

Q147. What is an ANR and how did you cut 70%?

Answer (simple). An ANR is App Not Responding: the main thread was blocked for about five seconds and the user sees a freeze dialog. It feels like the app died. You cut ANRs by getting I/O and heavy work off that thread.

Answer (interview). You find them with StrictMode, traces, and the Play ANR dashboard, then you move disk, JSON, and network off the main thread and watch Choreographer frames. The resume claim of about 70 percent fewer ANRs and about 30 percent better performance is only honest if you name the measurement window and the build that changed. Do not parse JSON or hit disk on the main thread in checkout. After the fix you keep watching the same dashboard so the win does not regress.

Example. Do not parse JSON or hit disk on the main thread in checkout. The Colan-era 70 percent ANR drop and 30 percent performance lift are quoted from that before-and-after window, not from a lab anecdote.

Q148. Flutter BLoC vs Provider vs setState?

Answer (simple). setState is local to a widget. Provider passes values down the tree. BLoC takes events in and pushes state out, which is easier to test without drawing the UI.

Answer (interview). Staff apps and TV monitors benefit from predictable, testable state, which is why BLoC often wins over scattering setState. Provider is fine for simple dependencies. You still keep side effects out of widgets. Pick one pattern per app so a TV monitor and a handheld are not two religions.

Example. An Android TV monitor can use a BLoC that subscribes to cart fleet events and maps them to view state. Tests can fire those events without a TV in the room.

Q149. Platform channels / FFI in Flutter?

Answer (simple). Platform channels and FFI are how Dart talks to native Android or iOS when you need hardware the Flutter plugins do not cover. Keep that bridge small. Errors should come back as typed results, not as mystery strings.

Answer (interview). Channels are for scan, pay, and similar device calls; FFI is for tight native libraries. Threading matters because a blocking native call can still jank the UI. Typed error codes beat a boolean. An SDK lite that calls native scan or pay should treat the channel as an untrusted boundary the same way a backend treats HTTP.

Example. An SDK lite may call native scan and pay over a small platform channel. The Dart side validates the result and never assumes the native call succeeded because it did not throw.

Q150. iOS interop / KMP at a high level?

Answer (simple). Kotlin Multiplatform lets you share some logic across Android and iOS without sharing all of the UI. Flutter shares UI another way. You should not oversell a stack you did not ship deeply.

Answer (interview). KMP uses expect and actual to fill in platform gaps. If you have shipped mostly Android and Flutter, say that honestly rather than claiming to be a UIKit specialist. iOS via Flutter or a thin interop layer is a real skill, but it is not the same as owning a large UIKit codebase. Interviewers prefer an accurate boundary to a fake native resume line.

Example. Honest answer: I have shipped Android and Flutter, and iOS through Flutter or interop, and I will not pretend to be a deep UIKit specialist unless that is actually true.

Q151. Offline-first mobile?

Answer (simple). Offline-first means the app still works in a dead aisle and then syncs when the radio returns. The phone holds a local source of truth. Not every feature can work that way, especially payments.

Answer (interview). You queue writes, resolve conflicts, and replay idempotent commands when connectivity is back. Scanning and cart edits can be local; charging a card usually cannot complete without the server and an idempotent retry. Design the conflict rule before you ship the cache. A trolley in a Faraday-cage corner of a store is the test, not the office Wi-Fi.

Example. You can scan locally and keep the cart in Room, but pay requires connectivity and an idempotent retry. Offline-first is not a promise that payments work in a dead zone.

Q152. Security on device: keystore, certificate pinning, secrets?

Answer (simple). Do not hide API keys in the APK and hope nobody unpacks it. Use the operating-system keystore for secrets the device must hold. Pinning is a maybe, not a default you cargo-cult.

Answer (interview). Obfuscation is not security. Attestation can raise the bar for tampered clients. Real secrets such as Bedrock keys stay on the server. Certificate pinning helps against some network attackers and hurts when you rotate certs without a pin update, so you enable it with a break-glass story. The trolley APK should hold tokens that are scoped and rotatable, not cloud credentials.

Example. Bedrock keys never live in the trolley APK. The app talks to our gateway, and the gateway holds cloud credentials in the server environment.

Q153. Play/App Store staged rollout?

Answer (simple). You ship to 1 percent of users, watch crashes and ANRs, then walk to 100 percent. A store binary is a slow lever. Feature flags are a faster one.

Answer (interview). Upload mapping files so crash stacks are readable, watch ANR dashboards, and do not confuse a Play rollout with a model-file update. A vision model can be a feature-flagged download inside an already-shipped app. Flags let you disable a bad detector without waiting for store review. Staged rollout is how you avoid a 100 percent bad morning.

Example. A vision model file update may be a feature-flagged download rather than waiting for a Play release. The binary rollout still goes 1 percent to 100 percent while we watch ANRs.

Q154. Deep links and notifications?

Answer (simple). A deep link opens a particular screen from a URL. A notification pokes the user and may do the same. Neither should leak personal data in transit or on a lock screen.

Answer (interview). Authenticate deep links so a guessed cart URL is not a data breach. Keep PII out of push payloads; a jammed kiosk alert can carry a device id, not a shopper name. Notification taps should land on a screen the role is allowed to see. Treat both as public surfaces even if the app is internal.

Example. Staff can be notified of a jammed kiosk with a device id, not a shopper name. The deep link opens the staff screen only after the same authz check as the rest of the app.

LLM, RAG, and agents (intermediate)

Q155. What is Generative AI? What is an LLM?

Answer (simple). Generative AI creates new content rather than only labelling what already exists. A large language model predicts the next tokens from the context you gave it. That prediction can be useful and it can also be confidently wrong.

Answer (interview). Under the hood you have a transformer decoder, a tokenizer, a context window, and a real hallucination risk. Production use adds grounding, evaluation, cost control, and safety, because a next-token model is not a database. Lisa can generate a recipe answer; YOLO cannot, and you should not use an LLM as the only allergen source. Say what is generated versus what is retrieved.

Example. Lisa generates answers; YOLO does not. We never use an LLM as the only allergen source, because a fluent sentence is not a certified ingredients database.

Q156. Tokens and context windows?

Answer (simple). Tokens are pieces of words, not always whole words. The context window is how much the model can see at once. If you dump the whole store manual into that window you pay a lot and the model still gets lost.

Answer (interview). Cost scales with tokens, and a tokenizer mismatch between your counter and the model will silently truncate or inflate bills. RAG should retrieve a handful of chunks, not 100 thousand tokens of catalogue. Basket plus top chunks is the usual packing for Lisa. Long context is not a strategy if retrieval already failed.

Example. We send the basket plus the top few chunks, not the whole store manual. That keeps Lisa inside the window and inside the token budget.

Q157. What is a transformer / self-attention (intermediate)?

Answer (simple). Self-attention lets each word look at the other words that actually matter for it. That is how 'this' can attach to a SKU mentioned earlier in the cart. Stack enough of those layers and you have a transformer.

Answer (interview). Each head has queries, keys, and values, then you add residuals, a feed-forward block, and positional encodings so order is not lost. Encoder-only, decoder-only, and encoder-decoder stacks are different deployment shapes. Multi-head attention is several of those looks in parallel. In an interview, connect the mechanism to a failure such as lost-in-the-middle rather than only drawing Q, K, and V.

Example. An LLM can connect the word this to the SKU mentioned earlier in the cart list because attention can look back across that context. If the cart is huge, that look-back still competes with everything else in the window.

Q158. Embeddings and cosine similarity?

Answer (simple). An embedding turns meaning into a list of numbers, like a map pin for a sentence or a product. Cosine similarity asks how aligned two of those arrows are. Nearby pins can share a meaning even when the words differ.

Answer (interview). Normalize vectors before cosine, use an approximate nearest-neighbor index at scale, and remember out-of-domain queries will still get a nearest neighbor that may be junk. When you upgrade the embedding model you must re-embed the corpus, because mixed versions are not comparable. Version that model in the index metadata. Retrieval quality starts here, before the LLM writes a word.

Example. The query cheap sparkling water should sit near value seltzer even if the words differ. If we upgrade the embedding model, we re-embed the catalogue rather than mixing two maps.

Q159. Prompting vs RAG vs fine-tuning (intermediate)?

Answer (simple). Prompting is the instructions you give right now. RAG is opening a trusted book and then writing. Fine-tuning is extra practice that actually changes weights.

Answer (interview). Facts that change, such as weekly promos, belong in RAG. Style and JSON format often belong in a prompt, or in a small fine-tune if the prompt cannot hold the shape. Fine-tuning does not create a live truth source, and prompting does not either. Combine them on purpose: RAG for SKU facts, prompt for tone, LoRA for a stubborn output schema.

Example. Weekly promos go through RAG, while JSON shape is a prompt or a LoRA if thousands of labelled receipts justify it. Prompting alone does not make a promo true.

Q160. Temperature and top-p?

Answer (simple). Temperature and top-p control how adventurous decoding is. Low values make the model more steady and repetitive. High values make it more surprising and more willing to wander.

Answer (interview). Neither knob guarantees facts, because a fluent wrong answer can still come out at temperature zero. For tool calls and JSON you want temperature near zero so the schema stays stable. RAG answers that must cite a chunk also want a low temperature. Creative copy can run warmer, but checkout extraction should not.

Example. Missing-SKU extraction runs at temperature near zero so the JSON schema stays boring and valid. We do not turn temperature up to make a factual stock answer 'more interesting'.

Q161. What is RAG (intermediate)?

Answer (simple). Retrieval-augmented generation finds trusted pages first, then writes the answer from them. The model is a speaker, not the filing cabinet. If retrieval misses, the sentence will still sound sure.

Answer (interview). The pipeline is ingest, chunk, embed, retrieve, optionally rerank, generate, cite, and abstain when the chunks are silent. You evaluate retrieval and generation separately so you know which half broke. Shopping assistant recipes with LangChain or LlamaIndex are this pattern over catalogue and policy. Citations are how a shopper or a lawyer can see the source.

Example. Lisa's shopping assistant and recipe flow retrieve catalogue and policy chunks, then generate with those rows in context. If the chunk does not list an ingredient, the answer must abstain rather than invent a SKU.

Q162. BM25 vs dense vs hybrid (intermediate)?

Answer (simple). BM25 matches words. Dense retrieval matches meaning. Hybrid does both and then usually reranks, which is why it is the default when you can afford it.

Answer (interview). BM25 wins on SKUs and exact ids such as EZY-1044. Dense wins on paraphrase such as a spicy pasta night. Hybrid with reciprocal rank fusion and a reranker is the practical default, tuned on a labelled query set. Explain a failure mode for each, because none of them is magic on every query.

Example. EZY-1044 should hit BM25. A shopper who types spicy pasta night needs dense retrieval, and hybrid is how Lisa does not make the shopper choose which language to speak.

Q163. Chunking and overlap?

Answer (simple). Chunking cuts documents into useful pages so retrieval can pick a passage. A little overlap stops a sentence from being torn in half. Bad chunks make even a strong model sound lost.

Answer (interview). Prefer structure-aware splits that keep headings with their clauses, use about 10 to 20 percent overlap, and treat tables and code as special rather than naive token windows. A return-policy clause must stay with its heading or RAG will cite the wrong rule. Evaluate with questions that need two parts of the same section. Chunking is a product decision, not a preprocessor default you never revisit.

Example. A return-policy clause stays in the same chunk as its heading. If we split in the middle, Lisa will retrieve a fragment and invent the missing condition.

Q164. What is a vector database (intermediate)?

Answer (simple). A vector database stores embeddings and finds nearest neighbors, often with metadata filters. It is a search engine for meaning. It is not automatically secure or correctly filtered.

Answer (interview). Pinecone, Qdrant, Milvus, Weaviate, pgvector, FAISS, and similar stores differ on ops, filters, and recall, so you benchmark the query you will actually run. Apply ACL and tenant filters in the query, not as an afterthought on the result list. You still need hybrid search and eval. A vector database without metadata filters is how a shopper retrieves a staff-only chunk.

Example. The ACL filter belongs in the vector query so a shopper never retrieves staff stock-audit chunks. Filtering after retrieval is a leak waiting to happen.

Q165. Hallucination (intermediate)?

Answer (simple). Hallucination is confident nonsense: a fluent answer that is not supported by the world or the documents. You will not drive it to zero. You can make it rarer and make the system abstain.

Answer (interview). Ground the model in retrieved chunks, demand citations, abstain when chunks are silent, evaluate faithfulness, keep temperature low for factual tasks, and optionally add a critic. Never promise zero hallucinations. Allergen and price answers are where this matters most. A shopping assistant that invents gluten-free is a safety bug, not a cute LLM quirk.

Example. If the retrieved chunk is silent about gluten, Lisa must not invent a gluten-free claim. Abstain, ask a follow-up, or point the shopper to the pack, rather than sounding sure.

Q166. LangChain vs LlamaIndex (intermediate)?

Answer (simple). LangChain is Lego for applications and tools. LlamaIndex is a librarian for data and RAG indexes. You can combine them, and you should still keep business logic in your own code.

Answer (interview). Indexes and ingestion often sit well in LlamaIndex. Tools, agents, and graphs often sit well in LangChain or LangGraph. Test retrieval and tools with fakes so a demo key is not required to unit-test checkout. Frameworks are not an excuse to bury pricing rules inside a chain class you cannot read.

Example. We can build recipe indexes in LlamaIndex and expose tools through LangChain or LangGraph. The add-to-cart rule still lives in our service code with tests that do not call an LLM.

Q167. What is an AI agent vs a chain (intermediate)?

Answer (simple). A chain is a fixed recipe of steps. An agent chooses the next allowed step as it goes. Agents fail in more ways, so you start with a chain unless the task truly branches.

Answer (interview). Agents need state, tools, budgets, and stop conditions, because a loop that can call tools can also loop until the money is gone. A SKU lookup is a chain. A multi-step recipe that then finds missing items may be an agent, still with a step cap. In an interview, say what the agent is not allowed to do.

Example. SKU lookup is a chain: retrieve, answer, stop. A recipe plus missing-items flow can be an agent that may call get_stock, but it still cannot pay without a human.

Q168. ReAct pattern?

Answer (simple). ReAct means the model thinks, acts with a tool, looks at the result, and repeats. It is a loop with observations, not a single prompt. You cap the loop so it cannot think forever.

Answer (interview). Reasoning plus acting plus observing is useful when the next tool depends on the last result, such as out-of-stock then search-alternatives. Use structured tools rather than free-form side effects, and cap iterations. Log each thought and tool for traces. ReAct without a budget is a cost incident.

Example. A Lisa flow can call get_stock, see out of stock, then search_alternatives. That ReAct loop is capped, and it still cannot apply an invented discount.

Q169. Tool calling (intermediate)?

Answer (simple). Tool calling is the model filling in a form and your code running the function. The model is not the security boundary. Your validator and authorization checks are.

Answer (interview). Define a JSON schema, validate with Pydantic, authorize the caller, execute with timeouts, and return a typed result. Human-in-the-loop belongs on money and irreversible actions. Treat the model output as untrusted the same way you treat a public HTTP body. If the schema is loose, the model will wander into a dangerous argument.

Example. add_to_cart uses a strict schema and Pydantic, and pay requires a human in the loop. The model may propose the tool call; it does not get to charge a card by itself.

Q170. MCP (intermediate)?

Answer (simple). MCP is a standard plug for tools and context so hosts and servers can talk without a custom glue each time. It is a protocol. It is not automatically safe.

Answer (interview). You still need authentication, authorization, validation, and audit on every tool the server exposes. A catalogue MCP server is reasonable. A raw SQL MCP server without a guard is a data breach with extra steps. Treat MCP like any other remote API, because that is what it is.

Example. A catalogue MCP server can expose search with the same ACL as the HTTP API. We would not expose raw SQL over MCP without a guard and an audit log.

Q171. LangGraph (intermediate)?

Answer (simple). LangGraph is a flowchart with memory: nodes do work, edges choose the next node, and state is the backpack you carry. You can loop, checkpoint, and wait for a human. It is how you make an agent inspectable.

Answer (interview). Cycles, checkpoints, human-in-the-loop, retries, and step limits are the features that matter in production. Lisa can retrieve, generate a recipe, list missing SKUs, then confirm, with a recorded state at each hop. Graphs fail clearly when you forget a termination condition. Prefer a graph you can draw on a whiteboard over a hidden recursive agent.

Example. Lisa's graph retrieves, generates a recipe, lists missing SKUs, then confirms with the shopper. Each step is a node we can checkpoint and test without running the whole loop.

Q172. CrewAI / AutoGen (intermediate)?

Answer (simple). CrewAI and AutoGen are helpers for role-based or conversational multi-agent setups. They make it easy to spin up a planner talking to a researcher. Easy is not the same as better.

Answer (interview). Give agents narrow roles, share state explicitly, and measure whether extra agents lift the eval against one agent or a graph. Often LangGraph is enough. Multi-agent debate that cannot change a groundedness number is theatre. Cost and latency grow with every extra speaker.

Example. Use a planner, researcher, and executor only if an eval shows lift versus one agent on the same recipe task. Otherwise Lisa stays a single graph with tools.

Q173. LangSmith / Langfuse / LangFlow / n8n?

Answer (simple). LangSmith and Langfuse are flight recorders and eval harnesses for LLM traces. LangFlow and n8n are visual pipes that are handy for prototypes and ops glue. Visual is not the same as production control.

Answer (interview). Redact PII in traces, because a prompt log is a new database of shopper text. Use Smith or Fuse to score groundedness and to inspect a failed tool call. Use n8n for something like a Telegram alert pipe on an independent trading venture, not for checkout. A visual builder that cannot version, review, or roll back is not your payment path.

Example. Trace a failed add_to_cart in LangSmith or Langfuse. Use n8n for a Telegram alert pipe on TAN, not for ezyCart checkout.

Q174. Hosted vs local LLM (intermediate)?

Answer (simple). A hosted LLM is a model you call over the internet, the way you would phone a specialist. A local LLM is a model you run on your own machines, so the tokens do not have to leave your building.

Answer (interview). Quality usually still favors frontier hosted models, while cost flips as soon as a FAQ or extraction path is hot enough to keep a GPU busy all day. Privacy and data-residency push you local when prompts contain receipts, diets, or camera-derived text you do not want in another vendor's log. Latency is a round trip plus queue on a hosted API, versus a cold start and a capacity plan you own with Ollama or vLLM. Operations is the rest of the iceberg: keys, quotas, evals, and paging a vendor versus paging your GPU host. Sit every provider behind one abstraction with fallbacks across Bedrock, Vertex AI, OpenAI, Anthropic, Gemini, Ollama, and vLLM so a single outage or price spike is not a product outage. Choose hosted for a Lisa pilot that needs frontier quality; choose local when a measured FAQ slice meets quality at a lower token cost or a harder privacy bar.

Example. We would pilot Lisa on a hosted frontier model so the shopping assistant is allowed to be good while we learn the eval. Then we would measure a local Llama 3 FAQ slice on the same labelled questions, and only move that slice off the hosted API if groundedness, privacy, and cost all hold.

Q175. LoRA / PEFT (intermediate)?

Answer (simple). LoRA is a way to train a small adapter instead of the whole model. PEFT is the family name for that idea. You keep the base frozen and teach a thin extra path.

Answer (interview). You pick rank and alpha, freeze the base, and evaluate the actual task metric, watching for overfit on a few thousand labels. You can merge the adapter for simplicity or serve it beside the base. LoRA is good for a JSON extractor's style; it is a bad place to hide SKU facts that should live in RAG. If 5 thousand labelled receipts already memorize, you shrank a memorizer, not a general assistant.

Example. A LoRA JSON extractor can learn the receipt schema. SKU facts still live in RAG, because we do not want to fine-tune every promo into weights.

Q176. Quantization (intermediate)?

Answer (simple). Quantization uses fewer bits for the numbers so the model is smaller and faster. You must recheck quality, especially on the small or rare classes. Average metrics can hide a disaster.

Answer (interview). INT8, 4-bit, and FP8, plus post-training quantization versus quantization-aware training, are the usual menu, and TensorRT or TF Lite are where those bits actually run. Look at per-class recall, not only mean mAP. If small SKUs drop after INT8, mix precision or skip quantizing that head. Quantization is a measured trade, not a checkbox on an export script.

Example. INT8 YOLO on the cart is the default speed play. If small SKUs drop on the per-class report, we mix precision rather than pretending the average mAP still represents those bottles.

Q177. vLLM and KV cache (intermediate)?

Answer (simple). vLLM is a fast server for open models. The KV cache remembers keys and values from past tokens so the model does not re-read the whole prompt every step. That memory is why streaming feels affordable.

Answer (interview). Paged KV, continuous batching, and prefix caching are how vLLM keeps GPUs busy with many chats. A stable system prompt can sit in a prefix cache. This is a server-side story, different from on-device ONNX Runtime or TF Lite on a cart. Do not talk as if vLLM runs on the trolley NPU.

Example. Lisa's system prompt can be prefix-cached on a vLLM box. Cart hardware still runs TF Lite for vision; that path is not vLLM.

Q178. SSE (intermediate)?

Answer (simple). Server-sent events are a one-way stream from the server to the client, like a ticker of tokens. If the client hangs up you must stop the work. A spinning GPU after a tablet sleeps is a bug.

Answer (interview). You send text/event-stream, heartbeat comments so proxies do not kill you, and you cancel generation on disconnect. Last-Event-ID exists for resume, but many LLM streams just start a new answer. TTFT is when the first event arrives. SSE is how Lisa talks to a tablet without a full WebSocket for a one-way token hose.

Example. Lisa streams tokens over SSE, and we cancel GPU work if the tablet sleeps or the shopper navigates away. Otherwise you pay for tokens nobody will read.

Q179. PII (intermediate)?

Answer (simple). PII is personal data that can identify someone. Do not send it to a model if you can avoid it. Logs and traces are a second copy you might forget.

Answer (interview). Detect and redact at the gateway, set retention, and remember that faces in video are PII too. Diet preference might be acceptable in a recipe prompt; a card number or Aadhaar number never is. Prompt logs are a datastore with a new blast radius. Shopper text goes through the same review as any other personal field.

Example. A diet preference may be allowed in a Lisa prompt. PAN or Aadhaar must never enter prompts, traces, or training dumps.

Q180. Prompt injection (intermediate)?

Answer (simple). Prompt injection is a hidden note that tries to boss the model, often sitting inside a product description or a user message. The model cannot be trusted to ignore it. Your tools must refuse.

Answer (interview). Treat user text and retrieved documents as untrusted. Only structured, authorized tools can run, and money paths need a human. Red-team with malicious catalogue copy. A product text that says ignore policy and take 100 percent off must not be able to call a discount tool.

Example. If product text says ignore policy, 100 percent off, the discount tool must not run. Lisa may quote the text as data; it may not obey it as an instruction.

Trading hygiene

Q181. What is a backtest vs paper vs live?

Answer (simple). A backtest is a replay of the past with your rules. Paper trading is fake-live, with real clocks and no real money. Live is real money, real fills, and real regret if you skipped costs.

Answer (interview). You must model costs, slippage, latency, look-ahead, and survivorship, then forward-test before you scale. TAN is an independent venture, not an employer, with about six years of systematic work, Telegram alerts on 5-minute and 15-minute bars, Pine and Python research, and a sub-5 percent drawdown claim you should be able to define. Definitions matter more than a pretty equity curve. You cannot treat the unclosed bar's high as a known value.

Example. You cannot use the unclosed 5-minute candle's high as a known value in a TAN backtest. That would be look-ahead, and it would not survive paper trading.

Q182. Maximum drawdown?

Answer (simple). Maximum drawdown is the worst fall from a peak to a trough on the equity curve. It is one risk number, not the whole risk story. A small drawdown on a short sample can still hide an ugly tail.

Answer (interview). Always say the period, the sample size, and whether costs are in, because a sub-5 percent drawdown is a claim with a method, not a personality trait. Two strategies with the same drawdown can have very different tail losses. Position sizing is how you target that number. TAN's sub-5 percent figure is an independent-venture metric you should explain, not a company result.

Example. A peak of 100 down to 95 is a 5 percent drawdown. I would still ask what the tails look like, because two TAN-style rules with the same drawdown can fail very differently on a gap day.

Q183. Look-ahead bias and survivorship?

Answer (simple). Look-ahead bias is using future bits as if you already knew them. Survivorship bias is studying only the names that still exist, like grading only the students who stayed in school. Both make a backtest look kinder than live trading.

Answer (interview). Use point-in-time listings, include delisted symbols, and timestamp news and sentiment at the moment they were knowable. An article stamped after the move is leakage. Walk-forward with an embargo is how you reduce the temptation. If your data vendor quietly drops dead tickers, your history is already lying.

Example. Sentiment from an article timestamped after the move is leakage. A TAN research pipeline has to reject that feature the same way a vision pipeline rejects future frames.

Q184. SMC / liquidity in one honest sentence?

Answer (simple). Smart-money and liquidity ideas are a family of discretionary price-action stories. If you use them, they must become testable rules with a stop, or they are folklore. Honesty beats mystique in an interview.

Answer (interview). Do not mystify order blocks as magic. Encode the rule, test it walk-forward, put risk limits on it, and accept that many SMC sketches fail once costs are in. TAN can use SMC-inspired signals only as rule-based code with position sizing. If you cannot write the rule down, you cannot defend it.

Example. TAN uses rule-based, SMC-inspired signals with position sizing, validated walk-forward, not folklore on a screenshot. If a liquidity idea cannot become a coded rule, it does not go live.

Q185. Why FinGPT is not a trading system?

Answer (simple). FinGPT reads finance language, but it does not know your fills, your fees, or your kill switch. A language model is not a trading system or a book of orders.

Answer (interview). At best it is a feature candidate, such as a sentiment score, and you still validate lift after costs. Risk limits still rule the account. Auto-market-ordering from a generated sentence is how you donate money to the spread. Keep the model upstream of a boring position sizer.

Example. Sentiment from a finance language model can be one XGBoost feature in research. It never becomes an automatic market order on TAN, because fills and fees are the real system.

Q186. Position sizing and risk limits?

Answer (simple). Position sizing is how big a bet you take. Risk limits are the max pain you allow before you flatten. Together they matter more than a model's confidence score.

Answer (interview). Volatility targeting, stop rules, and kill switches are independent of the machine-learning toy that suggested the direction. A sub-5 percent drawdown target drives size. If the model is 90 percent sure and the limit says no, you do not trade. That separation is the professional answer.

Example. TAN's sub-5 percent drawdown target drives position size, not the model's confidence alone. A confident signal that would breach the limit is a pass, not a larger order.

Cloud (Bedrock, Vertex, SageMaker)

Q187. AWS Bedrock in one minute?

Answer (simple). Bedrock is a managed door to foundation models on AWS. You call models without standing up every GPU yourself. You still owe evals, IAM, and a fallback.

Answer (interview). Talk about IAM, data handling, guardrails, regions, latency, and price, because a managed API is still a production dependency. Sit Bedrock behind the same abstraction as other providers so you can fall back. Lisa can route to Claude on Bedrock for a pilot, then fail over if throttled. Bedrock does not replace your groundedness tests.

Example. Route Lisa to Bedrock Claude for a hosted pilot, with a fallback on the same router interface. If Bedrock throttles, the shopper should still get a degraded but safe path, not a blank bubble.

Q188. GCP Vertex AI?

Answer (simple). Vertex AI is Google's managed platform for training, serving, and app-level generative AI. It is another door, with Google's IAM and regions. It is not a reason to rewrite your product logic in a vendor notebook.

Answer (interview). You still care about IAM, endpoints, feature stores, and monitoring, and you avoid lock-in where it would hurt a later move. The honest architecture is the same router interface you use for Bedrock. Eval, grounding, and cost caps do not change because the logo did. Choose Vertex when the rest of the estate is already on GCP, not because a slide said it was unified.

Example. Lisa's router should call Vertex with the same interface as Bedrock. Switching clouds then becomes a config and an eval, not a rewrite of the shopping assistant.

Q189. SageMaker vs 'just a container'?

Answer (simple). SageMaker is AWS's product for training and hosting models. A plain container is you plus FastAPI plus a GPU node. Both can work; they bill different kinds of toil.

Answer (interview). Use managed training when the operations value beats the lock-in, such as a bursty training job. Many teams still serve FastAPI plus ONNX Runtime themselves for the latency path they understand. Do not put the cart's sub-100ms detector behind a generic hosting wrapper if a local runtime is already faster. Be explicit about which half is managed.

Example. A training job can run on SageMaker while the live latency path stays our FastAPI plus ONNX Runtime service. Those are different problems, and SageMaker winning training does not mean it should serve the cart.

Q190. Databricks in an AI stack?

Answer (simple). Databricks is a data and ML platform on Spark and a lakehouse. It is strong for features, training tables, and governance. It is not an edge runtime for a trolley.

Answer (interview). Use it for catalogue analytics, feature tables, and training sets that need governance. Do not pretend a Spark cluster will meet sub-100ms cart inference. The boundary is: heavy data work in the lakehouse, live infer on device or a slim service. That split keeps roles honest in an interview.

Example. Catalogue analytics can live in Databricks. Cart inference stays on device with ONNX or TF Lite, because a lakehouse cluster will not make a frame-by-frame SLO.

Q191. IAM least privilege?

Answer (simple). Least privilege means every service gets the smallest key that still opens its own door. A shared god role is how a batch job becomes a breach. You audit who used what.

Answer (interview). Give each job a role, ban long-lived keys in Git, and review CloudTrail-style logs. An embed job can read the catalogue bucket without decrypting every KMS key in the account. Humans and services should not share credentials. Least privilege is also how a prompt-injection bug in one tool cannot become admin on the warehouse.

Example. The embed job may read the catalogue bucket and must not have kms:Decrypt on every key in the account. That is least privilege, not a delay in the sprint.

Q192. Blue/green or canary deploys?

Answer (simple). Blue/green means old and new sit side by side, then you switch. A canary sends a little traffic to the new thing, watches, then continues. Both exist so you can roll back without a speech.

Answer (interview). For prompts and models you canary on eval plus business KPIs, not only on HTTP 200. If groundedness drops, revert instantly. Five percent of Lisa traffic on a new prompt should auto-revert rather than wait for a meeting. Infrastructure canaries and prompt canaries share the idea and need different metrics.

Example. Send 5 percent of Lisa traffic to prompt v12 and auto-revert if groundedness drops. That is a canary for a language-model change, not only a load-balancer trick.

Error codes — HTTP, LLM, RAG, and serving

Meanings in full sentences, then the interview action: retry, fail closed, fallback, or page. Checkout scanning does not share fate with chat GPUs.

HTTP, vendor, RAG, and GPU errors

Q193. What does HTTP 400 mean on an LLM or RAG API, and what do you do?

Answer (simple). HTTP 400 means the client sent a request the server cannot understand, such as a malformed JSON body or a missing required field. It is the caller's homework, not a GPU failure. In an interview I say I fix the payload and I do not retry 400 in a loop, because the same bad body will fail forever.

Answer (interview). A 400 Bad Request on a FastAPI RAG or agent endpoint usually means Pydantic or the HTTP layer rejected the shape of the request before a model was even called. I log the validation errors, return a typed problem body, and treat the case as a client bug in traces so on-call does not page GPUs. I never map a 400 to a silent empty answer, because Lisa would then look as if she had no catalogue. In production I add contract tests so Android and the agent tools cannot ship a field rename that produces a wall of 400s. Retrying 400 is a cost bomb, not resilience. The interview story is: classify the error, fail closed, teach the client.

Example. If add_to_cart arrives without a sku, FastAPI returns 400 with a field error, the agent tells the shopper the barcode was empty, and we do not call Bedrock at all.

Q194. What does HTTP 401 mean, and how is it different from 403?

Answer (simple). HTTP 401 means the server does not know who you are, usually a missing or expired token. HTTP 403 means the server knows who you are and still will not let you do that thing. Shoppers hear both as 'access denied', but the fix is different: refresh login versus stop asking for someone else's cart.

Answer (interview). 401 Unauthorized is unauthenticated: no Bearer token, a bad JWT, or an expired credential. 403 Forbidden is authenticated but not allowed: wrong tenant, wrong role, or an ACL miss on a RAG collection. I never collapse them into one 'auth error' in an LLM gateway, because a shopper with an expired token needs a refresh, while a shopper querying staff stock-audit chunks needs a hard deny. In traces I tag 401 as client-session and 403 as authorization so security reviews can see permission probes. On mobile the ezyCart app refreshes on 401 and does not retry 403. Interviewers want that distinction out loud, not a shrug.

Example. A staff JWT that tries a shopper-only Lisa route should come back 403, while a tablet with an expired token should come back 401 and refresh, never as a hallucinated 'I cannot find that product'.

Q195. What does HTTP 403 mean for RAG and tools?

Answer (simple). HTTP 403 means you are signed in and still not allowed. In RAG it often means the retriever would have returned a document the caller must not see. The safe move is deny, log, and abstain, not to let the model paraphrase the forbidden page.

Answer (interview). Authorization belongs in the retriever and in the tool allowlist, not only in a later prompt instruction. If a shopper's query would retrieve staff-only stock-audit chunks, I filter by ACL at search time and return 403 or an empty authorized set, then Lisa abstains. A 403 on a money tool such as discount or pay is a feature, not an outage. I audit 403 spikes because they can be either a client bug or a probing attacker. MCP and LangChain tools do not become safe because they returned a friendly sentence. The interview action is: show the ACL filter, the deny, and the audit log.

Example. Lisa must not quote a staff stock-audit chunk to a shopper; the retriever filters it, the API can surface 403 to a misconfigured client, and the model says it does not have that information.

Q196. What does HTTP 404 mean on model, index, or SKU routes?

Answer (simple). HTTP 404 means the thing you named is not there: a SKU, a collection, a prompt id, or a model deployment. It is not a permission error and it is not a GPU outage. I tell the user the resource is missing and I do not invent a substitute as if it existed.

Answer (interview). On a catalogue API, 404 is an unknown SKU. On a model gateway, 404 or a vendor model_not_found means the deployment name is wrong or the vendor retired the id. On a vector store, a missing collection should be 404 or a typed index-not-found, not a 500. I fail closed for checkout SKUs and I abstain in Lisa rather than guess. I alert when 404 rates jump after a deploy, because that is often a renamed index alias or a client still on an old path. Interviewers want you to separate 'not found' from 'forbidden' and from 'the model crashed'.

Example. GET /products/EZY-1044 returning 404 means that SKU is not in the live catalogue, so Lisa must not add it; a missing vector collection after a bad alias swap is the same class of error at RAG scale.

Q197. What does HTTP 409 mean in carts, payments, and idempotent LLM tools?

Answer (simple). HTTP 409 means your change collided with the current state, such as two devices updating the same cart version. It is a conflict, not a validation typo. The interview move is to retry with the new version or return the original idempotent result, not to double-charge.

Answer (interview). 409 Conflict is how I surface optimistic-concurrency failures and some idempotency clashes. For payments and add_to_cart I prefer storing an idempotency key and returning the original receipt on replay, which can be 200, while a true version clash is 409. Agent tools must use the same keys so a retried tool call after a 502 does not add pasta sauce twice. I never treat 409 as retry-with-the-same-body forever without reading the new state. In ezyCart, two scan events with different ids are not a 409; two writers on one cart row are. That distinction is what a senior answer sounds like.

Example. If the tablet retries add_to_cart with the same event id, we return the original line instead of a 409 storm; if two writers race the cart version, we return 409 and the client reloads state.

Q198. What does HTTP 422 mean, and how does it show up in LLM tool calls?

Answer (simple). HTTP 422 means the JSON was readable but semantically invalid, such as qty of minus one or a SKU that fails a schema. FastAPI and Pydantic use it for validation that is deeper than 'this is not JSON'. I do not send that body to the model; I tell the agent the field is wrong.

Answer (interview). 422 Unprocessable Entity is the contract for schema-valid JSON that still fails business validation. Pydantic field validators on FastAPI and on tool arguments should return 422 with a pointer to the field. For LLM structured output I validate the model JSON the same way: one repair attempt, then fail closed. Mapping a 422 into a 200 with an error string inside the body is how tools start hallucinating success. In interviews I say 400 versus 422 is less important than 'do not call the LLM, do not retry blindly, show the field'. Money tools use extra=forbid so extra keys cannot sneak a discount.

Example. If Lisa emits add_to_cart with qty=-1, Pydantic raises a validation error, we return 422 to the tool layer, and the graph asks the shopper to confirm a positive quantity instead of hitting the cart service.

Q199. What does HTTP 429 mean for OpenAI, Anthropic, Bedrock, and your own gateway?

Answer (simple). HTTP 429 means slow down: you hit a rate limit or a quota window. Retrying immediately makes it worse. I back off, honor Retry-After if present, shed load to cache or a smaller model, and I never hide 429 as a generic 500.

Answer (interview). 429 Too Many Requests is the honest signal for tokens-per-minute, requests-per-minute, or a per-tenant quota I set on the Lisa gateway. Vendor APIs and our own gateway should both emit 429 rather than queue until the shopper's tablet looks frozen. I use jittered exponential backoff, a max attempt count, and a fallback route: semantic cache, extractive FAQ, or a local vLLM Llama 3 slice. I meter 429s per tenant so one viral recipe loop cannot starve the store. In an interview I separate rate_limit from insufficient_quota: one is a window, the other is billing. Both can surface as 429 depending on the vendor, so I read the error code inside the body, not only the HTTP status.

Example. If Bedrock throttles Lisa, the gateway returns 429 with Retry-After, retries twice with jitter, then answers a FAQ from cache or a local vLLM model instead of hanging the tablet.

Q200. What does HTTP 500 mean on an AI service, and what should you not do?

Answer (simple). HTTP 500 means the server hit an unexpected failure, such as an unhandled exception in FastAPI. It is our bug until proven otherwise. I do not return 500 for validation, and I do not retry 500 on a non-idempotent charge without a key.

Answer (interview). 500 Internal Server Error should be rare, typed in logs, and tied to a trace id the on-call can open in LangSmith or OpenTelemetry. I distinguish 500 from 502/503/504, which often mean a dependency or overload rather than a crash in our process. I never convert a content_filter or a Pydantic miss into a 500, because that trains clients to retry unsafe or invalid work. For LLM backends I wrap vendor errors into a stable envelope: retryable versus not, plus the vendor code. Checkout paths fail closed. The interview answer is the taxonomy, the trace, and the rollback, not 'restart the pod'.

Example. An unhandled None in a reranker is a 500 with a trace id; a bad Lisa prompt that the model refuses is not a 500, and we do not restart GPU nodes to hide an application bug.

Q201. What does HTTP 503 mean, and how do you use it for GPU backpressure?

Answer (simple). HTTP 503 means the service is temporarily unavailable, often overloaded or not ready. It is a polite closed sign, not a mystery crash. Clients should retry with backoff or take a degraded path, and load balancers should stop sending traffic until readiness returns.

Answer (interview). 503 Service Unavailable is how I shed load when the vLLM queue, the ONNX pool, or a dependency is past SLO. Readiness probes should fail so Kubernetes or the gateway stops new traffic, while liveness stays up unless the process is actually wedged. I pair 503 with Retry-After and a degraded mode: cache-only FAQ, skip the reranker, or route to a smaller model. Returning 200 with an empty completion is worse, because the shopper thinks Lisa answered. GPU OOM can become 503 after we catch it, rather than a crashing tight loop. Interviewers want backpressure, not heroic infinite queues.

Example. If the vLLM queue would blow Lisa p95, we return 503, serve cached FAQs, and keep ezyCart scanning on the WebSocket path so checkout does not wait on a GPU.

Q202. What do HTTP 502 and 504 mean in front of an LLM or vector database?

Answer (simple). HTTP 502 means a proxy got a bad answer from upstream. HTTP 504 means the proxy waited too long. Both often sit in front of vLLM, Bedrock, or a vector DB. I treat them as dependency failures, retry only if the tool is idempotent, and I cut the prompt or the k so the next call can finish.

Answer (interview). 502 Bad Gateway and 504 Gateway Timeout are typical when API Gateway, nginx, or a mesh in front of FastAPI cannot complete a long prefill. I set timeouts deliberately: shorter than the gateway, longer than p95 TTFT, and I stream SSE so a long generation is not one blocking HTTP call. I do not raise the gateway timeout to five minutes as a strategy; I shrink context, use prefix cache, or split tools. Vector search that hangs should time out with a 504-class error and a BM25 fallback. The interview action is timeout budgets and fallbacks, not a bigger number in nginx.

Example. A long RAG prefill that dies at the mesh is a 504; Lisa should then retry a shorter context or fall back to BM25 plus a smaller model, not wait on a second full 10-million-document search.

Q203. What is insufficient_quota, and how do you talk about it in an interview?

Answer (simple). insufficient_quota means the vendor will not run the request because the account is out of credits or over a hard billing cap. It is not a bad prompt. I fail the request, page billing or the platform owner, and I switch to a fallback model or a local vLLM path if the product must stay up.

Answer (interview). OpenAI-style insufficient_quota and similar Bedrock or Anthropic billing errors are capacity of money, not capacity of GPUs. I surface them as a typed non-retryable error distinct from 429 rate limits, because retrying a dead quota only burns latency. Dashboards should show quota remaining, not only HTTP codes. For Lisa I keep a documented fallback: another vendor behind the same router, or a local Llama 3 FAQ slice. I do not hide quota death as a generic 500, and I do not invent a cost savings figure in the interview. The senior move is prepaid alerts, per-tenant caps, and a kill switch before the bill becomes a surprise.

Example. If the hosted pilot for Lisa hits insufficient_quota, the router fails that path, serves FAQ from local Llama 3 or cache, and pages the owner rather than looping the same OpenAI call.

Q204. What is context_length_exceeded, and what do you do?

Answer (simple). context_length_exceeded means the prompt plus the completion budget is larger than the model's window. The model never saw the end of your RAG dump. I cut history, cut chunks, or map-reduce the documents, and I do not raise max_tokens to 'see if it fits'.

Answer (interview). This error is a design smell: we stuffed system prompt, chat history, and fifty retrieved chunks into one window. Fixes are rerank-to-five, lost-in-the-middle packing, hierarchical summarize, a larger-window model only when eval says it helps, and a hard cap in the gateway. Tokenizer mismatch can cause a surprise here if we counted characters or counted with the wrong tokenizer. I fail the request with a typed error the agent can handle: shrink and retry once, then abstain. I never silently truncate the middle of a legal clause. Interviewers want the packing strategy, not 'we switched to a 128k model' as the only plan.

Example. Lisa recipe RAG that stuffs forty chunks will hit context_length_exceeded; we rerank to five, keep the clause with its heading, and retry once before abstaining.

Q205. What is a rate_limit error versus a 429 you emit yourself?

Answer (simple). A vendor rate_limit error means their window is full. A 429 you emit yourself means your gateway is protecting tenants and GPUs. Both mean wait, but the lever is different: change the vendor plan versus shed your own traffic.

Answer (interview). I parse the vendor body for rate_limit_exceeded, retry-after, and which dimension failed: RPM, TPM, or concurrent connections. I do not blindly retry Bedrock, OpenAI, and Anthropic on the same policy, because their windows differ. Internally I still return 429 to Android and to agent tools so they share one client policy. I keep a token bucket per store so one merchant cannot consume the vendor quota for everyone. In interviews I mention jitter, max retries, and a degraded model. I do not claim we 'never hit rate limits' if we run a public shopping assistant.

Example. When OpenAI returns rate_limit_exceeded, Lisa's gateway waits on Retry-After, then may route the FAQ slice to vLLM, while the trolley WebSocket scan path is unaffected.

Q206. What does content_filter or a safety/content policy error mean?

Answer (simple). A content_filter error means the vendor or your guardrail refused the input or the output as unsafe. It is a successful safety decision, not a crashed model. I show a safe refusal to the shopper and I log enough to debug without storing the raw harmful text longer than policy allows.

Answer (interview). OpenAI content_filter, Anthropic refusals, Bedrock Guardrails, and our own classifiers can all block self-harm, scams, or medical overreach. I do not retry a filtered call with a jailbreak 'ignore the policy' rewrite. I distinguish input filter from output filter, because a retrieved product page that tries prompt injection should be dropped before generate. Allergen questions that the RAG cannot ground should abstain even if the filter would have allowed a guess. Metrics should count filter hits separately from 500s so leadership does not treat safety as downtime. The interview action is layered guardrails plus a human policy, not turning the filter off to raise answer rate.

Example. If a retrieved HTML comment tries to force a 100 percent discount, the injection defense and a content policy layer must block the tool, and Lisa refuses instead of calling a discount API.

Q207. What does model_not_found mean, and how do you prevent it in production?

Answer (simple). model_not_found means the name you called does not exist, was renamed, or was retired by the vendor. It is a configuration error. I pin model ids in one router, eval a successor before cutover, and I never scatter raw 'gpt-x' strings across forty services.

Answer (interview). Vendors deprecate models on a calendar. A Principal-level answer is an abstraction: Lisa asks the router for 'recipe-mid' or 'complaint-frontier', not for a hardcoded snapshot in the Android app. When a model id 404s, the router fails over to the last-evaled fallback and pages. CI should fail if a configured id is not in the allowlist. I do not discover deprecation in production on a Saturday. Edge vision uses versioned ONNX files with the same idea: yolo-store-v17, not 'whatever was on disk'. Interviewers are listening for deprecation drills, not for memorized model names.

Example. If a hosted frontier id is retired, Lisa's router serves the previously evaled successor or a local Llama 3 FAQ path, and we do not leave a raw vendor id in the trolley APK.

Q208. What is an LLM timeout, and how is it different from TTFT slowness?

Answer (simple). A timeout means you stopped waiting; the call may still be burning GPU on the other side unless you cancelled. Slowness is a high time-to-first-token that still succeeds. I set timeouts, cancel on SSE disconnect, and I measure TTFT p95 separately from total time.

Answer (interview). Client timeouts, gateway timeouts, and vendor timeouts are three clocks that must nest. If the tablet gives up and we do not cancel, we pay for unused tokens. FastAPI StreamingResponse for Lisa must abort the vendor request on disconnect. I classify timeouts as retryable only when the operation is idempotent and we did not already stream tokens the user might have seen. Prefill-heavy RAG is the usual root cause, so I shrink context before I extend the timeout. Interviewers want cancellation, not a 120-second nginx number. Sub-100ms is a vision SLO on device; chat TTFT is a different budget and I never mix the two numbers.

Example. If the shopper locks the tablet, we cancel the Lisa generation so Bedrock or vLLM does not keep decoding into the void, which is a timeout-and-cancel story rather than a 500.

Q209. What is invalid_request_error on OpenAI-style APIs?

Answer (simple). invalid_request_error means the vendor rejected the arguments: a bad parameter, an unsupported combination, or a schema the model endpoint does not accept. It is usually not transient. I fix the client contract and I do not retry the same JSON.

Answer (interview). This family covers unknown parameters, negative max_tokens, mixing tools with an incompatible response format, and asking a model for a feature it does not have. I keep a compatibility matrix in the router: which deployment supports tools, JSON mode, or vision. Mapping invalid_request_error to 400 or 422 on our public API is correct; mapping it to 500 is a lie. Agent graphs should treat it as a developer bug and fail the CI fixture. I mention it in interviews to show I read vendor error types instead of retrying everything. Logs include the parameter name, not the full prompt if it contains PII.

Example. If a Lisa tool path asks a local Llama 3 deployment for a response_format it does not support, the router returns a typed invalid request and we fall back to Pydantic-validated plain JSON instead of spinning.

Q210. How do authentication_error and permission_error differ on vendor LLM APIs?

Answer (simple). authentication_error means the key is missing, wrong, or revoked. permission_error means the key is valid and still not allowed to use that model or that operation. One is 'who are you', the other is 'this key cannot use GPT-class X'.

Answer (interview). I keep vendor keys in a secret manager, not in the Android APK and not in LangSmith payloads. A rotated key that 401s should page, because every Lisa call will fail. A permission_error on a newly enabled model often means IAM or a project allowlist, which is the Bedrock and Vertex equivalent of 403. Least privilege still applies: the embed job role cannot invoke the frontier chat model. In interviews I connect this to IAM least privilege and to the 401 versus 403 HTTP story so it sounds like one mental model. I do not paste keys into the whiteboard.

Example. The FastAPI Lisa service role can invoke the chat model, the nightly embed job cannot, and a revoked key is an authentication failure that pages on-call rather than a shopper-facing hallucination.

Q211. What is Anthropic overloaded_error or HTTP 529, and what do you do?

Answer (simple). overloaded_error or a 529-style overloaded response means the vendor is too busy, not that your JSON is wrong. It is retryable with backoff, and it is a reason to have a second provider. I do not treat it as a prompt quality problem.

Answer (interview). Anthropic has used overloaded_error when the fleet is saturated. The operational response is the same as a 503: jittered retry, then failover to Bedrock, Vertex, OpenAI, or vLLM behind the same abstraction. I keep this distinct from content refusals so eval dashboards do not punish prompts for vendor weather. Capacity planning still matters: if we depend on one vendor without a fallback, overloaded_error is an outage. Interviewers want multi-provider routing here, which matches the hosted-versus-local story on Lisa. I do not claim we never see overload.

Example. If Anthropic is overloaded during a recipe spike, Lisa's router retries once, then serves the same evaled path on Bedrock or a local Llama 3 FAQ slice.

Q212. What vector database errors must you name in an interview?

Answer (simple). The usual vector DB failures are a missing collection, a dimension mismatch, a bad metadata filter, and a timeout on the query. They look like 404, 400, or 503 at the HTTP layer. I never let them become a confident wrong answer from the LLM.

Answer (interview). Index or collection not found means the alias swap failed or the client is on the wrong environment. Dimension mismatch means the query embedding was produced by a different model than the index, which is a broken deploy. Filter syntax or type errors mean an ACL or tenant field is wrong, and I fail closed rather than search without the filter. Timeouts and memory errors mean shard sizing and HNSW parameters, plus a BM25 fallback. I version embedding models in metadata and dual-run indexes during upgrades so 10 million documents are not silently incomparable. The interview action is typed errors, BM25 fallback, and alias rollback, not 'restart Pinecone'.

Example. If we upgrade the embedding model and forget to re-index, query vectors have the wrong width, the vector DB errors, and Lisa must fall back to BM25 instead of answering from a random nearest neighbor.

Q213. What is an embedding dimension mismatch, and why is it a deploy bug?

Answer (simple). A dimension mismatch means you asked a 384-wide index to store or search a 1024-wide vector. The database should reject it. It always means two embedding models are in play without a migration.

Answer (interview). I pin the embedding model id next to the index name, and I refuse to mix them. Onboarding a new model on 10 million documents is a shadow index, dual retrieve, recall compare, then alias swap, with rollback to the previous generation. Serving a query with the old tokenizer or the new model against the old index is a silent quality disaster if the DB were to coerce sizes, so I want a hard error. FastAPI should catch it as 500/503 only after we classify it as a config incident. Interviewers use this to see whether you have operated RAG or only demoed it.

Example. A Lisa catalogue-v2 shadow index uses the new embedding width; live traffic stays on catalogue-v1 until recall matches, so shoppers never query across mismatched dimensions.

Q214. What GPU OOM errors mean on vLLM, PyTorch, or ONNX, and what do you do?

Answer (simple). GPU out of memory means the card ran out of VRAM: the batch, the KV cache, or the model weights did not fit. It is not a wrong answer from the model. I reduce batch, quantize, cap context, or move the job, and I return 503 rather than crash-loop the replica.

Answer (interview). CUDA OOM on vLLM is often KV cache growth from long RAG prompts and high concurrency, which is why paged attention and prefix cache exist. PyTorch training OOM is a different story: batch size, gradient checkpointing, or a LoRA rank that is too wide. On-device TF Lite and ONNX on the cart are CPU or NPU memory problems, not data-center CUDA, and I do not mix those numbers with the sub-100ms vision SLO. I catch OOM, shed load, and keep a smaller model warm. I never retry the same huge context on the same GPU and hope. The interview answer names prefill versus decode memory, then the lever you turn.

Example. If Lisa's vLLM replica OOMs on a stuffed RAG prompt, we cap chunks, return 503 to new sessions, and keep ONNX vision on the trolley running, because that path is not on the same GPU.

Q215. What is a tokenizer mismatch, and how does it show up?

Answer (simple). A tokenizer mismatch means you counted or split text with a different tokenizer than the model uses. Your 'this fits in 8k' estimate is a lie, and RAG chunks may be too big or oddly cut. I tokenize with the model's tokenizer, not with whitespace or a generic GPT guess.

Answer (interview). Context_length_exceeded, surprise bills, and broken truncation all follow from counting characters or using tiktoken for a Llama model. Chunkers must use the same tokenizer as the embedding or chat model they feed, or at least a conservative upper bound. Fine-tunes and LoRA jobs must train with the right tokenizer files; serving with a different vocab is a silent quality bug. In interviews I say we unit-test token counts on fixtures, including Tamil and Malay product names, because UTF-8 character length is not token length. I do not treat tokenizer files as optional extras in the Docker image.

Example. A Tamil recipe name that looks short in characters can consume many tokens; Lisa's chunker uses the model tokenizer so we do not blow the window or bill a surprise completion.

Q216. How do Pydantic ValidationError and 422 protect agent tools?

Answer (simple). Pydantic raises ValidationError when types or constraints fail. FastAPI turns that into HTTP 422. For agents, the same validation wraps tool arguments so the model cannot pass a string where an int belongs. I treat the model as an untrusted client.

Answer (interview). Tool JSON is not trusted input. I define a schema, parse with Pydantic, reject extras on money tools, and return a typed error the graph can turn into a user question. One repair generation is enough; then I fail closed. This is the same discipline as public HTTP APIs on FastAPI. LangChain or MCP tools that skip validation are how prompt injection becomes a side effect. In interviews I connect 422, structured output, and HITL for pay. I do not 'ask the model to be careful' as the control.

Example. add_to_cart requires sku as a string and qty as a positive int; if the LLM emits qty as the word 'two', validation fails, we ask the shopper to confirm, and we never hit the cart service with a guess.

Q217. What SSE and client-disconnect errors should you handle?

Answer (simple). If the client hangs up, the server should stop generating. An SSE disconnect is not a 500; it is a cancelled job. I cancel the vendor call so we do not pay for tokens nobody will read.

Answer (interview). Server-sent events use text/event-stream. FastAPI must notice a disconnected tablet, cancel the asyncio task, and end the vLLM or Bedrock stream. Heartbeats keep proxies from killing an idle-looking stream during a slow tool call. Last-Event-ID is optional replay, not a substitute for idempotent tools. Mixing WebSockets for IoT with SSE for Lisa tokens is intentional: different backpressure. Interviewers want disconnect cancellation as a cost and correctness control. I do not buffer the entire answer then flush, because that throws away TTFT.

Example. When the ezyCart tablet sleeps mid-sentence, Lisa's SSE handler cancels the GPU work; the cart WebSocket for scans stays up independently.

Q218. What does a 401 or 403 from a vector DB or object store mean during ingest?

Answer (simple). It means the embed job's credentials cannot read the bucket or write the index. It is IAM, not a chunking algorithm failure. I stop the job, fix the role, and I do not mark the index live.

Answer (interview). Nightly re-embed of a catalogue should fail closed if S3 or GCS returns 403, otherwise you swap an alias onto a half-written index. Least privilege still holds: the job can read the catalogue bucket and write one index prefix, not decrypt every KMS key. I surface this as a pipeline failure in CI or Airflow, not as Lisa 500s the next morning. Interviewers connect this to IAM least privilege and to RAG delete/update correctness. I would rather have yesterday's complete index than today's empty one.

Example. If the embed role loses s3:GetObject, the job errors, catalogue-live keeps pointing at yesterday's build, and shoppers still retrieve last known promos instead of a blank index.

Q219. When should you retry an AI error, and when must you not?

Answer (simple). Retry 429, 503, 502, timeouts, and vendor overload with jitter and a cap. Do not retry 400, 401, 403, 404, 422, insufficient_quota, invalid_request, or content_filter. Never retry a non-idempotent pay tool without the same idempotency key.

Answer (interview). I classify errors in one gateway table: retryable, user-fixable, and incident. Bounded exponential backoff with jitter honors Retry-After. After the budget I degrade: cache, BM25, smaller model, or abstain. 409 needs a reload of state, not a blind replay of the old body. Agent loops that retry a typed empty SKU are a bug, not resilience. This is the same pattern I use on ezyCart WebSocket scans: at-least-once delivery plus idempotent handlers. In interviews I would rather hear this taxonomy than a list of status codes without actions.

Example. Lisa retries a 503 from vLLM twice, never retries a 422 from add_to_cart, and retries add_to_cart after a 502 only with the same event key so pasta sauce cannot appear twice.

Q220. How do you present this error taxonomy to a hiring manager who is not an SRE?

Answer (simple). I say: four buckets. The caller sent a bad form, the caller is not allowed, the brain or GPU is busy, or we have a real bug. Each bucket has a shopper-visible sentence and an engineer action. I do not dump twenty acronyms unless they ask.

Answer (interview). Bad form is 400 and 422 and invalid_request: tell the user what field failed. Not allowed is 401, 403, permission_error, and content_filter: refresh login, deny, or refuse safely. Busy is 429, 503, overloaded, OOM after catch, and timeouts: wait, fallback, cancel. Real bug is 500, dimension mismatch, tokenizer/config incidents: trace, rollback, page. I draw that on the whiteboard next to Lisa and the trolley scan path so they see checkout does not share fate with chat. This is how I would talk as Principal AI without turning the interview into a status-code recitation.

Example. I would say: if Lisa cannot answer, the shopper still scans; 429 means wait or use a cached FAQ; 403 means we will not read staff documents; 500 means we have a bug and a trace id, not a creative model.

Level 3 — Advanced (GenAI, agents, RAG, MLOps, production)

Starts with the 25 industry questions (RAG, cost, agents, eval/safety, serving, production). Then senior RAG, agents, evals, serving, whiteboard design, and resume probes.

25 most-asked industry questions

Q221. Design a RAG pipeline that handles 10 million documents.

Answer (simple). You never dump ten million documents into a prompt. You index them once, shortlist a few dozen candidates, then read a handful of the best chunks before the model writes. The librarian metaphor is the whole design: catalogue first, then retrieve, then quote.

Answer (interview). At Retailetics I would treat a ten-million-document corpus as an ingest-and-retrieve system, not as a giant context window. Every document is ingested with tenant, ACL, source, and time metadata so later retrieval can filter before it ranks. Chunking is structure-aware rather than a naive token window, and each chunk is embedded and stored in sharded ANN indexes beside a BM25 inverted index. At query time we apply metadata filters, run hybrid BM25 plus dense retrieval, expand with multi-query if recall is weak, then rerank down to five to eight chunks. Generation must cite those chunks and abstain when evidence is thin. Upserts and deletes stay incremental, an evaluation set guards recall at k, groundedness, p95 latency, and dollars per query, and Lisa on recipes over catalogue and policy is the same pattern at store scale rather than a dump of the whole library.

Example. On Lisa and the recipes RAG built with LangChain and LlamaIndex, we never stuffed the full catalogue into the prompt; we retrieved a few recipe and policy chunks and generated from those.

Example. A shopper ACL must not retrieve staff stock-audit chunks, so tenant and permission filters run before hybrid search, not after generation.

Q222. Debug a RAG system that retrieves relevant context but generates wrong answers.

Answer (simple). If the right page came back but the sentence is wrong, search is not the only patient. You inspect packing, prompting, contradictions, and whether the model actually used the evidence. Then you log the exact window the model saw.

Answer (interview). I start by confirming retrieval quality with labelled recall at k so I am not debugging generation while the wrong passage is in the window. If retrieval is healthy, I look at prompt instructions to cite or refuse, lost-in-the-middle packing, contradictory chunks, temperature, and models that ignore context. Faithfulness evaluation on a frozen set tells me whether the writer invented a SKU or a promo that was never in the chunk. I log the exact packed window, the citations, and the refusal decision so a bad answer is replayable. On recipes RAG the typical failure is retrieving ingredients and then inventing a catalogue SKU, which we constrain to retrieved rows. Lisa fails the eval if she cites a promotion that is not in the evidence window.

Example. When recipe RAG retrieved ingredients and then invented a SKU, we constrained generation to retrieved catalogue rows instead of letting the model free-write product codes.

Example. Lisa fails the evaluation if she cites a promo that is not present in the retrieved chunk, even when the surrounding policy page was relevant.

Q223. Choose the right chunking strategy for complex legal contracts.

Answer (simple). Do not tear a clause in half. Keep numbered sections with their headings, keep definitions with the clauses that use them, and keep tables intact. Naive five-hundred-twelve-token windows destroy legal meaning.

Answer (interview). Contracts need structure-aware chunking: numbered clauses, a definitions glossary that can be joined at retrieval time, exhibits kept as units, modest overlap, and table integrity. I evaluate with questions that require a definition plus a later operative clause so a window that split them will fail. Overlap helps when a cross-reference sits on a boundary, but overlap is not a substitute for respecting headings. I avoid naive fixed token windows because they cut through shall and shall-not language. In retail the return policy behaves like a mini-contract, so the window stays attached to the Returns heading rather than mixing with shipping. The same discipline applies to membership terms that Lisa must quote accurately.

Example. A store return policy is a mini-contract, so the retrieved window stays attached to the Returns heading instead of mixing half a clause with an unrelated shipping paragraph.

Q224. Compare BM25, dense retrieval, and hybrid search for different use cases.

Answer (simple). BM25 wins when the shopper used the exact words or an identifier. Dense retrieval wins when they paraphrased. Hybrid search plus a reranker is the default because real questions mix both.

Answer (interview). BM25 is lexical and dominates on SKUs, error codes, and rare identifiers that embedding models smear. Dense retrieval captures paraphrase and intent when the shopper says something spicy for pasta instead of a product code. Hybrid retrieval with reciprocal rank fusion is my default, followed by a cross-encoder reranker, and I tune fusion weights on a labelled set rather than by folklore. I explain failure modes in the interview: BM25 misses synonyms, dense misses exact IDs, and hybrid without metadata filters still leaks the wrong tenant. On ezyCart that split is concrete, because EZY-1044 must hit BM25 while a recipe question must hit dense plus policy chunks. The labelled set decides the blend, not a vendor slide.

Example. EZY-1044 is a BM25 problem, while a shopper asking for something spicy for pasta tonight is a dense and hybrid problem that still reranks before Lisa writes.

Q225. Add a reranker and prove its impact with evaluation metrics.

Answer (simple). You shortlist cheaply, then a careful reader reorders the shortlist. You prove the reader helped with an offline test and an online canary, not with a demo of one query.

Answer (interview). I retrieve fifty to a hundred candidates with hybrid search, then a cross-encoder reranks to about five chunks for the generator. Offline I report nDCG, MRR, and recall at k on a frozen labelled set, and I keep the retriever recall in the table so a reranker cannot hide a broken index. Online I watch groundedness, latency, and cost, because a beautiful nDCG that adds two hundred milliseconds may still lose. The change ships behind a flag and a canary so we can turn it off. I never rerank ten million documents; the point of ANN plus BM25 is to feed the reranker a shortlist. On recipes RAG, fifty recipe chunks become five for the LLM, and that is the measurable step.

Example. We retrieve about fifty recipe chunks and rerank to five for the LLM. We do not rerank ten million catalogue documents, because that would miss the point of the first-stage index.

Q226. Reduce LLM inference costs by 70% without sacrificing answer quality.

Answer (simple). Stop sending easy homework to the expensive model. Cache, compress, route, and quantize, then prove the savings on a frozen exam that still measures correctness. Seventy percent is a measured dollars-per-successful-task number, not a slogan.

Answer (interview). I treat cost as a product metric: dollars per successful task on a frozen evaluation, holding correctness, groundedness, and safety constant. Levers are prompt and RAG compression, a semantic cache for near-duplicate questions, routing easy intents to small models, quantization or distillation, batching, and hard token caps. FAQs and cache hits should never touch a frontier model. Complex recipe reasoning can still go to a strong model when the eval says quality requires it. Edge vision already uses ONNX and TF Lite per frame instead of a cloud GPU, which is the same idea at the cart. I would not claim a seventy percent cut until the exam and the production meter agree, and I would not invent a dollar total that is not on the resume.

Example. FAQ traffic hits a semantic cache or a small model, while complex recipes still go to a strong model when the frozen eval says quality requires it.

Example. Cart vision already uses ONNX and TF Lite per frame under a sub-100ms budget instead of calling a cloud GPU for every camera frame.

Q227. Reduce time to first token in a production chatbot.

Answer (simple). Start talking sooner, because a blank bubble feels like an outage. Stream tokens, shorten the prefill, and do not block the first word on every tool finishing.

Answer (interview). Time to first token is dominated by prefill, queueing, and any synchronous tool work before the first byte. I stream with SSE, shorten system plus history plus chunks, reuse prefix and KV cache for a stable system prompt, and consider speculative decoding where the draft model is trustworthy. Greetings and simple FAQs can use a faster model. Tools that are not required for the first sentence run in parallel after streaming starts. I track TTFT p50 and p95 separately from total latency so a long recipe does not hide a slow first token. Lisa should stream while a promo tool is still running, which is the same lesson as not blocking a trolley UI on a slow backend call.

Example. Lisa streams the first sentence while a promo tool is still running, and we keep the system prompt stable so prefix cache actually hits.

Q228. Design a semantic caching system and track cache hit rates.

Answer (simple). Near-duplicate questions should reuse a good answer instead of paying for another generation. Hits must respect ACL, freshness, and personalization, or the cache becomes a leak.

Answer (interview). I embed a normalized query and count a hit only when similarity, ACL, and source versions all match. Cache keys include a hash of the retrieved corpus versions so a policy update invalidates answers. Metrics are hit rate, stale-hit rate, answer quality on hits versus misses, dollars saved, and latency. Personalized prices and live stock bypass the cache. Version hashes travel with the entry so a prompt or index change does not serve yesterday's sentence. On ezyCart a basket-hash cache can reuse a recipe for an identical basket, but we never cache another shopper's member price. That is the same tenancy discipline as FastAPI request isolation.

Example. A basket-hash cache can reuse a recipe for an identical cart, but we never cache another shopper's member price, because that would be both a privacy bug and a correctness bug.

Q229. Build a model router that balances cost, latency, and response quality.

Answer (simple). A receptionist sends easy tickets to a junior and hard tickets to a senior. The router is that receptionist, with fallbacks, budgets, and an evaluation that proves the split.

Answer (interview). An intent and complexity classifier routes cache, extractive lookup, a small locally served model, a mid-size model with RAG, or a frontier model. Each route has fallbacks, rate limits, and token budgets so a Bedrock throttle degrades to a vLLM Llama instead of hanging the tablet. I A/B the router on the same frozen eval rather than on vibes, which is the LLM abstraction I talk about as Lead GenAI work at Retailetics. SKU lookup should not use an LLM at all. Recipe questions can use a mid-size model plus recipes RAG. Complaints and unsafe edge cases go to a stronger model. The resume claim is the routing idea plus measured quality, not a catalogue of every vendor name.

Example. SKU lookup uses no LLM, a recipe question uses a mid-size model plus RAG, a complaint can go to a frontier model, and a Bedrock throttle falls back to a locally served Llama rather than stalling Lisa.

Q230. Build a tool-using AI agent with reliable execution.

Answer (simple). The model may ask, but your code is the adult. You validate arguments, run an allowlisted tool, and stop. Money and safety never execute from untrusted text alone.

Answer (interview). I implement the agent as a state machine in LangGraph, with CrewAI or AutoGen only when a research spike needs role play. Tools have JSON schemas, allowlists, timeouts, retries, and idempotency keys. Human-in-the-loop sits in front of payment and irreversible stock changes. Traces go to LangSmith or Langfuse so a loop is visible. MCP is a connector, not a security boundary, so authorization still lives in our gateway. Lisa may search automatically, must confirm add-to-cart, and must never pay from a sentence in a retrieved document. FastAPI and Spring Boot already treat request bodies as untrusted, and LLM tool calls get the same treatment.

Example. Lisa may search inventory automatically, must confirm add-to-cart with the shopper, and must never fire a pay tool from untrusted retrieved text.

Q231. Debug an agent stuck in an infinite tool-calling loop.

Answer (simple). If the same calculator runs forever, you cap the tries and notice the repeats. Loops are a product bug, not a personality quirk of the model. Step budgets and typed errors are how you stop an empty SKU from retrying forty times.

Answer (interview). I look for duplicate tool signatures, oscillating arguments, and empty errors that the model interprets as try again. The graph has a step budget, a force-final node, and a circuit breaker on identical calls. Traces show the repeating signature so on-call is not reading poetry. Typed errors after two failures beat a raw empty string that invites another retry. On Lisa, get_stock with an empty SKU retrying forty times is a schema and budget failure. After two typed errors the agent must ask the shopper instead of hammering inventory. That is the same discipline as bounding WebSocket retries on the cart.

Example. If get_stock retries on an empty SKU, we return a typed error after two attempts and then ask the shopper, instead of letting the agent hammer inventory forty times.

Q232. Design long-term memory for an AI assistant.

Answer (simple). This chat is short-term memory. Long-term memory is a curated notebook, not a junk drawer of every token. Users can delete it, and secrets never go in it.

Answer (interview). I separate working thread state, episodic summaries, a structured profile, and enterprise RAG. Writes are validated and permissioned, not a transcript dump. Retrieval uses recency times importance so a one-off complaint does not outweigh a stable diet preference. Users can inspect and delete memories. Secrets, card PAN, and trading parameters do not belong in chat logs. Lisa may remember a vegetarian preference and must never remember a card number. TAN Intelligence parameters live in versioned config for that independent venture, not in an assistant memory store, and TAN is not an employer.

Example. Lisa may remember a vegetarian preference and must never store a card PAN. TAN indicator parameters stay in versioned config for that independent venture, not in shopper chat logs.

Q233. Enforce valid JSON output from an LLM.

Answer (simple). Ask for a form, not an essay, and reject a broken form. One repair attempt is polite; failing closed is required when the payload moves stock or money.

Answer (interview). I use schema-constrained decoding or tool calling, then Pydantic validation at the boundary. One repair pass may retry a malformed object, then we fail closed rather than guessing. Temperature stays low for structured calls. This is the same untrusted-body stance FastAPI already applies to HTTP. add_to_cart must be an object with a SKU and a quantity greater than zero, not a sentence that happens to contain those words. Spring Boot services that execute the side effect re-validate, so a clever string never skips the schema. Constrained decoding reduces repairs but does not replace validation.

Example. add_to_cart must be JSON with a SKU and a quantity greater than zero. FastAPI already treats request bodies as untrusted, and LLM output gets the same Pydantic gate before any cart mutation.

Q234. Detect hallucinations and reduce unsupported answers.

Answer (simple). If the book is silent, say you do not know. Hallucinations are not a zero percent problem, but unsupported claims can be measured and refused.

Answer (interview). I ground answers in retrieved evidence, require citations, and run an NLI or judge check against the context window only. The model must abstain when evidence is missing. An evaluation set tracks unsupported claims, and production monitoring samples for the same failure. You will not reach zero percent, and anyone who promises that is selling. Allergen statements without a chunk are refused. Low-confidence YOLO detections do not bill a shopper, which is the vision analogue of abstention. Lisa's recipes RAG is held to the same standard: no invented ingredient that is not in the retrieved card.

Example. Lisa must not make an allergen claim without a supporting chunk, and a low-confidence YOLO detection must not bill a shopper, because both are unsupported answers with real cost.

Q235. Build an evaluation pipeline for a customer-support / shopping chatbot.

Answer (simple). Build a practice exam from real tickets and make every prompt change sit that exam. If add-to-cart fires on a missing item, the exam fails, even if the prose sounded friendly.

Answer (interview). The pipeline is a golden set covering retrieval, answer quality, tool correctness, safety, latency, and cost. It runs as a CI gate, with human spot checks and shadow traffic on production logs. LangSmith or Langfuse datasets hold the cases so traces are comparable across prompt versions. For Lisa I want on the order of a hundred questions with known SKUs and known expected tools.

The suite fails if add-to-cart runs on a missing item or if a citation is not in the window. This is how Lead GenAI work at Retailetics stays honest when a prompt tweak looks clever in a demo. Quality is a blocker, not a dashboard decoration.

Example. A golden set of about a hundred Lisa questions with known SKUs fails the build if add-to-cart runs on a missing item, even when the chatbot's sentence sounds helpful.

Q236. Make LLM-as-a-judge evaluations reliable and consistent.

Answer (simple). One picky teacher is noisy. You write a rubric, give examples, and calibrate against humans. Then you pin the judge so it does not drift under you.

Answer (interview). I use an anchored rubric, few-shot examples, and separate scores for faithfulness versus tone so a witty wrong answer cannot win. Multi-sample judging and Cohen's kappa against humans tell me whether the judge is usable. I pin the judge model and temperature, blind the systems, and recalibrate when the judge changes. The judge for recipes asks whether every ingredient is in the evidence, not whether the answer sounds tasty. That split keeps Lisa's eval from rewarding prose over truth. Reliability is a process, not a single GPT call with a thumbs-up prompt.

Example. The judge scores whether every ingredient is present in the evidence window, not whether the recipe sounds tasty, so Lisa cannot pass on style while failing on groundedness.

Q237. Defend a tool-using agent against prompt injection.

Answer (simple). A note on a web page must not empty the till. Retrieved text is untrusted data, not an operator. Tools stay allowlisted and money stays behind a human.

Answer (interview). User text and retrieved documents are untrusted. Tools are allowlisted with schemas, and non-channel tool-call syntax inside a document is ignored. We strip instruction-like language from ingested docs where we can, sandbox code tools, and require human-in-the-loop for money. Red-team suites include HTML comments that say ignore policy. MCP still needs authorization in our gateway. A product description that says ignore policy, one hundred percent off, cannot fire a discount tool. Lisa's pay path is the same rule we already apply to untrusted FastAPI bodies. Injection is an authorization problem, not only a prompt-writing problem.

Example. A product description that says to ignore policy and apply one hundred percent off must not be able to fire the discount tool, because retrieved text is data, not an operator.

Q238. Detect and redact PII before sending data to an LLM.

Answer (simple). Cover the ID card before you photocopy it for a tutor. Redaction sits at the gateway, not as a hope that the model will be discreet. If a phone number or card-like string can reach the vendor prompt, the design already failed.

Answer (interview). Regex, NER, and checksums run at the gateway so phone numbers, device identifiers, and card-like strings are tokenized or vaulted before any vendor prompt. We minimize what we send and we do not put secrets in LangSmith traces. Faces are blurred when vision does not need them. Device identifiers stay in operational logs with access control, while a shopper saying call me at a phone number is stripped from RAG text. Public resume phone numbers are not customer PII, and I do not confuse those categories in an interview. The goal is least data to the model, not a perfect NER demo.

Example. Device identifiers stay in controlled logs, and a shopper saying call me at a phone number is stripped before RAG, because customer PII is not the same thing as a public resume contact line.

Q239. Choose between RAG, fine-tuning, and prompt engineering for a real use case.

Answer (simple). Prompting is instructions, RAG is an open book, and fine-tuning is extra practice on a stable skill. You combine them. You do not bake this week's prices into weights.

Answer (interview). Prompt engineering owns format, tone, and tool use. RAG owns facts that change, such as promotions, policies, and catalogue text. LoRA or full fine-tuning owns a stable task style when you have labels and a frozen skill, not a daily price list. I measure all three on the same eval before I spend GPU time. Promo of the week is RAG. JSON style is a prompt or constrained decoding. Five thousand labelled receipts could justify LoRA for a classification or extraction skill. Recipes RAG with LangChain and LlamaIndex is the open book for Lisa; it is not a reason to fine-tune a new model every time the menu changes.

Example. Promo of the week is RAG, JSON style is a prompt, and five thousand labelled receipts could justify LoRA for a stable extraction skill, while the live catalogue still stays in retrieval.

Q240. Fine-tune a small language model using LoRA on 5,000 examples.

Answer (simple). Bolt on a small trainable adapter instead of rebuilding the engine. Five thousand examples overfit easily, so you watch a held-out set and stop early.

Answer (interview). I freeze a base such as Llama 3 or Mistral, train a LoRA adapter with a modest rank, alpha, and dropout, and keep a clean train, validation, and test split. Task metrics, not loss theater, decide early stopping. I merge or serve the adapter in vLLM, and I quantize only after quality holds. Five thousand rows overfit quickly, especially if labels are noisy. I would LoRA a JSON extractor for dishes and missing SKUs while the catalogue stays in RAG. I would not shuffle time-series labels, which is the same leakage hygiene TAN Intelligence uses on walk-forward work for that independent venture. Fine-tuning does not replace recipes RAG for changing facts.

Example. LoRA can teach a small model to emit JSON with dishes and missing SKUs, while the catalogue stays in RAG. I would not shuffle time-series labels, because that leaks the future the same way a leaked trading split would.

Q241. Serve open-source LLMs at scale using batching, KV caching, and quantization.

Answer (simple). Share the oven, remember the story so far, and use a lighter cookbook when quality allows. Edge devices are not the same runtime as a GPU server.

Answer (interview). vLLM-style serving uses continuous batching, paged KV cache, prefix cache, and tensor parallelism to raise tokens per second without a static wait for a full batch. AWQ, GPTQ, or INT8 quantization cut memory when the eval still passes. SLOs are TTFT, tokens per second, and error rate, with fallback to Bedrock or Vertex when the pool saturates. Token budgets protect cost. Edge carts do not run vLLM; they run ONNX and TF Lite under a sub-100ms vision budget. FAQs can sit on a local Llama, hard RAG can sit on a frontier model, and cart vision stays on TensorRT or TF Lite. That split is how FastAPI backends and device inference stay in their lanes.

Example. FAQ traffic can sit on vLLM Llama, hard RAG can sit on a frontier model, and cart vision stays on TensorRT or TF Lite under a sub-100ms budget rather than borrowing the LLM GPU.

Q242. Implement SSE streaming with proper handling of client disconnects.

Answer (simple). Talk while you think, and stop if they hung up so you do not burn GPU on a tablet that went to sleep. Streaming is a cancellation problem, not only a flush problem.

Answer (interview). I use text/event-stream, flush tokens as they arrive, send heartbeats, and cancel GPU work on disconnect. Last-Event-ID is optional for resume, but checkout chat usually restarts a turn cleanly. FastAPI StreamingResponse must not buffer the whole answer. WebSockets remain the IoT path for cart events, while SSE is the token path for Lisa. If a tablet sleeps, we cancel generation immediately. That is the same resource hygiene as dropping stale camera frames instead of inferring a backlog. Disconnect handling is what makes streaming cheaper than it looks in a happy-path demo.

Example. If the tablet sleeps we cancel Lisa's generation immediately. WebSockets stay the IoT path for cart events, and SSE stays the token path for the chatbot.

Q243. Trace and debug every step of a multi-agent workflow.

Answer (simple). Give every helper a flight recorder for tools, tokens, and why they stopped. If you cannot replay a loop from those traces, you cannot operate the agent in production. Correlation identifiers have to cross the API and the graph, or the recorder is theater.

Answer (interview). I combine OpenTelemetry with LangSmith or Langfuse and LangGraph checkpoints. Prompts are redacted, correlation identifiers cross FastAPI and Spring Boot, and we replay traces after incidents. Alerts fire on loops, cost spikes, and missing tool spans. Planner to retriever to add_missing_sku should be as inspectable as IoT sequence identifiers on the trolley. The discipline is the same as production WebSockets: if the event is not sequenced and logged, you will argue from memory. Tracing is how a Principal AI answer stays operational rather than theatrical.

Example. A planner that hands work to a retriever and then to add_missing_sku must carry the same correlation discipline as IoT sequence identifiers on the trolley, so a loop is replayable.

Q244. Roll back a prompt change that degrades production performance.

Answer (simple). Prompts are code. You revert them like an API, with versions, canaries, and an automatic trip back to last-good. Cowboy hot-edits in a vendor console are an incident waiting to happen.

Answer (interview). Prompt text lives in git and config with an identifier, CI eval, canary traffic, and automatic rollback on groundedness, tool-error rate, or latency regressions. Last-good is pinned. A change that says be more creative and then invents ingredients reverts by prompt id, not by folklore. This matches how we already treat FastAPI configs and ONNX model aliases. I do not hot-edit production prompts in a vendor console during an incident except as a documented emergency, and even then I snapshot the id. Quality gates block the deploy; dashboards do not replace them.

Example. When a be more creative prompt invented ingredients, we reverted by prompt identifier to the last-good version rather than trying to patch the sentence live in a vendor console.

Q245. Design a ChatGPT-style application for 1 million daily active users.

Answer (simple). A million daily users is a factory of queues, caches, routing, and graceful failure, not one Python script. Daily active users are not the same as concurrency, and GPUs are an extra bottleneck on top of ordinary web scale.

Answer (interview). I start from auth, rate limits, a conversation store, and routes for cache, small models, RAG, and agents. Model serving autoscale across vLLM or Bedrock, responses stream with SSE, and ACL, PII, and safety sit on every path. Evaluation, cost per session, and cell-based isolation keep one noisy tenant from drowning the fleet. SLOs cover TTFT, useful answers, safety, and dollars. The hot path on ezyCart is cart WebSocket events; the chat path is Lisa. Colan taught indexing, caching, and backends that ran about forty percent faster for around five hundred thousand users, and those lessons still apply, with GPUs as the new bottleneck. I do not invent a dollar cost for this design.

Example. The hot path is cart WebSocket events, the chat path is Lisa, and the Colan-scale lesson of about five hundred thousand users with faster backends still applies, except GPUs are now the extra bottleneck.

RAG at scale

Q246. How do you handle document updates and deletions in RAG?

Answer (simple). If a policy is shredded, the assistant must not quote it tomorrow. Updates and deletes are part of retrieval correctness, not a weekend reindex hobby.

Answer (interview). Ingest is idempotent. Deletes become tombstones or immediate removals with a search-time version filter so a stale vector cannot win. TTL covers promotions that expire. Chunker or embedding-model changes force a re-embed, not a hopeful overwrite of a few rows. Deletion is an evaluation case, not an ops footnote. Expired promo vectors must not retrieve, and membership price updates invalidate the semantic cache. Recipes RAG over catalogue and policy has to follow the same rule or Lisa will quote last week's deal. Incremental upserts keep ten million documents operable without a full rebuild on every edit.

Example. Expired promo vectors must not retrieve, and a membership price update must invalidate the semantic cache so Lisa cannot quote last week's deal.

Q247. Lost-in-the-middle — what is it and the fix?

Answer (simple). Models skip the boring middle of a long pile of pages. You put the best evidence first and last, you shorten the pile, and you rerank harder. Five tight chunks beat forty mediocre ones because the writer actually reads the evidence.

Answer (interview). Lost-in-the-middle means a long context window under-attends to passages that sit in the middle. Fixes are reranking so only a few chunks remain, placing the strongest evidence at the start and end, hierarchical summarization, and refusing to stuff forty mediocre chunks. Five tight chunks beat forty weak ones for Lisa. This is why I measure faithfulness against the packed window, not against the whole corpus. RAG quality is often a packing problem after retrieval already looked fine. I would rather spend budget on a reranker than on a longer context that the model will ignore.

Example. Five tight recipe chunks beat forty mediocre ones in Lisa, because a long middle-heavy window is how grounded recipes turn into invented steps.

Q248. Parent-child / hierarchical / late chunking?

Answer (simple). Retrieve a small bite so recall is sharp, then bring the parent section so the writer has enough context. That is parent-child. Late chunking embeds a whole document and then pools spans.

Answer (interview). Small child embeddings improve recall on a specific sentence, while the parent window gives the generator the surrounding clause. Hierarchical indexes store summaries for routing and leaves for quoting. Late chunking embeds the full document with overlapping spans and pools those token embeddings into chunk vectors, which can preserve context that naive splits lose. I pick the method with an eval, not with a blog post. For recipes RAG I retrieve a step and expand to the full recipe card before generating. Contracts get the same treatment so a definition child can pull the parent clause. Complexity is allowed only when recall and faithfulness both move.

Example. We retrieve a recipe step as the child and expand to the full recipe card as the parent before Lisa generates, so recall stays sharp and the writer still sees the whole method.

Q249. Hybrid fusion: RRF vs weighted score?

Answer (simple). Two ranked lists need a fair merge. Reciprocal rank fusion does not care that BM25 scores and cosine scores live on different planets. Weighted sums need calibration first.

Answer (interview). Reciprocal rank fusion is scale-free and is my default merge for BM25 and dense ranks. A weighted score mix can win after you calibrate both score distributions on a labelled set, but uncalibrated sums are a common foot-gun. After fusion I still rerank with a cross-encoder. I explain this as operations, not as a religious preference. On ezyCart, BM25 rank plus dense rank go through RRF and then a reranker before Lisa sees chunks. If a labelled SKU set says a weighted mix is better, we switch and re-eval. Fusion is a measurable component, not a one-time architecture decision.

Example. BM25 rank and dense rank merge with reciprocal rank fusion, then a cross-encoder reranks, so Lisa never sees raw uncalibrated score sums.

Q250. How do you evaluate RAG (senior)?

Answer (simple). Did we find the right page, and did we use it honestly? Those are two different exams, and you fail the build if either drops. Senior RAG evaluation is that split plus latency, cost, and permission leaks, not a single thumbs-up.

Answer (interview). I separate retrieval metrics from generation metrics. Retrieval uses recall at k, nDCG, and MRR on labelled passages. Generation uses faithfulness, answer relevance, citation accuracy, and abstain quality. Latency, cost, and permission-leak tests sit beside quality because a correct leak is still a failure. A golden set of about a hundred cases with labelled passages is the CI gate. Recipes RAG and Lisa fail the build on a recall drop or a faithfulness drop, not on a vibe. Senior evaluation is this split plus traces, not a single thumbs-up from a judge model.

Example. A golden set of about a hundred labelled passages fails CI if recall or faithfulness drops, which is how Lisa's recipes RAG stays honest when someone tweaks chunking.

Q251. RAG security beyond ACL filters?

Answer (simple). Filters are necessary and not sufficient. You still have injection in documents, secrets in traces, and a confused deputy if the assistant can call tools the user could not.

Answer (interview). Authorization happens before retrieval, indexes are per tenant when isolation requires it, and injection tests sit in CI. DLP watches prompts and traces, encryption covers indexes at rest, and audit logs show who retrieved what. A retrieved HTML comment must not become a tool call. Shopper ACL versus staff stock-audit chunks is the retail version of this review. FastAPI and

Spring Boot already enforce authn and authz on HTTP, and RAG is not allowed to bypass that with a helpful sentence. Security for RAG is a data-flow diagram, not an Elasticsearch filter checkbox.

Example. A retrieved HTML comment must not become a tool call, and a shopper ACL must never see staff stock-audit chunks even if hybrid search thinks they are relevant.

Q252. When is RAG the wrong tool?

Answer (simple). RAG is the wrong tool when you need a guaranteed calculation, a millisecond SKU lookup, or a stable classifier. Do not retrieve a novel in order to add two plus two.

Answer (interview). Deterministic functions and SQL own facts that must be exact. RAG owns language over documents. I keep cart totals as arithmetic, stock as an API, and policy explanation as RAG. Mixing those is how Lisa quotes a stale price from a chunk. At checkout, YOLO and RT-DETR decisions are model inference with a threshold, not retrieval. TAN Intelligence signals are computed from Pine and Python rules, not from a RAG over blog posts, and TAN is an independent venture rather than an employer. Knowing when not to use RAG is part of Principal AI judgment.

Example. A cart total is arithmetic and a live price comes from SQL, while a policy explanation is RAG. Lisa must not treat a week-old chunk as the bill.

Q253. Multi-index routing?

Answer (simple). First guess which library you need, then search that library. Searching every index on every question adds noise and cost. A router with an evaluation is cheaper than one giant undifferentiated pile of ten million documents.

Answer (interview). A query classifier sends allergen and policy questions to the policy index, recipe how-tos to the recipe index, and SKU identifiers to the catalogue or BM25 SKU index. Routing cuts noise, latency, and dollars. Mistakes are visible in traces and in per-index recall. I would rather add a router with an eval than build one giant undifferentiated index for ten million documents. Lisa's recipes RAG is already a distinct corpus from staff policy. FastAPI can expose the router as an ordinary service so Spring Boot callers do not reimplement it. Routing is retrieval architecture, not a chat-flavor setting.

Example. An allergen question routes to the policy index rather than a recipe blog index, so Lisa does not answer a safety question from a cooking paragraph.

Q254. Embedding drift and re-index jobs?

Answer (simple). A new embedding model makes the old map wrong, so you rebuild instead of mixing spaces. Version the embeddings, dual-write, and compare recall before you swap the alias.

Answer (interview). Embedding vectors are versioned in metadata so a query embedded with model B never searches an index built with model A. Blue-green indexes and dual-write during migration keep rollback cheap. Overnight rebuilds of a catalogue index run as jobs with recall comparison on a labelled set. I do not mix spaces and hope cosine still means something. Recipes RAG follows the same rule when we change the embedding model under LangChain or LlamaIndex. Re-index cost and backfill SLA are part of the design, not an afterthought. Drift is an operational event with a runbook.

Example. We upgrade an embedding model with a shadow index, compare recall on the labelled set, and only then swap the live alias, including for the recipes RAG corpus.

Agents, tools, and memory

Q255. When is a multi-agent system worse than one agent?

Answer (simple). Too many cooks add cost and failure points. You add agents only when specialization is real and the evaluation says the extra graph wins. Formatting JSON with three roles is the classic case where one function and a schema would have been enough.

Answer (interview). I measure task success, latency, and dollars against a single-agent or chain baseline. Extra agents are justified by genuine specialization, different tools, or different permissions, not by a framework demo. Formatting JSON with three CrewAI roles is worse than Pydantic. Lisa should not become a committee to add pasta sauce. LangGraph stays simple until evaluation proves a supervisor split helps. At Principal AI level the burden of proof is on the more complex graph. Complexity is a budget you spend, not a status symbol.

Example. Formatting JSON with three CrewAI roles is worse than validating with Pydantic, and Lisa does not need a committee of agents to add pasta sauce.

Q256. Human-in-the-loop patterns?

Answer (simple). The model may suggest, and a human taps OK on money and safety. Auto is for reversible low-risk steps. Deny is the default when the timeout expires.

Answer (interview). LangGraph interrupts, timeouts, and default-deny policies sit in front of irreversible tools. The audit log records who approved, from which device, and on which trace. Add-to-cart can be automatic after confirmation rules, while pay requires the shopper tap. I do not hide HITL behind a prompt that says please be careful. FastAPI and the Android client already know how to confirm; the agent must use that path. Human-in-the-loop is a control, not a slide. Allergen or medical edge cases abstain or escalate rather than guessing.

Example. Add-to-cart can follow an explicit shopper confirm, and pay always requires the shopper tap, with an audit of who approved on which trace.

Q257. Idempotent tool calls?

Answer (simple). Retrying a charge must not charge twice. Every side-effecting tool needs an idempotency key and a stored result so a network blip is safe to retry. Agents inherit that rule from ordinary backends; they do not get a special exception because a model is involved.

Answer (interview). Idempotency keys come from the trace, the tool name, and canonical arguments. Results are stored so a retry returns the first outcome. add_to_cart retried after a WebSocket blip uses the same key. This is ordinary backend engineering applied to agents, the same as FastAPI handlers that must survive at-least-once delivery. Exactly-once LLM side effects are a myth; idempotent tools are the design. Spring Boot IoT gateways already think this way for device events. Agents that lack keys will double-scan in production.

Example. add_to_cart retried after a WebSocket blip uses the same idempotency key so a cart line is added once, which is the same at-least-once discipline as the IoT path.

Q258. Agent memory vs RAG vs scratchpad?

Answer (simple). The scratchpad is this task's notebook, long-term memory is a curated preference store, and RAG is the company books. Do not dump the full history into any of them. Mixing those three is how a card number leaks into a prompt.

Answer (interview). LangGraph state holds structured working memory such as the basket, missing SKUs, and the current step. RAG retrieves policies and recipes. Long-term memory holds validated preferences. I summarize instead of concatenating full history, because lost-in-the-middle and cost both punish dumps. Lisa's diet preference is memory, the return policy is RAG, and the current missing pasta sauce is scratchpad. Mixing those three is how you leak a card number into a prompt. Clear types make the agent operable.

Example. State holds the basket, missing SKUs, and step; RAG holds policies and recipes; memory holds a diet preference. Lisa does not dump full chat history into any of those stores.

Q259. How do you bound agent cost?

Answer (simple). Set a maximum on steps, tokens, dollars, and wall time, then stop. Unbounded loops are a cost incident, not a clever agent. Those budgets live in the gateway and the graph, because a prompt that asks the model to be brief will be ignored.

Answer (interview). Budgets live in the graph and in the gateway, not only in a prompt that asks the model to be brief. Cheap routes run first. Kill switches disable the agent path. A recipe agent might be allowed a handful of tool calls and a few thousand generated tokens before it must finish or ask. I would not invent a platform-wide dollar savings claim. Colan's performance work taught me to meter backends; LLM tokens need the same metering. On-call pages on a cost spike because a loop at two in the morning is how bills become incidents.

Example. A recipe agent is bounded to a small number of tool calls and generated tokens so a stuck loop cannot become an unbounded inference bill.

Q260. Deterministic fallbacks for agents?

Answer (simple). If the clever helper fails, a boring function still works. Scanning a barcode must add the item even when Lisa is down. That fallback is how a trolley stays a product instead of becoming a demo that dies with the model.

Answer (interview). The LLM is an overlay on deterministic checkout, not the only path. SKU barcode scan never requires an LLM. Feature flags shed to the function path when quality or latency trips. This is how ezyCart stays a trolley people can lean on. YOLO and RT-DETR already have confidence thresholds and fallbacks; agents need the same. FastAPI can serve the deterministic add-item API while the agent graph is disabled. A Principal AI design that cannot degrade is a demo.

Example. A barcode scan still adds the item if Lisa is down, because checkout cannot depend on the agent overlay being healthy.

Q261. AutoGen vs CrewAI vs LangGraph — senior choice?

Answer (simple). Pick the simplest orchestration you can actually operate. Production wants control, state, and human-in-the-loop more than it wants a role-play demo. The library is not the differentiator once traces, evals, and security are required.

Answer (interview). LangGraph is my production default because state, interrupts, and checkpoints are explicit. CrewAI is useful for role-based research spikes. AutoGen is conversational and easy to overfit to a demo. Evaluation, traces, and security requirements are identical whichever library you pick, so the library is not the differentiator. Production Lisa would be LangGraph. I would not run all three

in one path. At Retailetics the operable graph wins over the trend. Switching cost is real, so I spike with a metric and then commit.

Example. Production Lisa is a LangGraph state machine, CrewAI stays a research spike, and we do not run AutoGen, CrewAI, and LangGraph in one production path.

Q262. How do you test agents without hitting real APIs?

Answer (simple). Fake the tools, record the answers, and assert the calls. A golden trace is a unit test for behavior, not a transcript of a lucky demo. CI should never need a live payment API to prove that pay did not fire.

Answer (interview). Contract tests check schemas, golden traces check tool sequences, and simulators return recorded inventory so CI never needs a live stock API. Property tests assert that pay never runs without confirm, and a fixture where milk is out of stock must search alternatives and must not pay. This is ordinary software testing applied to LangGraph, the same way FastAPI test clients already fake dependencies. I do not call production payment APIs from CI.

Example. A fixture where milk is out of stock must call search_alternatives and must not call pay, which we replay without touching real inventory or payment APIs.

Q263. Supervisor / router / swarm topologies?

Answer (simple). One boss is easier to debug than a circle of peers. Swarms need stronger invariants before I would put them on a trolley. A supervisor that dispatches specialists is the default until evaluation proves otherwise.

Answer (interview). A supervisor graph is the default: a planner node dispatches retriever, vision, or FAQ specialists and collects results. Router topologies are a simpler special case of that. Swarms of peer agents create loops and unclear authority unless invariants are strict. I choose topology with traces and task-success metrics, not with a conference talk. Lisa's planner dispatching retriever versus vision versus FAQ is a supervisor. Extra peers would have to beat that baseline. Operability beats novelty on the shop floor.

Example. A planner node dispatches retriever, vision, or FAQ work and collects results, which is a supervisor topology I can trace, unlike an unconstrained swarm.

Q264. What is a graph checkpoint and why?

Answer (simple). A checkpoint is a save-game of the agent so you can resume after a crash or a human pause. Without it, a shopper who walks away mid-recipe starts from zero.

Answer (interview). LangGraph checkpointers store state by thread identifier so an interrupt or a process restart can resume. PII in checkpoints is encrypted and minimized. The shopper who walks away mid-recipe resumes the same thread on the tablet. This is the conversational analogue of WebSocket session resume, not a mysterious AI feature. I do not store card PAN in checkpoints. Thread identifiers stay per cart session rather than a global dict. Checkpoints make HITL practical because the graph can wait without forgetting.

Example. If a shopper walks away mid-recipe, we resume the same LangGraph thread on the tablet instead of starting the recipe from zero.

Evals, safety, and MLOps

Q265. What is MLOps for LLM apps (not only sklearn)?

Answer (simple). You version prompts, tools, indexes, and models, then you evaluate, deploy, watch, and roll back. An APK ship is not the whole story once an assistant can change behavior without a binary.

Answer (interview). Artifacts include prompt identifiers, embedding models, chunkers, index builds, and the agent graph. Quality metrics sit beside infrastructure metrics. Canaries and rollbacks are mandatory. Merchant deploys still matter for ezyCart, and they now sit next to prompt and model eval jobs rather than replacing them. This is MLOps for GenAI as I practice it as Lead GenAI at Retailetics. Colan-era backend performance work still informs how we watch p95, but the new artifacts are prompts and indexes. If you cannot roll back a prompt, you do not have MLOps.

Example. Merchant deploys still ship APKs, and they now sit beside prompt and model evaluation jobs, because Lisa can change behavior without a new binary.

Q266. Offline vs online vs human eval?

Answer (simple). Offline is the exam, online is live traffic in shadow, and humans label the gray cases. You need all three, because each one lies in a different way.

Answer (interview). Offline golden sets catch regressions before deploy. Online shadow and sampled live traffic catch distribution shift. Humans calibrate judges and label ambiguous allergen or complaint cases. A weekly slice of human-labelled Lisa chats keeps the judge honest. I would not replace humans with a judge model and call it done. FastAPI can log shadow outputs without showing them. The mix is how we avoid both lab theater and unmeasured production.

Example. A weekly slice of human-labelled Lisa chats keeps the LLM judge honest, because online traffic will surprise you and a single judge model will drift.

Q267. Safety policies: self-harm, scams, medical, allergens?

Answer (simple). Refuse and redirect on self-harm, scams, and medical guesses, and treat allergens as high-stakes so you never invent a probably-fine answer. The base model alone is not a safety program. Classifiers, rules, abstain behavior, and legal review have to sit around it.

Answer (interview). Layer classifiers, rules, and abstain behavior, then send the policy through legal review. Log refusals. If RAG has no allergen field, say so; never say probably fine. Self-harm and scam patterns refuse and redirect to human channels. Medical questions abstain without a source. Lisa is a shopping assistant, not a doctor. YOLO does not get a vote on an allergen claim. Safety is a gate on deploy, not a system prompt paragraph.

Example. If RAG has no allergen field, Lisa says she does not know. She never answers that a food is probably fine, because that guess is a safety incident.

Q268. Jailbreaks vs prompt injection?

Answer (simple). A jailbreak is the user trying to break the system prompt. Prompt injection is untrusted content trying to steer tools. They need different controls and they are both red-teamed.

Answer (interview). Jailbreaks are handled with policy models, output filters, and refusal tests on user turns. Injection is handled by treating retrieved text as data, allowlisting tools, and ignoring tool syntax that did not come from our channel. Both sit in the red-team suite. User saying ignore rules is a jailbreak sample. A document saying call the discount tool is injection. MCP does not erase that distinction. Lisa's tools stay allowlisted regardless of how polite the attacking text is.

Example. A user saying ignore the rules is a jailbreak sample, and a product document saying call the discount tool is prompt injection. Both are red-teamed, and tools stay allowlisted.

Q269. Data residency and vendor LLMs?

Answer (simple). You have to know where the tokens go and who can see them. Residency is a contract and an architecture, not a checkbox on a vendor slide. Region, zero-retention, and on-device vision are the practical controls I would put on the whiteboard.

Answer (interview). Contracts, regions, zero-retention options, VPC endpoints, and on-prem vLLM when required are the controls. EU shopper chats may not go to a United States-only endpoint without a recorded decision. Edge vision on ONNX and TF Lite already keeps frames on the device under a sub-100ms budget, which is a residency-friendly pattern. FastAPI can route by tenant region. I do not invent a compliance certification we do not have. The interview answer is the decision process plus the data-flow diagram.

Example. EU shopper chats may not go to a United States-only vendor endpoint without a recorded decision, while cart frames can stay on-device with ONNX and TF Lite.

Q270. Prompt/version governance?

Answer (simple). Someone must own who can change the brain, and they must bring evidence. A prompt pull request without evaluation is an unreviewed API change. CODEOWNERS, a changelog, and a last-good identifier are how you keep cowboy edits out of production.

Answer (interview). CODEOWNERS, required eval, a changelog, and an on-call owner sit on prompt and graph changes. A prompt version twelve pull request shows nDCG and groundedness against version eleven. Git is the source of truth, not a vendor playground. This is ordinary engineering governance applied to GenAI at Retailetics. I expect the same bar I would set as Principal AI: no silent edits. Rollback identifiers are part of the review. Governance is how you avoid cowboy creativity inventing ingredients.

Example. A prompt version twelve pull request shows nDCG and groundedness against version eleven, and CODEOWNERS plus a required eval block a silent edit.

Q271. Drift in RAG (content drift vs embedding vs user drift)?

Answer (simple). The books change, the map changes, or the questions change. Each drift needs a different monitor, and all three will happen in a live catalogue.

Answer (interview). Content drift is new or edited documents and is watched with fresh-doc lag and retrieval of known ids. Embedding drift is a model or chunker change and needs a re-index. User drift is new phrasing and is watched with unknown-intent rate and thumbs. A new promo language that is not in the index shows up as a spike in abstains. Lisa's recipes RAG needs all three monitors because menus, embeddings, and shopper slang all move. Drift is an MLOps problem, not a one-time ingest.

Example. When new promo language is missing from the index, abstains spike, which is content drift, and Lisa should refuse rather than invent the missing deal.

Q272. Cost accounting per tenant?

Answer (simple). Each store should see its LLM bill. A viral recipe loop must not unbounded-bill the platform, so you have quotas, honest meters, and a 429 when a tenant runs away. Accounting is how finance and on-call share a number.

Answer (interview). We meter tokens, cache hits, and GPU milliseconds per tenant, then apply quotas and a 429 when a loop runs away. Pass-through plus margin is a business choice; the engineering requirement is that the meter exists. I do not invent a dollar amount for the platform. Colan-scale work with about five hundred thousand users taught the need for backend cost and performance discipline, including about forty percent faster backends, and token metering is the GenAI continuation of that. Lisa's recipe loop is the failure story I use. Accounting is how finance and on-call share a number.

Example. A viral recipe loop cannot unbounded-bill the platform, because each tenant has a token quota and the gateway returns 429 when the loop runs away.

Q273. Feature flags for models?

Answer (simple). You flip brains without a full redeploy. Flags cover prompt, temperature, reranker, and model identifier, and they combine with canaries so rollback is a switch. Each flag needs an owner and a default-safe value or the matrix becomes untestable.

Answer (interview). Flags let us turn a reranker off if the GPU pool is hot and accept the quality hit explicitly. They also pin a prompt id and a model id per tenant or cohort. Canary plus flag is how rollback becomes a switch. ezyCart already needs this for ONNX vision models on device; server LLMs get the same treatment. FastAPI reads flags at request start so a session is consistent. I do not combine flags into an untestable matrix. Each flag has an owner and a default-safe value.

Example. We can turn the reranker off if the GPU pool is hot, accept the quality hit explicitly, and keep Lisa up rather than waiting for a full redeploy.

Q274. Shadow mode?

Answer (simple). The new brain answers in the log while the old brain still talks to the shopper. You compare distributions, then you graduate the new path. Privacy rules still apply to those logs, because shadow mode is not an excuse to keep raw PII.

Answer (interview). Shadow traffic scores the candidate prompt or model without changing the user-visible answer. Privacy rules still apply to logs. A version twelve Lisa can draft on a slice of traffic before cutover. Shadow plus eval plus canary is the promotion path. FastAPI can fork a shadow span with redaction. I never skip privacy because it is only a log. Shadow mode is how you learn without gambling the floor.

Example. Version twelve can draft Lisa answers on a slice of shadow traffic before cutover, while shoppers still hear the current production prompt.

Q275. Rollback of an index?

Answer (simple). If an embed job goes bad, you point search at yesterday's index. Immutable builds and alias swaps make that a one-line change rather than a panic rebuild.

Answer (interview). Index builds are immutable, aliases swap, and we keep N generations. catalogue-live can point at build-2026-10-28 and point back if recall collapses. Recipes RAG indexes follow the same alias pattern. Rollback is tested, not hoped. This matches ONNX alias rollback on device. Ten million documents make rebuild time a reason to keep prior generations on disk. Search aliases are as important as prompt ids.

Example. The catalogue-live alias points at an immutable build, and we keep prior generations so a bad embed job is an alias swap rather than a night of unreplicable panic.

Q276. Eval for agents: success rate of trajectories?

Answer (simple). Pretty text is not success. Success is the tools doing the right thing, once, with the right arguments, and landing in the right end state. Golden trajectories make that a CI gate instead of a demo you remember fondly.

Answer (interview). I score exact tool sequence, argument match, end state, and side-effect counts. Missing pasta sauce added once is success; added twice or paid is failure. Golden traces make this CI. LangGraph spans are the evidence. This is stricter than chatbot BLEU and it should be. Lisa is an operator of cart tools, not a poet. Trajectory eval is how agent quality becomes a gate.

Example. Success is that missing pasta sauce is added once, not twice, and that pay is not called, which a golden trajectory can assert in CI.

Q277. Red teaming cadence?

Answer (simple). Schedule the attacks: injection, PII, jailbreak, cost bombs, and loops. Cadence means CI plus a slower human exercise, not a one-off scary demo.

Answer (interview). An automated suite runs in CI, and a quarterly human red team goes deeper. We track coverage so the same three prompts are not the whole program. The suite includes ignore previous instructions hidden in a product HTML comment. Lisa's discount and pay tools are always in scope. Cadence is what makes red teaming a control. I would rather show a boring coverage table than a single clever jailbreak anecdote.

Example. The automated suite includes ignore previous instructions hidden in a product HTML comment, and Lisa's discount and pay tools are always in that scope.

Q278. Guardrails vs fine-tune vs policy model?

Answer (simple). Put rules on the outside, optionally train a model to behave, and still filter the output. Layers fail closed when any layer is unsure. Fine-tuning tone does not replace allowlisted tools or an output filter that strips a leaked card number.

Answer (interview). The stack is regex and PII detection, then a policy classifier, then the base LLM, then an output filter. Fine-tuning a policy model can help tone and refusal style, but it does not replace allowlisted tools. An output filter still strips accidental card numbers if the LLM slipped. Lisa gets all layers. I do not claim a custom safety model we did not build. Layering is the honest Principal AI answer.

Example. An output filter still strips accidental card numbers even if the LLM slipped, because guardrails sit outside the model rather than trusting a system prompt.

Q279. What should block an AI deploy?

Answer (simple). Failed tests, eval regressions, secret scans, vulnerabilities, and a missing canary plan should block. Quality is a blocker, not a dashboard nice-to-have.

Answer (interview). I write the gate list: unit and contract tests, retrieval and faithfulness deltas, safety suites, secret scanning, dependency vulnerabilities, and a documented canary with rollback. A groundedness drop on the order of a few percent blocks. A copy tweak that does not touch eval may still need review if it faces shoppers. This is the same bar I would set as Principal AI at Retailetics. APK gates from mobile work still apply, and prompts join them. If it cannot roll back, it cannot ship.

Example. A groundedness regression blocks the deploy, and even a copy tweak that faces shoppers still needs review, because quality is a gate rather than a chart.

Serving, inference, and platforms

Q280. Prefill vs decode and why TTFT cares?

Answer (simple). Prefill reads the prompt; decode writes the next word. The first word waits on prefill, which is why a fat RAG prompt feels broken even when tokens per second look fine later.

Answer (interview). TTFT is mostly queue time plus prefill of system, history, and chunks. KV cache helps later tokens more than the first token of a new prompt, unless prefix cache hits a stable prefix. Long RAG prompts kill TTFT, so I cut packing before I buy more GPUs. Lisa should not send forty chunks if five will do. Tracking TTFT separately from total latency keeps this honest. Serving math starts here. A chatbot that streams slowly on the first byte has a prefill problem, not a copy problem.

Example. Cut system prompt, history, and chunks before buying more GPUs, because Lisa's first token waits on prefill of a fat window more than it waits on decode speed.

Q281. Continuous batching vs static batch?

Answer (simple). Seat new diners as seats free instead of waiting for a full table of eight. Continuous batching is how many Lisa sessions share a GPU without lockstep.

Answer (interview). vLLM-style paged attention lets new requests join a running batch. The trade-off is fairness versus utilization, and deadlines still apply so a giant prompt cannot starve interactive chat. Many Lisa sessions share a GPU this way. Static batching is simpler and slower to fill. I explain this as an operations choice with SLOs, not as trivia. FastAPI in front still needs backpressure when the batcher is full. Continuous batching is why open-source serving can hold interactive traffic.

Example. Many Lisa sessions share a GPU without waiting for lockstep, because continuous batching seats new requests as KV pages free rather than waiting for a static eight-wide batch.

Q282. Speculative decoding?

Answer (simple). A small model guesses a few tokens and the large model checks them. It helps decode speed when the draft is good, and it adds complexity when outputs are highly random.

Answer (interview). Speculative decoding is a decode-time speedup, not a prefill miracle. It pays when the draft model agrees with the target often, which is more true for FAQ-like chats than for highly creative text. Extra serving complexity has to beat a simpler route to a smaller model. I would trial it behind a flag with TTFT and tokens-per-second on the Lisa FAQ route. I would not claim a production speedup we did not measure. It is a tool, not a default.

Example. Speculative decoding is more useful on FAQ-like Lisa chats than on highly random outputs, and I would only keep it if a flagged trial moved TTFT and tokens per second.

Q283. Multi-LoRA serving?

Answer (simple). Many small adapters can sit on one base model. You still must not mix tenants' adapters, because tone is not an excuse to leak another merchant's behavior.

Answer (interview). vLLM LoRA slots trade memory for convenience. Per-merchant tone adapters can share a base while catalogue RAG stays shared and permissioned. Mixing adapters by accident is a tenancy bug. I would rather a single well-eval'd adapter than a zoo. Recipes RAG remains the source of facts; LoRA is style or a stable skill. Multi-LoRA is an infrastructure choice with an isolation story. Principal AI review asks who can enable an adapter.

Example. A per-merchant tone adapter can share a base model while catalogue RAG stays shared and permissioned, and we must not mix tenants' adapters by accident.

Q284. GPU vs CPU vs NPU placement?

Answer (simple). Big GPUs live in the cloud for LLMs, while NPU and CPU live on the cart for vision and lexical search. You profile thermals, batch size, and precision instead of mixing jobs blindly. Sending every camera frame to a cloud GPU is how you miss the sub-100ms budget.

Answer (interview). YOLO and RT-DETR run on device NPU or CPU with ONNX or TF Lite under a sub-100ms budget. BM25 is a CPU problem. LLMs sit on GPUs with batching. Thermal, batch, and precision constraints differ at the edge versus in the rack. ezyCart is this split in production. I would not send every camera frame to a cloud GPU. Placement is an SLO and a battery story, not a brand preference.

Example. YOLO runs on the cart NPU, the LLM runs on a cloud GPU, and BM25 runs on CPU, which is how ezyCart keeps vision inside a sub-100ms on-device budget.

Q285. Backpressure when GPUs are full?

Answer (simple). Keep a waiting line with a maximum length, then say 429 or 503 instead of melting. Shed load to a smaller model or a cache before you drop interactive traffic silently.

Answer (interview). Queues have timeouts. Load balancers prefer least outstanding requests. Degradation routes FAQs to cache when vLLM p95 queue wait exceeds a tight budget. Lisa should fail loudly rather than pile frames of prompts. This matches WebSocket backpressure on carts and camera latest-frame policy. FastAPI returns the 429; Spring Boot gateways can shed IoT bursts separately from chat. Backpressure is part of the SLO, not an embarrassment.

Example. If the vLLM queue wait exceeds a tight p95 budget, we route FAQs to cache only and return 429 rather than letting Lisa pile prompts until the GPU melts.

Q286. How do you design multi-region LLM?

Answer (simple). Do not put one GPU in one basement for the planet. Regions follow residency and latency, KV cache is regional, and failover is explicit. Sticky sessions help prefix cache, but they must not make a region failure undraggable.

Answer (interview). Sticky sessions help prefix cache but hurt failover; stateless routing is simpler and slightly colder on cache. Data residency can force a region even when another GPU pool is cheaper. Malaysia versus India shoppers on ezyCart need that conversation in the design, not after a lawyer emails. I do not invent a multi-region spend figure. FastAPI at the edge of each region talks to a regional model pool. Multi-region is cells plus policy, not a global in-memory dict.

Example. Malaysia versus India shoppers raise residency and latency together, so KV cache and model pools stay regional rather than assuming one GPU basement for every store.

Q287. Exactly-once LLM side effects?

Answer (simple). You usually cannot have exactly-once model calls. You make tools idempotent and you accept at-least-once delivery with an outbox and stored results. Promising exactly-once tokens from a vendor API is how you get a double charge in production.

Answer (interview). The outbox pattern, idempotency keys, and at-least-once tools are the design. Retry add_to_cart with the same key after a 502. This is backend truth that agents inherit. WebSocket blips already taught us duplicate scans if sequence identifiers are missing. I will not promise exactly-once tokens from a vendor API. Side effects live in our systems of record. The interview wants that humility plus a key.

Example. We retry add_to_cart with the same idempotency key after a 502, because tool delivery is at-least-once and a second charge would be a product incident.

Q288. Schema evolution for tool JSON?

Answer (simple). Add optional fields and do not rename in place. Version the tool, pin agents to a version, and forbid extra fields on money tools so a model cannot smuggle a discount into an unknown key. Tool schemas are public APIs for models, and they evolve like FastAPI contracts.

Answer (interview). Pydantic extra-forbid on payment and cart mutations. add_to_cart version two may add an optional notes string. Agents pin a schema version so a rolling deploy does not half-understand a payload. FastAPI and Spring Boot already version public APIs; tools are public APIs for models. I reject unknown fields on money paths. Evolution is documented in the ADR beside the graph. Compatibility is a test, not a hope.

Example. add_to_cart version two may add an optional notes string, while money tools still forbid extra fields so a model cannot smuggle a discount into an unknown key.

Q289. How do you do CI for prompts?

Answer (simple). A pull request runs the golden set and posts the numbers in the comment. CI uses fixtures and a cheap model when it can, not production keys on every commit.

Answer (interview). A pull request fails the build on a quality delta, stores traces as artifacts, and can run pytest plus a Ragas or LangSmith eval subset in GitHub Actions. Recorded fixtures keep CI deterministic, and production keys stay out of the default path. Lisa's golden set is the gate because prompt CI is how we treat text like code. Reviewers read the delta table, not only the new wording.

Example. A GitHub Action runs pytest plus a retrieval and faithfulness eval subset, and the pull request comment shows the delta so a prompt change is reviewed as code.

Q290. Canary a vision model on devices?

Answer (simple). A small cohort of carts gets the new ONNX build while the rest stay. You watch crashes, thermals, and a proxy for accuracy, and you keep an instant disable flag.

Answer (interview). Device cohorts, night-shift lighting, crash rate, thermal throttling, and an mAP proxy decide promotion. The flag disables the new YOLO or RT-DETR file instantly. Night shift is a good first cohort because lighting is harsher. Sub-100ms is still the budget. This is the edge analogue of a server model canary. I do not roll a vision model to every trolley at once. Canary is how indoor cameras stop being a fleet-wide incident.

Example. A night-shift cohort gets the new ONNX first because lighting is harsher, and an instant disable flag rolls back YOLO or RT-DETR without waiting for an app-store cycle.

Q291. Feature store vs just SQL?

Answer (simple). A feature store is a disciplined way to get the same features at train and serve. It is worth it at scale with many models, and it is overkill for one XGBoost on a single table.

Answer (interview). Promo propensity features might belong in a warehouse-centric feature store. Checkout SKU truth is a SQL and API problem, not a feature-store problem. Train/serve skew is the reason the store exists. I would not stand up a feature platform to look modern. Lisa's live price is SQL. Principal AI judgment is using the simpler truthful path. Cost of a platform has to beat a well-tested SQL path.

Example. Promo propensity features may live in a warehouse feature path, but checkout SKU and live price are SQL and API problems, not a reason to stand up a feature store.

Q292. Online vs offline features for XGBoost?

Answer (simple). Offline features come from batch tables. Online features are a low-latency lookup at request time. If the code paths differ, you have skew, and the model was trained on a fantasy.

Answer (interview). Point-in-time correctness matters. Do not train on a rolling thirty-day average that is not available live. Trading work in TAN Intelligence as an independent venture uses walk-forward hygiene for the same reason, with Pine and Python, and I would not shuffle a time split. Retail promo models need the same honesty. Document which features are online. Skew is a silent production killer. The interview wants that split named.

Example. Do not train on a rolling thirty-day average that is not available live at request time, because that is train/serve skew dressed up as a better feature.

System design, leadership, and this resume

Q293. Design an enterprise RAG assistant (whiteboard).

Answer (simple). Draw a secure librarian: who may read, which pages, which quotes, and how we grade the answers. Then walk ingest, retrieve, generate, and delete as one system.

Answer (interview). I start with users, tenants, and SLOs, then ingest, structure-aware chunking, hybrid retrieval, rerank, ACL, grounded generation, citations, evaluation, telemetry, updates, deletes, and cost. Trade-offs are named out loud: latency versus recall, shared versus per-tenant indexes, and abstain versus a helpful guess. Lisa and recipes RAG are the concrete retail instance of this whiteboard. I would not skip permissioning to look faster. The diagram is sources to indexes to retrieve to rerank to LLM to validate to telemetry. That is the Principal AI sketch I want on the board.

Example. The whiteboard path is sources, indexes, retrieve, rerank, LLM, validate, and telemetry, with ACL on retrieve and citations on generate, which is Lisa's recipes RAG drawn as an enterprise assistant.

Q294. Design a tool-using agent (whiteboard).

Answer (simple). Approved tools, stop rules, and a human for danger. The board should look like a state machine with budgets, not like a cloud of talking heads.

Answer (interview). I draw a state machine, JSON schemas, least privilege, step and token budgets, human-in-the-loop, persistence, traces, deterministic fallbacks, and idempotent tools. Search inventory can be free. Purchase requires confirmation. Lisa is that drawing on a trolley. LangGraph is the implementation I would name. FastAPI executes tools after validation. The whiteboard fails if pay can be reached from retrieved text. Operability is part of the design, not a later story.

Example. Search inventory freely, and purchase requires confirmation. That is the Lisa whiteboard: an allowlisted state machine rather than an unconstrained swarm.

Q295. Design a real-time vision pipeline (whiteboard).

Answer (simple). A real-time vision pipeline is camera, then prepare, then recognize, then emit an event. You watch frames per second and end-to-end milliseconds, because a trolley cannot queue a second of stale reality.

Answer (interview). I draw capture, preprocess, infer, postprocess, track, timestamp, backpressure, device metrics, and fallback. ezyCart takes a frame to a SKU event on WebSockets under a sub-100ms model budget with YOLO or RT-DETR on ONNX or TF Lite. Latest-frame policy drops backlog. FastAPI or Spring Boot receives the event, not a raw video warehouse. I do not invent a hardware bill. The board must show failure when confidence is low. Vision is a pipeline with SLOs, not a model file.

Example. On ezyCart a frame becomes a SKU event on a WebSocket under a sub-100ms model budget, with YOLO or RT-DETR on ONNX or TF Lite and a latest-frame drop policy.

Q296. Design WebSockets for thousands of carts.

Answer (simple). Think of many phone calls, and do not let one bad cart kill the exchange. Shard by store, send heartbeats, apply backpressure, and authenticate every device. Sequence identifiers and horizontal gateways are what make thousands of trolleys operable.

Answer (interview). We shard by store, send heartbeats, apply backpressure, choose sticky sockets versus a broker such as Kafka, support replay, and authenticate per device. Horizontal gateways sit in Spring Boot or FastAPI with partition key store_id. Sequence identifiers prevent duplicate scans. This is the ezyCart IoT path that Lisa and vision events share. Thousands of carts is a gateway and broker problem before it is an AI problem. I keep AI overlays off the critical heartbeat path. Capacity is connection count plus message rate, not a ChatGPT metaphor.

Example. The IoT gateway in Spring Boot or FastAPI shards by store_id, heartbeats each cart, and replays by sequence identifier so one bad trolley does not take down the store.

Q297. Investigate a slow API (senior).

Answer (simple). Find the slow station with traces instead of guessing. Compare to a known-good baseline, read EXPLAIN, check dependencies, reproduce under load, and change one thing at a time. If SQL is most of p95, you index before you rewrite the framework.

Answer (interview). I compare against a known good baseline, pull distributed traces, run EXPLAIN on suspects, check dependencies and resources, reproduce under load, then apply one fix. If SQL is most of p95, I index or rewrite the query before I rewrite FastAPI. Colan-era work that made backends about forty percent faster started from measurement, not from a framework swap. Lisa's chat p95 is a different budget from cart event p95. Senior debugging is a method. I narrate the method in the interview because that is the job.

Example. If SQL is most of p95 I add the index or fix the query before I rewrite FastAPI, because the slow station is in the trace, not in a guess about the framework.

Q298. Deduplicate records in Python (talk through).

Answer (simple). Remember identifiers you have seen and keep first or last according to a stated policy. Say the policy out loud, because first versus last changes a charge.

Answer (interview). An O of n set keyed by event identifier is the default. Collision policy is explicit: keep first for charges, keep last for the latest cart snapshot if that is the rule. Stable sort first if arrival order is required. Device event identifiers already need this so a duplicate scan is not charged twice. I would write the loop on the board rather than wave at pandas. Tests cover duplicates at start, middle, and end. This is intern-level code with senior-level policy.

Example. We keep a seen set of event identifiers and skip duplicates so the same scan is not charged twice, and we say whether we keep the first or the last when identifiers collide.

Q299. Bounded retry pattern?

Answer (simple). Retry a few times with jittered waits, then fail. Do not retry forever, and do not retry a non-idempotent call blindly. Classify 429 and 503 as retryable and treat a 400 validation error as a bug, not a blip.

Answer (interview). Jittered exponential backoff, honor Retry-After, cap attempts, and classify retryable errors. 429 and 503 yes; 400 validation no. Tool calls that mutate need idempotency keys. WebSocket reconnects for carts use the same pattern. Lisa's get_stock may retry; pay does not retry without a key and a human confirm. Bounded retry is an SRE habit in agent clothing. Unbounded retry is a loop incident.

Example. 429 and 503 are retryable with jittered backoff; a 400 validation error is not. Pay does not retry without an idempotency key and a confirm.

Q300. SQL index example you can write?

Answer (simple). You should be able to write CREATE INDEX and then EXPLAIN. Talk about composite order, covering indexes, and write cost so it does not sound memorized.

Answer (interview). A single-column index on customer_id is the whiteboard starter. Composite order follows the query, covering indexes trade write cost for read speed, and extras still hit the heap. I would EXPLAIN a slow orders lookup before adding a second index. Live price and stock stay in SQL even when RAG mentioned an old price. Indexing is how Colan-scale backends got faster, including the about forty percent backend improvement, not by guessing. I can write the statement and the reason.

Example. CREATE INDEX idx_orders_customer_id ON orders(customer_id) is the starter, and I follow it with EXPLAIN so we know whether the planner actually uses it.

Q301. How do you make architecture decisions?

Answer (simple). Write down requirements, options, risks, and cost, then test the fork on a metric. Architecture Decision Records turn a debate into a decision you can revisit when the evidence changes. Bake-offs beat framework wars on the shop floor.

Answer (interview). I run bake-offs on a metric, record the ADR, and revisit when evidence changes. Edge versus cloud and chain versus agent are typical forks. YOLO versus RT-DETR is decided by mean average precision and milliseconds on store video, not by a Twitter thread. Recipes RAG chose LangChain and LlamaIndex after we could retrieve and cite, not because a blog said so. Principal AI work is making those calls explicit. Revisit is allowed; amnesia is not.

Example. YOLO versus RT-DETR was decided by accuracy and milliseconds on store video, which is the same ADR habit I use for chain versus agent.

Q302. Production incident response?

Answer (simple). Stop the leak, then find the hole, tell people, and learn without blame. Metrics come before theories: error rate, latency, GPU saturation, and queue depth first. A blame-free RCA is how the next incident still gets reported.

Answer (interview). Mitigate, diagnose, recover, then write an RCA. I look at error rate, latency, GPU saturation, and queue depth first. Missing device events send me to network, broker, schema, last deploy, and replay by event identifier. Lisa incidents add prompt id, index alias, and tool error rate. Communicate early. Blame-free RCA is how a team keeps reporting. On-call for GenAI is still incident response, not prompt poetry.

Example. Missing device events send me to network, broker, schema, last deploy, and replay by event identifier before I blame the model, because most trolley incidents are still systems incidents.

Q303. Engineering disagreement?

Answer (simple). Share the goal, bring evidence, maybe run a test, and record the call. Then disagree and commit without an ego bake-off. Two retrievers on the same golden set is a disagreement I can respect; a louder anecdote is not.

Answer (interview). Two retrievers on the same golden set is a disagreement I can respect. A louder anecdote is not. ADRs capture the losing option so we can reopen with data. I have no interest in winning a framework war that does not move Lisa's faithfulness. Principal AI facilitation is getting the eval on the table. Commit after the decision so the shop floor is not a fork. Disagreement is useful when it produces a number.

Example. If two retrievers disagree, they run on the same golden set, and we record the winner in an ADR instead of turning Lisa into an ego bake-off.

Q304. How do you mentor?

Answer (simple). Explain why, then how, then let them drive. Mentoring is increasing ownership with reviews, not typing for them forever. A junior who presents the ADR learns more than one who watches me type, and the quality bar still stays with the team.

Answer (interview). I pair on slices, review in public, and hand over ADRs they can present. Trade-offs are the lesson, not a secret keystroke. A junior who presents the API design learns more than one who watches me type FastAPI. Lisa evals are a good teaching artifact because the number is shared. Mentoring at Principal level is multiplying people who can say no to unmeasured agents. I still touch the critical path so the advice is real.

Example. We pair on API design and they present the ADR, which is how ownership moves without leaving Lisa's quality bar as tribal knowledge.

Q305. How do you prioritize?

Answer (simple). I prioritize by impact, risk, urgency, and effort, and I put brakes before paint. I make the trade-off visible on a shared board and kill zombie work that does not move a shopper-facing SLO. Checkout correctness beats a cosmetic dashboard every time.

Answer (interview). Checkout correctness beats a cosmetic dashboard. A faithfulness hole beats a new agent framework. I show the queue so PMs can argue with the same board. Colan taught the value of performance and stability for about five hundred thousand users, including about seventy percent ANR reduction and about thirty percent performance improvement, which is brakes-before-paint in mobile form. ezyCart checkout is the same rule. Prioritization is a quality of a Principal AI, not a personal kanban aesthetic.

Example. Checkout correctness beats a cosmetic dashboard, and a faithfulness hole in Lisa beats introducing a new agent framework that does not yet have an eval.

Q306. Tell me about a failure (framework)?

Answer (simple). Tell a real story with situation, your part, impact, fix, and a process change. Do not invent a tragedy, do not invent a dollar loss, and do not hide your role. Interviewers are listening for ownership and for what you changed so the miss cannot repeat.

Answer (interview). I use STAR and I say what I would do differently. A genuine integration miss such as a missing sequence identifier that caused duplicate scans in staging is the right shape: concrete, bounded, and process-changing. I would not invent a dollar loss. I would not blame a teammate to look senior. The process change is the point, such as replay tests and idempotency keys. Interviewers are listening for ownership. I keep the story in systems I actually touched at Retailetics.

Example. A missed sequence identifier that caused duplicate scans in staging is a real failure shape: I own the gap, we fixed replay and keys, and I do not invent a dollar loss to make it sound larger.

Q307. How do you measure project success?

Answer (simple). Success is people getting the job done, reliably, at acceptable cost. The model existing is not success. Adoption, task completion, quality, latency, and risk have to sit on the same scoreboard as the demo.

Answer (interview). I look at adoption, task completion, quality, latency, reliability, cost, and risk. Lisa is answer quality, add-to-cart success, TTFT, and dollars per successful task. ezyCart also has scan correctness and sub-100ms vision. I do not invent a revenue figure. Principal AI success is those SLOs plus the ability to roll back. A demo without meters is a prototype. The interview should hear the meters first.

Example. Lisa's success is answer quality, add-to-cart success, time to first token, and dollars per successful task, not the fact that a model is wired up.

Q308. Why should we hire you as Principal AI?

Answer (simple). I connect models, retrieval, agents, APIs, edge vision, and applications, and I can say no to fake metrics. That mix is the job, not a single vendor badge.

Answer (interview). I am Principal AI and Lead GenAI at Retailetics, where ezyCart, Lisa, recipes RAG with LangChain and LlamaIndex, YOLO and RT-DETR, Kimera-VIO indoor navigation versus beacons, FastAPI and Spring Boot WebSockets, and ONNX and TF Lite under a sub-100ms budget all have to work together. I also carry mobile-scale lessons from about five hundred thousand users, including about seventy percent ANR reduction, about thirty percent performance improvement, and about forty percent faster backends. TAN Intelligence is an independent venture with six years of custom indicators, Telegram alerts on five-minute and fifteen-minute charts, Pine and Python, and sub-five-percent drawdown discipline, and it is not an employer. A ninety-day plan would put evals and SLOs first. I am honest about tools I have not operated at ten million documents. Hire me for AI plus systems plus delivery.

Example. The hire case is AI plus systems plus delivery on ezyCart and Lisa, with honest scope about tools I have not run at ten million documents, rather than a list of badges.

Q309. How do you answer when you have not used a tool deeply?

Answer (simple). Say what is conceptual versus hands-on, map it to a pattern you do know, and propose a one-week spike with metrics. Do not bluff. Interviewers reward a learning plan over theater, and calibrated honesty is part of Principal AI credibility.

Answer (interview). Interviewers reward a learning plan over theater. I have hybrid search in production on recipes RAG; if I have not operated a particular vendor at ten million documents, I say so and I describe the proof of concept. Mapping to ingest, retrieve, rerank, and eval keeps the conversation senior. I would rather win trust than win a namedrop. Principal AI credibility is calibrated honesty. A spike with recall at k is a plan; a smile is not.

Example. I would say I have hybrid search in production, I have not operated that particular vendor at ten million documents, and here is a one-week proof of concept with recall at k.

Q310. Prototype vs production AI?

Answer (simple). A science fair notebook is not a trolley people lean on. Production has evaluation, authorization, observability, versioning, rollback, support, and cost.

Answer (interview). A Colab RAG is not Lisa. Production is a system with SLOs. ezyCart vision in ONNX and TF Lite, WebSockets in FastAPI and Spring Boot, and recipes RAG with citations is the production shape. Prototypes skip HITL and then people are surprised. I insist on the delta list when someone wants to ship a spike. Principal AI is that insistence. Support runbooks are part of done.

Example. A Colab RAG is not Lisa, because production adds evaluation, authorization, observability, versioning, rollback, support, and cost to the notebook demo.

Q311. How do you describe impact honestly?

Answer (simple). I describe impact with a baseline, the action, the measurement, the period, and what was me versus the team. Every number on the resume should have a method, and invented millions should never appear.

Answer (interview). The numbers I will defend are indoor navigation versus beacons at about two million, sub-100ms on-device inference, TAN Intelligence sub-five-percent drawdown as an independent venture, about seventy percent ANR reduction, about thirty percent performance improvement, about five hundred thousand users, and about forty percent faster backends. I will walk the method rather than chant the figure. I will not invent a new dollar amount under pressure. Impact without a baseline is marketing. Principal AI interviews should feel like a design review of the metric. Team versus me is stated.

Example. I will walk the method behind indoor navigation versus beacons, sub-100ms vision, sub-five-percent drawdown on TAN as an independent venture, and the Colan-scale ANR, performance, user, and backend figures, and I will not invent a new dollar amount.

Q312. ezyCart in 60 seconds?

Answer (simple). ezyCart is a smart trolley with cameras, sensors, a tablet, a backend, and AI checkout. The shopper builds a basket while the edge stack watches the cart, not a cashier lane first.

Answer (interview). Edge vision with YOLO or RT-DETR on ONNX or TF Lite runs under a sub-100ms budget. Kimera-VIO supports indoor navigation versus a beacon-heavy alternative. WebSockets move events through FastAPI and Spring Boot. Android is the last mile on the trolley. Lisa and recipes RAG with LangChain and LlamaIndex help with cooking and missing SKUs. Fail-safe scanning still works if the assistant is down. That is the sixty-second tour I give as Principal AI at Retailetics. I stop before a tool dump.

Example. Cameras and sensors go to edge AI, then the app and backend, then pay, with Lisa as the assistant overlay rather than as the only way to add an item.

Q313. Indoor navigation $2M story — what they will probe?

Answer (simple). They will probe software localization versus thousands of beacons, the baseline cost, the accuracy metric, textureless failure, your role versus the team, and how the savings were calculated. Saying two million without the arithmetic is how you lose the follow-up.

Answer (interview). Kimera-VIO indoor navigation is the software path we used instead of a dense beacon install. I walk baseline cost, accuracy, failure modes such as textureless corridors, and my role versus the team's. The about two million figure is the allowed savings story versus beacons, and I will not invent adjacent dollar amounts. I will not defend a number I cannot method. This is a systems story as much as a CV story. Principal AI interviews treat it as a measurement review. Honesty about failure cases builds more trust than a round figure.

Example. I do not just say two million. I walk the beacon baseline, Kimera-VIO accuracy, textureless failure, and the arithmetic we actually used, without inventing extra dollar amounts.

Q314. TAN Intelligence — how to talk about it?

Answer (simple). TAN Intelligence is an independent venture, not a second employer. Talk about custom indicators, six years, Telegram alerts on five-minute and fifteen-minute charts, Pine and Python, and sub-five-percent drawdown with risk limits.

Answer (interview). I keep TAN off the Experience list and on the R and D story. I talk leakage, costs, walk-forward validation, and the fact that this is not investment advice. Risk limits matter more than a lucky chart. Pine and Python are the implementation. Telegram delivers the five-minute and fifteen-minute alerts. I will not invent assets under management or dollar returns. Interviewers should hear independence clearly so they do not think I stacked jobs. Six years is duration of the venture, not tenure at a company named TAN.

Example. I keep TAN Intelligence off the Experience list and on the independent R and D story, with walk-forward hygiene and risk limits rather than a claim that it was an employer.

Q315. 90-day plan if hired?

Answer (simple). The first two weeks are SLOs, evals, traces, and cost. Then you attack the riskiest quality or latency gap. Then you ship a small agent or RAG improvement behind a flag. You do not boil the ocean.

Answer (interview). I would stand up a golden set and Langfuse or LangSmith before I introduce a new agent framework. A measured win beats a platform rewrite. ezyCart and Lisa give me a concrete place to look: groundedness, TTFT, tool errors, and on-device vision SLOs. I would meet PMs and legal on allergen and PII red lines early. Ninety days is three chapters, not thirty initiatives. Principal AI plans are sequenced. Shipping a flag-guarded improvement is the proof.

Example. Stand up a golden set and tracing before a new agent framework, then ship one flagged RAG or latency win rather than boiling the ocean in ninety days.

Q316. How do you work with PMs and legal on AI?

Answer (simple). Translate risk into options with numbers: allergens, PII, hallucination, and cost. Write the red lines down and use evaluation as a shared language.

Answer (interview). Legal gets abstain on medical and allergen questions without a source. PMs get a menu of quality versus latency versus cost, not a mystery model. Red lines in writing beat a Slack vibe. Lisa is a good shared object because a wrong ingredient is understandable. I do not hide uncertainty. Principal AI work is this translation. Eval numbers are how we disagree productively.

Example. Legal gets abstain on medical and allergen questions without a source, and PMs get quality versus latency versus cost as numbers instead of a mystery model.

Q317. On-call for LLM systems?

Answer (simple). Pages go out on errors, tail latency, cost spikes, loops, and a safety filter that is down. The runbook says rollback prompt, disable tool, or shed to FAQ cache.

Answer (interview). A cost anomaly at two in the morning kills a runaway agent graph first, then we diagnose. Prompt id, index alias, GPU queue, and tool error rate are the first four charts. Lisa can be disabled while barcode scan still works. This is ordinary on-call with new dashboards. FastAPI and Spring Boot health checks still matter. I want kill switches tested in daylight. On-call without a runbook is hero culture.

Example. A cost anomaly at two in the morning kills a runaway agent graph first, then we diagnose, while barcode scan still works if Lisa is shed to off.

Q318. Capacity planning for tokens?

Answer (simple). You estimate queries per second times tokens times dollars per token times cache hit rate, then you stress the peak. Retail is seasonal, so weekend recipes are not weekday SKU lookups.

Answer (interview). Peak versus average, headroom, and a cache-hit assumption that you actually measure are the plan. I do not invent a spend target. Weekend recipe traffic on Lisa differs from weekday SKU lookups that should not use an LLM. GPU hours follow that mix. Colan-scale planning for about five hundred thousand users is the same habit with a new unit called tokens. Capacity is a spreadsheet plus a load test. Surprise bills are a planning miss.

Example. Weekend recipe traffic on Lisa is not weekday SKU lookup traffic, so token capacity follows that mix instead of a single average queries-per-second guess.

Q319. How do you keep mobile supporting skills without leading with them?

Answer (simple). Mobile is how AI reaches the floor, not the headline of a Principal AI interview. You can still answer ANR, but you lead with vision, agents, and retrieval.

Answer (interview). Kotlin, Compose, Flutter, Android, and iOS are the last mile of vision and agents on ezyCart. I do not apply as a pure Android individual contributor if the role is Principal AI unless they want both. WebSockets into the ezyCart Android client is the punchline, not RecyclerView trivia. I can still talk ANR because about seventy percent ANR reduction and about thirty percent performance improvement on a fleet of about five hundred thousand users is real supporting evidence. The headline stays GenAI and systems. Supporting skills are offered, not leading.

Example. WebSockets into the ezyCart Android client is the punchline, and I can still answer ANR from a fleet of about five hundred thousand users, but I lead with vision, RAG, and agents.

Q320. Thread safety on Android vs in the agent backend?

Answer (simple). On Android you keep heavy work off the UI thread. On the backend you do not share mutable agent state across requests. Both are isolation stories.

Answer (interview). Immutable messages, per-thread or per-session state, and locks only when needed are the rules. One LangGraph thread identifier per cart session, not a global dict. FastAPI concurrency plus a shared mutable basket is a race. Compose and coroutines on device are the UI analogue. I can answer both without turning the interview into a mobile screen. Isolation is the common language. Principal AI backends fail in races just like apps fail with ANRs.

Example. One LangGraph thread identifier per cart session, not a global dictionary, which is the backend analogue of keeping inference off the Android UI thread.

Q321. How would you introduce evals to a team that only ships APKs?

Answer (simple). Start with a few dozen labelled questions and a CI number rather than a tracing platform. Show a caught regression, such as a creative prompt that invented ingredients, then invest in LangSmith or Langfuse once the team believes the number.

Answer (interview). Thirty labelled Lisa questions in CI will teach more than a LangSmith rollout with no cases. The story of a creative prompt that invented ingredients is the conversion moment. Then we add traces. Mobile teams already understand crash-free sessions; evals are the assistant analogue. I would pair, not lecture. Small and numerical beats a strategy deck. This is change management for Principal AI.

Example. The creative prompt that invented ingredients is the story that converts an APK-only team, after thirty labelled questions in CI have given them a number to defend.

Q322. SQL vs vector vs BM25 — who owns the truth?

Answer (simple). Transactional truth lives in SQL, while language search lives in BM25 and vectors. Do not let a week-old chunk impersonate a price or a stock count. Lisa should join those sources at answer time instead of treating retrieval as the system of record.

Answer (interview). Price and stock come from SQL or an API at answer time even if RAG mentioned an old price, so Lisa quotes a live promo from SQL rather than from a chunk alone. BM25 owns identifiers, vectors own paraphrase, and recipes RAG owns prose. Mixing that ownership is how you bill the wrong amount. FastAPI can join the three sources in one handler because truth has a system of record.

Example. Lisa quotes a live promo from SQL, not from a week-old chunk alone, while BM25 owns identifiers and vectors own paraphrase over recipe prose.

Q323. Event sourcing for carts?

Answer (simple). Store what happened, such as scanned and removed, then rebuild state. Event sourcing is great for replay and harder for reads, so you mix snapshots.

Answer (interview). Replay a cart to debug a double charge, because sequence identifiers already give ezyCart a partial event log over WebSockets and snapshots keep current basket reads cheap. I would event-source the cart, not the LLM tokens, because the debug story is the value. Spring Boot consumers can rebuild state from those events when replay matters more than simple CRUD. Choose the pattern when you need forensic reconstruction of scans and removals, not because event sourcing is fashionable.

Example. We replay a cart from scanned and removed events to debug a double charge, with snapshots so the current basket read stays cheap.

Q324. Backpressure on camera frames?

Answer (simple). If inference is slow, drop frames instead of queuing ten seconds of the past. Checkout wants the latest world, and counts come from tracking, not from every frame.

Answer (interview). Latest-frame policy keeps only the newest image. Track identifiers persist across kept frames so a drop does not double count. YOLO and RT-DETR on ONNX and TF Lite stay inside sub-100ms by refusing a backlog. This is the vision twin of GPU request shedding. ezyCart cannot bill from a delayed past. Metrics include drop rate and freshness. Backpressure here is correctness, not only capacity.

Example. We keep only the last frame and persist track identifiers across kept frames, so a slow YOLO call drops stale images instead of billing from a delayed past.

Q325. How do you version ONNX models on device?

Answer (simple). Ship a signed ONNX file with a version, a compatibility flag, and a rollback path. Disk space and input shapes are part of the version, not footnotes. Silently overwriting a detector on every trolley is how a bad build becomes a fleet incident.

Answer (interview). Artifacts are signed, input shapes stay compatible, device cohorts A/B the file, and we can disable it instantly, which is the same idea as a server index alias. yolo-store-v17.onnx carries a minimum app version, and the sub-100ms budget still applies after the swap. I do not silently overwrite a model on all trolleys; rollback is a previous file plus a flag. Device MLOps is still MLOps.

Example. yolo-store-v17.onnx ships with a minimum app version, a signature, and a rollback flag, so a bad RT-DETR or YOLO build is not a fleet-wide brick.

Q326. Coordinator vs IC at Principal level?

Answer (simple). You still touch the critical path, but you also set the quality bar and unblock others. You might write the skeleton and the eval harness, not every tool.

Answer (interview). Design reviews, eval gates, mentoring, and saying no to unmeasured agents are the coordinator half. Writing the LangGraph skeleton and the harness is the IC half. I still debug a production trace. Retailetics Principal AI and Lead GenAI work is this mix on ezyCart and Lisa. I do not disappear into slideware. I do not hoard all tool implementations either. The team should be able to ship behind the bar I set.

Example. I might write the LangGraph skeleton and the eval harness rather than every tool, and I still sit in the production trace, which is the Principal mix of coordinator and individual contributor.

Q327. What is a kill switch?

Answer (simple). A kill switch is a big red button that disables the clever path. Default to deterministic checkout when AI is sick, and test the button in daylight. A switch that nobody dares to press is decoration, not a control.

Answer (interview). Per-feature flags disable Lisa globally while scanning still works, server flags disable a tool, a prompt, or a model route, and device flags disable a new ONNX file. Kill switches are on-call's first move, they are tested in daylight, and a switch that nobody dares to press is decoration. ezyCart is still a trolley after the switch. Principal AI platforms ship with those red buttons.

Example. We can disable Lisa globally while barcode scanning still works, which is the kill switch that keeps checkout alive when the agent path is sick.

Q328. Observability for hallucinations in prod?

Answer (simple). You will not label every chat. You sample, you collect user flags, you run a critic on a slice, and you watch retrieval-empty rates. Do not wait for a viral wrong-allergen story.

Answer (interview). Thumbs-down plus an unsupported-claim critic on a slice of traffic is the practical design, with retrieval-empty and abstain rates as leading indicators and heavier sampling on allergen paths. Traces must keep the evidence window, and Lisa is the system I would instrument first. Observability without sampling is a hope. I would rather catch a thin slice daily than read a viral transcript.

Example. Thumbs-down plus an unsupported-claim critic on a slice of traffic, with heavier sampling on allergen paths, is how we watch Lisa without labelling every chat.

Q329. How do you handle multilingual RAG (Tamil/EN/MS)?

Answer (simple). Use a multilingual embedding model or translate then search, and evaluate per language. Do not assume English chunking and English token costs.

Answer (interview). Tokenizer cost differs across Tamil, English, and Malay. A Tamil recipe query against an English catalogue may need translation plus hybrid retrieval. I would build per-language slices in the golden set. Lisa in Malaysian and Indian stores cannot be English-only eval. Chunkers that split on English punctuation fail on other scripts. Multilingual is an eval problem first. I do not claim a language I did not test.

Example. A Tamil recipe query against an English catalogue uses translation plus hybrid search, and the golden set must score Tamil, English, and Malay separately.

Q330. Streaming tool calls (parallel tools)?

Answer (simple). Ask for stock and allergens at once if they are independent. Do not parallel a tool that depends on the other, and always merge with timeouts.

Answer (interview). Independent reads fan out with deadlines and then merge, so get_stock and get_allergens can run together while add_to_cart waits for both. LangGraph parallel nodes need the same cancellation as SSE disconnects, and FastAPI already fans out HTTP the same way. Dependency order is part of the schema, because parallelism is a latency win only when the merge is correct. Traces must show both legs.

Example. get_stock and get_allergens can run together, and add_to_cart waits for both, because independent reads fan out while mutations stay ordered.

Q331. What is structured output / outlines / grammar decoding?

Answer (simple). Force the model to speak JSON by construction rather than by begging. Then still validate, because constrained decoding is not a security boundary.

Answer (interview). Constrained decoding with a JSON schema or a grammar reduces repairs, and Pydantic still validates while money tools stay extra-forbid. Outlines or native json_schema for tool arguments is the implementation I would name, and Lisa's add_to_cart needs that pair. Constrained decoding plus validation is defense in depth, so I do not skip validation because the grammar looked strict. Structured output is how agents stop emitting prose into APIs.

Example. Outlines or native JSON schema can constrain tool arguments, and Pydantic still validates, so Lisa cannot smuggle prose into add_to_cart.

Q332. Chain-of-thought in production?

Answer (simple). Hidden thinking can help some models, and it can also leak and cost tokens. Do not stream raw chain-of-thought to shoppers, and do not treat it as evidence of truth.

Answer (interview). Use internal reasoning if the model family benefits, show shoppers a grounded short answer, and never treat a private scratchpad as a citation. Cost and PII leakage are reasons to keep it off the tablet. Lisa cites chunks, not inner monologue. I would not enable a long visible chain on a shop floor. Production is not a reasoning contest. Groundedness eval still wins.

Example. Use chain-of-thought internally if it helps, show shoppers the grounded short answer, and never treat the hidden scratchpad as evidence in Lisa's citations.

Q333. How do you prevent cost bombs (infinite generate)?

Answer (simple). Set maximum tokens, stop sequences, step budgets, per-tenant quotas, and anomaly alerts. Hard limits live in the gateway, not only in a polite prompt.

Answer (interview). The gateway kills runaway generations on wall time and on a small per-request cap, and I will not invent a platform savings figure to dress that up. Agent graphs have step budgets, Lisa cannot generate forever after a tablet disconnects, and alerts page on-call. Prompt-only limits will be ignored by a loop, so hard limits in the gateway are what production actually means. This is the same shedding as GPU backpressure.

Example. The gateway kills runaway generations on wall time and on a small per-request cap, because a prompt that says be brief will not stop an infinite Lisa loop.

Q334. A/B testing LLM apps vs classic A/B?

Answer (simple). Outputs are random, so you need more samples and quality metrics, not only conversion. A wittier Lisa that hallucinates more is a failed experiment even if clicks rise.

Answer (interview). Holdouts, more samples, and guardrail metrics such as safety and groundedness sit beside conversion, and CUPED can help if we have covariates. I would not ship a funnier assistant that invents ingredients, because classic button A/B under-samples LLM variance. Eval plus A/B is the pair, and Principal AI experiments protect those guardrails. Statistical humility is part of the answer.

Example. Do not ship a wittier Lisa that hallucinates more even if clicks rise, because LLM experiments need quality and safety guardrails beside conversion.

Q335. Knowledge graphs vs vector RAG?

Answer (simple). Graphs store explicit relations, and vectors store fuzzy similarity. Hybrid exists, and you use graphs when the relation is the product, such as allergens or substitutes. Ten million PDFs do not automatically want a knowledge graph.

Answer (interview). Allergens and substitutes are relations I would rather store explicitly. Recipe prose can stay in vectors. A SKU substitute graph plus vector recipes is a hybrid I like. Graphs are not a magic upgrade for ten million PDFs. Lisa should consult a substitute graph when milk is out of stock. I pick the representation from the question type. Principal AI is matching structure to the domain.

Example. A SKU substitute graph handles out-of-stock milk, while recipe prose stays in vectors, which is the hybrid rather than pretending every relation is a cosine neighbor.

Q336. How do you do RAG over tables?

Answer (simple). Do not embed a CSV as a blob and hope the model still remembers the number. Preserve rows, or use tightly allowlisted text-to-SQL, and test numeric exactness. Arithmetic that RAG round-trips through an LLM is how two plus two becomes a hallucination.

Answer (interview). Row-level chunks, schema linking, or constrained SQL are the options. Nutrition arithmetic should retrieve the row, not round-trip an LLM for two plus two. I test exact numbers. Lisa quoting a calorie field from a smashed blob is a failure. Tables are why RAG is sometimes the wrong tool. Allowlists prevent a model from dropping a table. Numeric eval is mandatory.

Example. A nutrition table should retrieve the row and use the number, not round-trip an LLM for arithmetic, because embedding a CSV as a blob is how exact values disappear.

Q337. Code interpreter / sandboxed tools?

Answer (simple). Let the model run code in a jail, not on the checkout host. There is no network, there are timeouts and resource limits, and you still assume injection risk inside the sandbox. A Python jail that can hit the payment API is not a sandbox.

Answer (interview). A sandboxed Python that can hit the payment API is not a sandbox. Lisa does not get a shell on the FastAPI host. Timeouts and memory caps are mandatory. Retrieved documents can still inject into the code string, so allowlists remain. I am conservative on the trolley. Sandboxes are extra privilege, not a toy. Principal AI says no by default.

Example. Do not give sandboxed Python a path to the payment API. Lisa's code tools stay without network and with timeouts, because injection still exists inside the jail.

Q338. How do you explain attention to a hiring manager who is not ML?

Answer (simple). Attention is a highlighter for the useful words in the sentence. If they want depth, you can add the query, key, and value sentence after the highlighter lands.

Answer (interview). I start with the highlighter metaphor, then I show how it connects it to trolley in a shopper question. If they lean in, I say queries ask, keys index, and values are what we blend. I do not start with matrices unless they ask. Lisa packing five chunks is a practical follow-on: we highlight evidence rather than the whole catalogue. Communication is part of Principal AI. The manager should leave able to repeat the highlighter line.

Example. Attention connects the word it to trolley in a shopper question, which is the highlighter I use before I ever mention query, key, and value matrices.

Q339. Staff/principal system design checklist you actually follow?

Answer (simple). Clarify, write SLOs, talk scale, draw, name data, name failure, name security, name observability, name the trade-off, then validate. It is a habit, not a poster.

Answer (interview). I say the checklist out loud and then apply it to their prompt. I always name p95, ACL, eval, and one failure mode. ezyCart and Lisa are where that habit shows. I would rather miss a trendy box than miss rollback. The checklist keeps whiteboards honest. Principal AI interviews are this ritual. Validation is a golden set or a load test, not a nod.

Example. I always name p95, ACL, evaluation, and one failure mode on the board, then I apply the rest of the checklist to their actual prompt instead of reciting it as a poster.

Q340. How do you keep from gold-plating agents?

Answer (simple). Start as a function, then a chain, then an agent if evaluation says so. Complexity is a budget you revisit, not a one-way upgrade. SKU lookup staying a function while recipes earned a graph is the discipline I want on a Principal AI team.

Answer (interview). SKU lookup stayed a function, while recipes earned a graph because missing SKUs and multi-step cooking needed tools. Quarterly revisits kill unused nodes, and Lisa does not need a swarm to format JSON. I ask what metric the extra node moves, because gold-plating is a Principal AI failure mode. Simplicity that meets SLOs is the win.

Example. SKU lookup stayed a function, and recipes earned a LangGraph because missing SKUs and multi-step cooking needed tools, not because agents were fashionable.

Q341. Mobile + AI interview: what they might still ask?

Answer (simple). They may still ask ANR, coroutines, Compose, Flutter BLoC, offline, and WebSocket reconnect. Those are supporting skills, and you tie every answer back to how the model reaches the shopper.

Answer (interview). Main-thread policy, device WebSockets, and on-device TF Lite are the bridge. About seventy percent ANR reduction, about thirty percent performance improvement, and about five hundred thousand users are the supporting scale story, plus about forty percent faster backends. I still lead with ezyCart vision, Lisa, and RAG. I will not let the interview become a pure Android screen unless they hire for that. Supporting answers stay short and tied to the trolley. Principal AI is the headline. Mobile is how it ships to a hand.

Example. Main-thread policy, device WebSockets, and on-device TF Lite are the bridge answers, always tied back to how YOLO and Lisa reach the shopper rather than to RecyclerView trivia.

Q342. How would you onboard a new embedding model with 10M docs?

Answer (simple). Build a shadow index, dual retrieve, compare recall, swap the alias, and keep rollback. Re-embed cost and backfill SLA are first-class, not a footnote.

Answer (interview). Query routing during migrate can send a slice to the new space. catalogue-v2 builds overnight, shadows for a couple of days, then the alias swaps if recall holds. Recipes RAG does the same with a smaller corpus first if I want a cheaper rehearsal. I do not mix embedding spaces. Ten million documents makes job design part of the architecture. Dual-write if updates arrive during backfill. Rollback is yesterday's alias.

Example. Build catalogue-v2 overnight, shadow it for a couple of days, compare recall, and swap the alias, which is the same onboard path I would rehearse first on the smaller recipes RAG corpus.

Q343. What is a 'cell' or 'bulkhead' in an AI platform?

Answer (simple). Cells are blast-radius boxes so one noisy tenant cannot drown the GPUs. Bulkheads separate batch embed from interactive chat so a backfill cannot starve time to first token.

Answer (interview). Per-tenant queues and separate pools for batch embed versus Lisa TTFT are the practical bulkheads, and Colan-scale multi-tenant backends taught isolation that GPUs make more expensive to ignore. Embed jobs cannot starve interactive chat, and cells also help residency. I draw this next to the router because platform Principal AI work is blast radius. Shared everything is a single incident.

Example. Embed jobs cannot starve Lisa's time to first token, because batch and interactive pools are bulkheaded and noisy tenants have their own queues.

Q344. How do you handle model deprecation from a vendor?

Answer (simple). Abstract the model, dual-run, demand evaluation parity, and put the sunset date on the calendar. Never hardcode a model name in forty repositories. The router owns the identifier so Lisa does not care what the vendor named the checkpoint.

Answer (interview). One router owns the identifier. A successor to a frontier model goes behind the same interface after the golden set says parity. Lisa should not care what the vendor named the checkpoint. I start dual-run before the sunset date. Deprecation is an incident if you discover it from an error. Principal AI vendors are rented; the interface is owned. Calendar plus eval is the plan.

Example. A successor model goes behind the same router interface after eval parity, so Lisa does not care what the vendor named the checkpoint and forty repos do not hardcode it.

Q345. Security review questions you pre-empt?

Answer (simple). Where PII goes, how tools are authorized, injection, logs, residency, and the supply chain of models. Bring a diagram and do not wait to be asked.

Answer (interview). I show cart camera to event without storing raw video, versus chat to LLM with redaction. Tool allowlists, HITL on pay, and trace redaction sit on the same slide. ezyCart vision plus Lisa is a clearer diagram than a generic chatbot. I mention ONNX supply chain signing. FastAPI and Spring Boot authz is in the picture. Pre-empting review is Principal AI manners. Security is a data-flow talk.

Example. The slide shows cart camera to event without storing raw video, versus chat to the LLM with redaction, so a security review does not have to drag the data flow out of me.

Q346. How do you document an agent for the next engineer?

Answer (simple). Give the next engineer a graph picture, tool contracts, budgets, an eval, a runbook, and the known failure modes. README, ADR, and golden traces beat a tribal walkthrough. If they cannot disable a pay tool on night one, the agent was never documented.

Answer (interview). Lisa's README shows the graph and the pay-tool never-list, golden traces are executable docs, and on-call runbooks live beside them. I document kill switches so the next engineer can disable a tool on night one. Documentation is part of shipping as Lead GenAI. If it is only in my head, it is not production.

Example. Lisa's README shows the graph and the pay-tool never-list, and golden traces plus a runbook are how the next engineer can disable a tool on night one.

Q347. Final: what is production GenAI in one sentence?

Answer (simple). Production GenAI is a measured, permissioned, observable, reversible system that uses models where they earn their keep. It is not a prompt in a slide.

Answer (interview). Quality, safety, SLOs, cost, and rollback are the bar. ezyCart, Lisa, recipes RAG, edge vision with YOLO and RT-DETR, FastAPI and Spring Boot, and evals are how I have practiced that bar as Principal AI and Lead GenAI at Retailetics. TAN Intelligence stays an independent venture on the side, not an employer, and it does not replace production discipline. I would rather ship a smaller graph that we can reverse than a swarm we cannot see. That sentence is the close of a senior interview. If we cannot measure it, it is still a prototype.

Example. ezyCart, Lisa, edge vision, FastAPI and Spring Boot, and evals are the bar: a measured, permissioned, observable, reversible system, not a prompt in a slide.

End of bank — 347 questions. Rehearse out loud. If an example is not yours, skip it.